UNIVERSIDADE FEDERAL DE ALAGOAS
INSTITUTO DE COMPUTAÇÃO
CURSO DE CIÊNCIA DA COMPUTAÇÃO

ERASMO DA SILVA SÁ JUNIOR
JOSÉ RIQUELME TEIXEIRA DA SILVA
VICTOR ANDRÉ LOPES BRASILEIRO

DESENVOLVIMENTO DE UM COMPILADOR PARA A LINGUAGEM MINIPAR

2026.2: orientação a objetos, componentes de software e paralelismo real com Fork e Join em computadores diferentes

Código-fonte, programas de teste e relatório no GitHub:
https://github.com/VictorBrasileiroo/projeto_minipar_2026-2

MACEIÓ-AL
2026

ERASMO DA SILVA SÁ JUNIOR
JOSÉ RIQUELME TEIXEIRA DA SILVA
VICTOR ANDRÉ LOPES BRASILEIRO

DESENVOLVIMENTO DE UM COMPILADOR PARA A LINGUAGEM MINIPAR

2026.2: orientação a objetos, componentes de software e paralelismo real com Fork e Join em computadores diferentes

Relatório técnico apresentado à disciplina Compiladores do Curso de Bacharelado em Ciência da Computação do Instituto de Computação da Universidade Federal de Alagoas, como requisito parcial para aprovação na disciplina.

Professor: Prof. Arturo Hernández Domínguez.

Maceió-AL
2026

RESUMO

O compilador da linguagem MiniPar 2026.2 traduz programas orientados a objetos, com blocos sequenciais e paralelos, para executáveis nativos capazes de distribuir tarefas entre núcleos de um computador ou entre computadores diferentes por meio das construções `fork` e `join`. Este relatório técnico, elaborado na disciplina Compiladores do Instituto de Computação da Universidade Federal de Alagoas, documenta a especificação, a modelagem, a implementação e a validação desse compilador. A linguagem é especificada por uma tabela de símbolos, que descreve cada palavra reservada, operador e função nativa com sua forma de uso, e por uma gramática BNF detalhada produção por produção, ambas derivadas do código. A implementação, em Python, organiza-se em componentes de software: analisador léxico; analisador sintático de descida recursiva, que constrói a árvore sintática abstrata; analisador semântico com tabela de símbolos e tratamento de erros; gerador de código intermediário de três endereços; e gerador de código C++, compilado pelo GCC com otimização. Um runtime em C++ executa as tarefas em threads ou em processos trabalhadores remotos, por meio de um protocolo TCP com mensagens enquadradas e erros explícitos. O sistema é modelado em UML. As interfaces de terminal e Web herdadas foram preservadas, e uma nova Web didática exibe os tokens, a árvore, os símbolos, o código intermediário e o C++ de cada compilação real. A validação executou os onze programas de teste com oráculos independentes: dez foram aprovados e o da rede neural ficou parcial, por diferença numérica em relação à referência; as interfaces e as suítes de integração também foram aprovadas. O programa de avaliações paralelas rodou entre dois computadores, com resultado idêntico ao da execução sequencial e aceleração de 2,33 vezes, e o MiniCompilador escrito na própria linguagem gerou Assembly ARMv7 executado no simulador CPUlator. O compilador atende aos requisitos centrais do enunciado.

Palavras-chave: compiladores; MiniPar; paralelismo; fork-join; componentes de software.

ABSTRACT

The MiniPar 2026.2 compiler translates object-oriented programs, with sequential and parallel blocks, into native executables able to distribute tasks among the cores of a computer or among different computers through the `fork` and `join` constructs. This technical report, written for the Compilers course of the Institute of Computing of the Federal University of Alagoas, documents the specification, modeling, implementation and validation of this compiler. The language is specified by a symbol table, which describes each reserved word, operator and native function with its form of use, and by a BNF grammar detailed production by production, both derived from the code. The implementation, in Python, is organized into software components: a lexical analyzer; a recursive-descent parser, which builds the abstract syntax tree; a semantic analyzer with a symbol table and error handling; a three-address intermediate code generator; and a C++ code generator, compiled by GCC with optimization. A runtime written in C++ executes tasks in threads or in remote worker processes, through a TCP protocol with framed messages and explicit errors. The system is modeled in UML. The inherited terminal and Web interfaces were preserved, and a new didactic Web interface shows the tokens, tree, symbols, intermediate code and C++ of each real compilation. Validation ran the eleven test programs against independent oracles: ten passed and the neural network test is partial, due to a numeric difference from the reference; the interfaces and the integration suites also passed. The parallel grading program ran between two computers, with a result identical to the sequential execution and a speedup of 2.33, and the MiniCompiler written in MiniPar itself generated ARMv7 assembly that ran in the CPUlator simulator. The compiler meets the central requirements of the assignment.

Keywords: compilers; MiniPar; parallelism; fork-join; software components.

LISTA DE FIGURAS

Figura 1 – Diagrama de casos de uso do MiniPar 2026.2 32
Figura 2 – Diagrama de componentes do MiniPar 2026.2 35
Figura 3 – Diagrama de classes do analisador léxico 36
Figura 4 – Diagrama de classes da AST: raiz e nós de expressão 37
Figura 5 – Diagrama de classes da AST: declarações e comandos 38
Figura 6 – Diagrama de classes do analisador sintático 39
Figura 7 – Diagrama de classes do analisador semântico e da tabela de símbolos 40
Figura 8 – Diagrama de classes do gerador TAC 41
Figura 9 – Diagrama de classes do gerador C++ 42
Figura 10 – Diagrama de classes do serviço de compilação 43
Figura 11 – Diagrama de classes da CLI e da Web original 44
Figura 12 – Diagrama de classes da Web didática 45
Figura 13 – Diagrama de classes do runtime: núcleo 46
Figura 14 – Diagrama de classes do runtime: execução remota 47
Figura 15 – Diagrama de classes do MiniCompilador do Teste 11 48
Figura 16 – Sequência de compilação e execução pela Web didática 49
Figura 17 – Sequência de Fork/Join local e remoto no binário gerado 50
Figura 18 – Fases do compilador MiniPar 2026.2 53
Figura 19 – Topologia da execução distribuída 58
Figura 20 – Web original: Teste 7 compilado e executado 62
Figura 21 – Web original: erro léxico de string sem fechamento 63
Figura 22 – Web didática: tabela de *tokens* 64
Figura 23 – Web didática: AST com o nó `VarDecl(x)` selecionado 65
Figura 24 – Web didática: escopos e símbolos 66
Figura 25 – Web didática: código de três endereços 67
Figura 26 – Web didática: C++ gerado 68
Figura 27 – Web didática: compilação pelo GCC e execução 69
Figura 28 – Web didática: erro léxico (`var x = 2 @ 3;`) 70
Figura 29 – Web didática: erro sintático (`var x = 2 + ;`) 71
Figura 30 – Web didática: erro semântico (`print(y);`) 72
Figura 31 – Web didática: Teste 11, caso 1, `((2+4)*7)-(5*4)` 73
Figura 32 – CPUlator: caso 1 do Teste 11 77
Figura 33 – CPUlator: caso 2 do Teste 11 78

LISTA DE QUADROS

Quadro 1 – Organização do repositório 14
Quadro 2 – Evolução da linguagem e do compilador de 2026.1 para 2026.2 . . 19
Quadro 3 – Tabela de símbolos da linguagem MiniPar 2026.2 20
Quadro 4 – Notação da gramática 24
Quadro 5 – Tabela da gramática da MiniPar 2026.2 24
Quadro 6 – Precedência e associatividade dos operadores 28
Quadro 7 – Regras estáticas de Fork/Join 29
Quadro 8 – Tradução de Fork/Join 29
Quadro 9 – Restrições de contexto verificadas pelo analisador semântico . . . 30
Quadro 10 – Descrição dos casos de uso 33
Quadro 11 – Componentes, arquivos e interfaces 34
Quadro 12 – Componentes exigidos e onde estão modelados 51
Quadro 13 – Tecnologias e ambiente 52
Quadro 14 – Catálogo de diagnósticos por fase 55
Quadro 15 – Mensagens do protocolo do runtime 58
Quadro 16 – Erros que o `join` pode devolver 59
Quadro 17 – Estruturas de dados do MiniCompilador 60
Quadro 18 – Rotas da Web didática e seu uso na tela 63
Quadro 19 – Comparação entre as interfaces 65
Quadro 20 – Teste 11: entradas rejeitadas e mensagens 77

LISTA DE CÓDIGOS

Código 1 – Fork e Join no Teste 8 28
Código 2 – TAC gerado para os `fork` e `join` do Teste 8 55
Código 3 – Trechos do C++ gerado para o Teste 8 57
Código 4 – Compilação e execução do Teste 7 pela linha de comando 61
Código 5 – Regras léxicas: espaços e comentários 85
Código 6 – Regras léxicas: identificadores 85
Código 7 – Regras léxicas: literais 85
Código 8 – Gramática sintática consolidada da MiniPar 2026.2 85
Código 9 – Pseudocódigo do fluxo de compilação [`service._pipeline`] 88
Código 10 – Pseudocódigo do analisador léxico [`Lexer.tokenize`] 88
Código 11 – Primitivas e erro do analisador sintático 89
Código 12 – Programa e comandos [`parse`, `_parse_statement`] 89
Código 13 – Declarações, blocos e controle 90
Código 14 – Expressões por precedência [`_parse_or` a `_parse_primary`] 91
Código 15 – Estruturas da tabela de símbolos 92
Código 16 – Operações da tabela de símbolos 92
Código 17 – Entrada do analisador semântico [`analyze`] 93
Código 18 – Relato de erros [`_report`] 93
Código 19 – Inferência e compatibilidade de tipos 94
Código 20 – Validação por nó [`validate_node`] 94
Código 21 – Regras de Fork/Join [`_validate_fork`, `_validate_join`] 96
Código 22 – Tradução de Fork/Join e `par` para TAC 96
Código 23 – Programa principal do MiniCompilador 97
Código 24 – Análise léxica do MiniCompilador 97
Código 25 – Gramática de expressões do MiniCompilador 98
Código 26 – Análise sintática do MiniCompilador 98
Código 27 – Análise semântica e tratamento de erros do MiniCompilador 99
Código 28 – Tabela de símbolos do MiniCompilador 99
Código 29 – Geração da AST, do TAC e do Assembly ARMv7 99
Código 30 – Saída do Teste 1 (entrada: `+`, `2`, `3`) 101
Código 31 – Saída do Teste 2 101
Código 32 – Trecho da saída do Teste 3 101
Código 33 – Saída do Teste 4 102
Código 34 – Recomendações do Teste 5 102
Código 35 – Saída do Teste 6 (entrada: `10 -3 -40 80 70 -100`) 102
Código 36 – Início do TAC gerado para o Teste 7 102
Código 37 – Trecho da saída do Teste 8 em modo paralelo remoto 103

Código 38 – Saída do Teste 9 em modo sequencial 104
Código 39 – Trecho da saída do Teste 10 em modo paralelo (grau 3) 104
Código 40 – Saída do MiniCompilador no caso 1 do Teste 11 105
Código 41 – Início do registro de um trabalhador na máquina B (Teste 8) 106
Código 42 – Saída do coordenador na máquina A (Teste 8) 106
Código 43 – Saída do coordenador com um trabalhador indisponível 106
Código 44 – Registro do trabalhador na máquina B (demonstração do runtime) . 107
Código 45 – Programa do Teste 1 108
Código 46 – Programa do Teste 2 109
Código 47 – Programa do Teste 3 109
Código 48 – Programa do Teste 4 110
Código 49 – Programa do Teste 5 112
Código 50 – Programa do Teste 6 114
Código 51 – Programa do Teste 7 115
Código 52 – Programa do Teste 8 116
Código 53 – Programa do Teste 9 118
Código 54 – Programa do Teste 10 121
Código 55 – Programa do Teste 11 (MiniCompilador) 124

LISTA DE TABELAS

Tabela 3 – Tamanho do código por parte do sistema 53
Tabela 4 – Suítes de testes de integração 75
Tabela 5 – Resultado dos programas de teste 75
Tabela 6 – Teste 4: saídas do compilador e da referência 76
Tabela 7 – Teste 11: casos obrigatórios 77
Tabela 8 – Tempos medianos na máquina A (carga de 5 milhões, 7 medições por cenário) 79
Tabela 9 – Rodada com trabalhadores em outro computador 79
Tabela 10 – Falhas repetíveis do runtime 80

LISTA DE ABREVIATURAS E SIGLAS

ABNT	Associação Brasileira de Normas Técnicas
API	*Application Programming Interface*
ARM	*Advanced RISC Machine*
AST	*Abstract Syntax Tree* (árvore sintática abstrata)
BNF	*Backus-Naur Form*
CLI	*Command-Line Interface* (interface de terminal)
CPU	*Central Processing Unit*
GCC	*GNU Compiler Collection*
IC	Instituto de Computação
IDE	*Integrated Development Environment*
IP	*Internet Protocol*
JSON	*JavaScript Object Notation*
LL(1)	Leitura da esquerda para a direita, derivação mais à esquerda, um símbolo de *lookahead*
NBR	Norma Brasileira
TAC	*Three-Address Code* (código de três endereços)
TCP	*Transmission Control Protocol*
UC	Caso de uso
UFAL	Universidade Federal de Alagoas
UML	*Unified Modeling Language*

SUMÁRIO

1 INTRODUÇÃO . . . 13
1.1 Objetivos . . . 13
1.1.1 Objetivo geral . . . 13
1.1.2 Objetivos específicos . . . 13
1.2 Disponibilidade do código-fonte . . . 14
1.3 Organização do relatório . . . 14

2 FUNDAMENTAÇÃO TEÓRICA . . . 16
2.1 Estrutura de um compilador . . . 16
2.1.1 Análise léxica . . . 16
2.1.2 Análise sintática . . . 16
2.1.3 Análise semântica e tabela de símbolos . . . 17
2.1.4 Código intermediário e geração de código . . . 17
2.2 Paralelismo e o modelo fork-join . . . 17
2.3 Sistemas distribuídos e comunicação por mensagens . . . 18
2.4 Componentes de software e UML . . . 18

3 A LINGUAGEM MINIPAR 2026.2 . . . 19
3.1 Visão geral . . . 19
3.2 Tabela de símbolos da linguagem . . . 20
3.3 Gramática . . . 24
3.4 Precedência e associatividade . . . 27
3.5 Orientação a objetos, blocos paralelos e canais . . . 28
3.6 Tarefas com Fork e Join . . . 28
3.7 Restrições de contexto . . . 30

4 ARQUITETURA E MODELAGEM UML . . . 31
4.1 Diagrama de casos de uso . . . 31
4.2 Arquitetura de componentes . . . 34
4.3 Diagramas de classes . . . 36
4.4 Diagramas de sequência . . . 49

5 IMPLEMENTAÇÃO . . . 52
5.1 Tecnologias e ambiente . . . 52
5.2 Fluxo de compilação . . . 53
5.3 Analisador léxico . . . 53
5.4 Analisador sintático e construção da AST . . . 54
5.5 Tabela de símbolos . . . 54

5.6 **Analisador semântico e tratamento de erros** 54
5.7 **Gerador de código intermediário** 55
5.8 **Gerador de código C++ e compilação nativa** 56
5.9 **Runtime de tarefas e execução distribuída** 57
5.10 **Serviço de compilação** 59
5.11 **MiniCompilador do Teste 11** 59

6 **INTERFACES DO COMPILADOR** 61
6.1 **Interface de terminal** 61
6.2 **Interface Web original** 61
6.3 **Interface Web didática** 63

7 **TESTES E VALIDAÇÃO** 74
7.1 **Estratégia de verificação** 74
7.2 **Testes de integração dos componentes** 74
7.3 **Programas de teste** 75
7.4 **Execução em dois computadores** 78
7.5 **Desempenho e falhas** 79
7.6 **Limitações** 80

8 **CONCLUSÃO** 81

REFERÊNCIAS 82

APÊNDICES **84**

APÊNDICE A – GRAMÁTICA BNF CONSOLIDADA 85

APÊNDICE B – PSEUDOCÓDIGOS DOS COMPONENTES 88
B.1 **Fluxo de compilação e analisador léxico** 88
B.2 **Analisador sintático** 89
B.3 **Tabela de símbolos** 92
B.4 **Analisador semântico e tratamento de erros** 93
B.5 **Gerador de código intermediário** 96
B.6 **MiniCompilador do Teste 11** 97

APÊNDICE C – SAÍDAS DOS PROGRAMAS DE TESTE 101
C.1 **Programas de teste** 101
C.2 **Execução entre dois computadores** 106

APÊNDICE D – CÓDIGO-FONTE DOS PROGRAMAS DE TESTE 108

D.1 **Teste 1: calculadora cliente-servidor** 108
D.2 **Teste 2: fatorial e Fibonacci em paralelo** 109
D.3 **Teste 3: neurônio** 109
D.4 **Teste 4: rede neural para a função XOR** 110
D.5 **Teste 5: sistema de recomendação** 112
D.6 **Teste 6: Quicksort com classes** 114
D.7 **Teste 7: Quicksort iterativo** 115
D.8 **Teste 8: avaliação com quatro avaliadores** 116
D.9 **Teste 9: avaliações e fractal de Mandelbrot** 118
D.10 **Teste 10: avaliações e dois fractais** 121
D.11 **Teste 11: MiniCompilador** 124

1 INTRODUÇÃO

Um compilador traduz um programa escrito numa linguagem-fonte para outra linguagem, em geral de nível mais baixo, e informa os erros encontrados nessa tradução (Aho *et al.*, 2008). A disciplina Compiladores do Instituto de Computação (IC) da Universidade Federal de Alagoas (UFAL) usa a linguagem MiniPar para exercitar esse processo de ponta a ponta. A versão 2025.1 gerava código intermediário e Assembly ARMv7 (Rêgo; Silva; Calixto, 2025); a versão 2026.1 acrescentou orientação a objetos, geração de C++ e uma interface Web (Minipar [...], 2026). A versão 2026.2, tema deste relatório, exige paralelismo real: uma tarefa pode ser dividida em subtarefas executadas em núcleos diferentes da mesma CPU ou em computadores diferentes (Hernández Domínguez, 2026).

O enunciado de 2026.2 pede um compilador orientado a objetos, modelado com componentes de software e UML, que faça as análises léxica, sintática e semântica, mantenha uma tabela de símbolos, gere código intermediário de três endereços e código C++ e compile esse C++ com o GCC, com otimização, até um executável. Pede também onze programas de teste e três interfaces: a de terminal, a Web herdada e uma segunda Web, de caráter didático (Hernández Domínguez, 2026). Este relatório apresenta o compilador resultante: a linguagem que ele aceita, a arquitetura, a implementação de cada componente, as interfaces e os resultados da validação.

1.1 Objetivos

1.1.1 Objetivo geral

Desenvolver o compilador da linguagem MiniPar 2026.2, orientado a objetos e organizado em componentes de software, capaz de traduzir programas com blocos sequenciais, blocos paralelos e tarefas `fork`/`join` para executáveis nativos que distribuam trabalho entre núcleos de um computador ou entre computadores diferentes.

1.1.2 Objetivos específicos

Para alcançar o objetivo geral, buscou-se:

a) especificar a linguagem MiniPar 2026.2 por uma gramática BNF e por uma tabela de símbolos que correspondam ao que o compilador de fato aceita, incluindo `fork` e `join`;

b) implementar os analisadores léxico, sintático e semântico, a tabela de símbolos e o tratamento de erros, documentados por pseudocódigos;

c) gerar código intermediário de três endereços (TAC) a partir da árvore sintática abstrata;

d) gerar código C++20 que chame um runtime próprio de tarefas e compilá-lo pelo GCC;

e) executar tarefas localmente, em threads, e remotamente, em processos trabalhadores acessados por TCP, com falhas explícitas e sem travamentos;

f) preservar as interfaces de terminal e Web herdadas e criar uma Web didática que mostre cada fase de uma compilação real;

g) implementar os onze programas de teste do enunciado, inclusive o MiniCompilador do Teste 11, que gera Assembly ARMv7 executável no simulador CPUlator, e demonstrar a execução entre dois computadores;

h) modelar o sistema em UML.

1.2 Disponibilidade do código-fonte

O código-fonte do compilador, do runtime e das interfaces, os programas de teste e este relatório estão no repositório https://github.com/VictorBrasileiroo/minipar-2026-2 (Brasileiro; Sá Junior; Silva, 2026). O Quadro 1 resume a organização das pastas.

Quadro 1 — Organização do repositório

Caminho	Conteúdo
`src/`	Compilador: `lexer.py`, `parser.py`, `ast_nodes.py`, `semantic.py`, `tac.py`, `codegen.py`, `cpp_template.py` e o serviço de compilação (`service*.py`)
`main.py`, `menu.py`, `app.py`	Interface de terminal e Web original
`runtime/`	Runtime `minipar_rt` (C++20, somente cabeçalhos) e seus testes
`web_didatica/`	Web didática (servidor Flask, páginas e JavaScript) e seus testes
`tests/`	Programas de teste 1 a 7 e resultados esperados
`tests/2026_2/`	Testes 8 a 11, *benchmark* e verificadores automatizados
`docs/`	Gramática BNF, pseudocódigos, diagramas UML e evidências de execução
`legacy/minipar-2026.1/`	Versão 2026.1 da linguagem, mantida como referência

Fonte: Elaboração própria (2026).

1.3 Organização do relatório

O capítulo 2 resume os conceitos de compiladores, paralelismo e componentes de software usados no projeto. O capítulo 3 especifica a linguagem MiniPar 2026.2, com a tabela de símbolos e a gramática detalhada. O capítulo 4 traz a arquitetura e a modelagem UML. O capítulo 5 descreve a implementação de cada componente. O capítulo 6 mostra as três interfaces. O capítulo 7 relata os testes, a execução entre

dois computadores e as medições de desempenho, e o capítulo 8 conclui o relatório. Os apêndices trazem a gramática BNF consolidada, os pseudocódigos, as saídas completas e o código-fonte dos programas de teste.

A apresentação segue o manual de normalização de trabalhos acadêmicos da UFAL (Vale; Lenzi, 2025) e as normas da Associação Brasileira de Normas Técnicas (ABNT) para trabalhos acadêmicos (ABNT, 2024), citações (ABNT, 2023) e referências (ABNT, 2018).

2 FUNDAMENTAÇÃO TEÓRICA

Este capítulo resume os conceitos usados no compilador: a estrutura de um compilador e suas fases, o modelo de paralelismo *fork-join*, a comunicação entre processos em computadores diferentes e a modelagem por componentes de software.

2.1 Estrutura de um compilador

Aho *et al.* (2008) dividem o compilador em uma parte de análise, que decompõe o programa fonte e verifica sua correção, e uma parte de síntese, que constrói o programa-alvo. A análise compreende as fases léxica, sintática e semântica e produz uma representação intermediária acompanhada da tabela de símbolos; a síntese gera o código-alvo a partir dessa representação. Cooper e Torczon (2014) chamam essas partes de *front-end* e *back-end* e destacam que a representação intermediária isola uma da outra. No MiniPar 2026.2, a análise produz a árvore sintática abstrata (AST), da qual saem dois ramos independentes: o código de três endereços e o código C++, entregue ao GCC.

2.1.1 Análise léxica

A análise léxica agrupa os caracteres do programa em *tokens*, pares formados por uma classe (identificador, número, operador, palavra reservada) e pelo lexema que a originou. Espaços e comentários são descartados, e a posição de cada *token* é guardada para as mensagens de erro (Aho *et al.*, 2008). Em analisadores escritos à mão, o autômato que reconhece os padrões é codificado como um laço que decide a classe pelo caractere inicial e prefere o lexema mais longo possível (*maximal munch*), como em <= antes de < (Cooper; Torczon, 2014).

2.1.2 Análise sintática

A sintaxe de uma linguagem é descrita por uma gramática livre de contexto, normalmente na notação BNF (*Backus-Naur Form*) ou em sua forma estendida, que admite repetição e opcionalidade (Aho *et al.*, 2008). O analisador de descida recursiva implementa cada não terminal como uma função que consome os *tokens* correspondentes. Quando cada escolha entre alternativas pode ser decidida pelo próximo *token*, a gramática é LL(1) e não há retrocesso. A precedência dos operadores é codificada em níveis: cada nível chama o imediatamente superior, e a repetição à esquerda (`termo { "+" termo }`) produz associatividade à esquerda sem recursão à esquerda.

2.1.3 Análise semântica e tabela de símbolos

A AST representa a estrutura essencial do programa, sem a pontuação que só serve à leitura. A análise semântica a percorre para verificar o que a gramática não expressa: se cada nome foi declarado, se os tipos são compatíveis com os operadores, se o número de argumentos corresponde ao de parâmetros e se comandos como `break` aparecem em contexto válido. Para isso o compilador mantém a tabela de símbolos, que associa cada identificador a seus atributos (categoria, tipo, parâmetros, linha) e respeita o aninhamento de escopos: a busca começa no escopo mais interno e segue para os externos (Cooper; Torczon, 2014).

2.1.4 Código intermediário e geração de código

O código de três endereços (TAC) é uma representação linear em que cada instrução tem no máximo um operador à direita da atribuição, como `t1 = a + b`; o fluxo de controle é expresso por rótulos e desvios (`ifFalse t goto L`) (Aho *et al.*, 2008). A síntese pode produzir código de máquina, Assembly ou código-fonte de outra linguagem. Gerar C ou C++ e delegar a um compilador maduro a seleção de instruções, a alocação de registradores e as otimizações é uma técnica difundida (Cooper; Torczon, 2014). No MiniPar, o C++ gerado segue o padrão C++20 (International Organization for Standardization, 2020) e é compilado pelo GCC com `-O3` (Free Software Foundation, 2026).

2.2 Paralelismo e o modelo fork-join

O modelo *fork-join* divide uma tarefa em subtarefas independentes (*fork*) e faz a tarefa principal aguardar o término delas antes de combinar os resultados (*join*). Conway (1963) propôs essas primitivas para sistemas multiprocessados, e Lea (2000) as popularizou em Java. O enunciado do MiniPar 2026.2 adota esse modelo e exige que cada subtarefa possa ser alocada a um núcleo da CPU ou a um computador diferente (Hernández Domínguez, 2026).

A aceleração (*speedup*) é a razão entre o tempo sequencial e o paralelo, e a redução percentual de tempo é a diferença relativa entre eles:

$$S = \frac{T_{\text{seq}}}{T_{\text{par}}} \tag{1}$$

$$R = 100 \cdot \frac{T_{\text{seq}} - T_{\text{par}}}{T_{\text{seq}}} \tag{2}$$

A lei de Amdahl (1967) limita a aceleração quando só uma fração p do trabalho pode ser paralelizada entre n unidades de execução:

$$S(n) \leq \frac{1}{(1-p) + \frac{p}{n}} \tag{3}$$

Na prática, o tempo paralelo inclui custos ausentes na execução sequencial, como a criação de threads ou processos e, no caso remoto, a serialização e a transmissão pela rede. Por isso a comparação justa exige que as duas execuções façam o mesmo trabalho (seção 7.5).

2.3 Sistemas distribuídos e comunicação por mensagens

Tanenbaum e Van Steen (2007) definem um sistema distribuído como um conjunto de computadores independentes que se apresenta ao usuário como um sistema único. Os processos não compartilham memória e se comunicam por mensagens. O protocolo TCP oferece um fluxo confiável e ordenado de bytes, mas não preserva fronteiras de mensagens; por isso, protocolos de aplicação enquadram cada mensagem, por exemplo com um prefixo que indica seu tamanho (Coulouris *et al.*, 2013). Esses autores destacam também a falha parcial: um componente remoto pode parar enquanto os demais continuam. A resposta usual é combinar limites de tempo (*timeouts*) com a detecção de conexões encerradas e tornar cada falha um erro explícito, abordagem seguida pelo runtime do MiniPar (seção 5.9).

2.4 Componentes de software e UML

Szyperski (2002) define componente de software como uma unidade de composição com interfaces especificadas e dependências de contexto explícitas, que pode ser implantada de forma independente. Sommerville (2011) apresenta a engenharia baseada em componentes como a montagem do sistema pela composição de componentes que se comunicam por interfaces bem definidas. A UML, mantida pelo Object Management Group (2017), é a linguagem-padrão de modelagem de sistemas orientados a objetos. Este relatório usa quatro de seus diagramas (Booch; Rumbaugh; Jacobson, 2006): o de casos de uso, o de componentes, o de classes e o de sequência.

3 A LINGUAGEM MINIPAR 2026.2

Este capítulo especifica a linguagem aceita pelo compilador. A seção 3.2 apresenta a tabela de símbolos da linguagem, com cada palavra reservada, operador, delimitador e função nativa e a forma como cada um é usado; a seção 3.3 apresenta a gramática, produção por produção, com a montagem de cada argumento e o nó da árvore sintática que ela produz. As duas tabelas foram derivadas do código do compilador (`src/lexer.py` e `src/parser.py`): onde um documento anterior e o código divergiam, prevaleceu o código.

3.1 Visão geral

Um programa MiniPar é uma sequência de comandos executados em ordem. A linguagem tem tipagem dinâmica com inferência estática conservadora: variáveis são declaradas com `var` ou com um nome de tipo, e o analisador semântico infere os tipos sempre que possível. Há funções, classes com atributos, métodos e herança simples, listas e matrizes, entrada e saída, comentários de linha, blocos sequenciais (`seq`) e paralelos (`par`), canais de comunicação e, a partir da versão 2026.2, tarefas com `fork` e `join`. O Quadro 2 resume o que a versão 2026.2 acrescentou em relação à versão 2026.1.

Quadro 2 — Evolução da linguagem e do compilador de 2026.1 para 2026.2

Aspecto	Versão 2026.1	Versão 2026.2
Paralelismo	Bloco `par` com threads no mesmo processo	Mantido; novas tarefas `fork f(...) [on destino]` e `join(h [, timeout])`, locais ou em outro computador
Runtime	Inexistente	Runtime `minipar_rt` em C++20: executor de tarefas, protocolo TCP e trabalhador no mesmo binário
Código de três endereços	Texto fixo para o Quicksort iterativo	Gerado da AST para qualquer programa, inclusive `fork`, `join`, `par` e `for-in`
Strings	Tipagem e operações incompletas	`substr`, `to_string`, concatenação e comparação
Saída concorrente	`print` não atômico entre threads	`print` atômico
Interfaces	Terminal e Web	Mantidas; nova Web didática com as fases da compilação
Diagnósticos	Mensagens impressas	Catálogo de códigos por fase, com linha e coluna

Fonte: Elaboração própria (2026).

3.2 Tabela de símbolos da linguagem

O analisador léxico percorre o texto uma vez e classifica cada lexema pelo primeiro caractere. Espaços, tabulações e quebras de linha separam *tokens* e são descartados; o comentário começa com # e vai até o fim da linha, sem comentário de bloco. Um identificador começa por letra ou sublinhado e continua com letras, dígitos ou sublinhados; como as funções de classificação de caracteres do Python aceitam Unicode, `ação` é um identificador válido. Quando o lexema de um identificador consta da lista de palavras reservadas, ele vira o *token* da palavra. Números são sequências de dígitos com parte decimal opcional (`1.` é aceito; `.5` e `1e3` não). Strings ficam entre aspas duplas, podem ocupar mais de uma linha e aceitam os escapes `\n`, `\t`, `\"` e `\\`. Entre os operadores, vale o lexema mais longo (<= antes de <).

O Quadro 3 é a tabela de símbolos da linguagem: para cada símbolo terminal, informa o *token* produzido, o significado e a forma de uso, isto é, como o símbolo é montado com seus argumentos. Os nomes entre < > são não terminais da gramática (seção 3.3); `[ ]` indica parte opcional e `...` indica repetição. A tabela inclui as funções nativas, que não são palavras reservadas no léxico (chegam ao analisador sintático como `IDENTIFIER`), mas têm nome reservado pelo analisador semântico. Não se deve confundir esta tabela, que descreve o vocabulário da linguagem, com a tabela de símbolos que o compilador constrói para cada programa, descrita na seção 5.5.

Quadro 3 — Tabela de símbolos da linguagem MiniPar 2026.2

Símbolo	Token	Significado	Forma de uso (montagem dos argumentos)	Exemplo
Palavras reservadas: declarações e objetos				
`var`	`VAR`	Declara variável ou atributo de classe	`var <nome> [= <expressao>];` — `<nome>` é um identificador novo no escopo; o valor inicial é qualquer expressão. Também abre o `for` de iteração: `for (var <x> in <lista>)`	`var total = 0;`
`func`	`FUNC`	Declara função global, local ou método	`func <nome>([<tipo>] <p1>, ..., [<tipo>] <pN>) { <comandos> }` — parâmetros são identificadores separados por vírgula, cada um com tipo opcional (lido e descartado); o corpo exige chaves	`func soma(int a, b) { return a + b; }`
`return`	`RETURN`	Encerra a função e devolve um valor	`return [<expressao>] [;]` — a expressão é omitida quando o próximo *token* é `;`	`return n * f;`
`class`	`CLASS`	Declara classe	`class <Nome> [extends <Super>] { <membros> }` — cada membro é `var <attr> [= <expr>];` ou `func <metodo>(...) {...}`	`class Pilha { var itens = []; }`
`extends`	`EXTENDS`	Herança simples	Só no cabeçalho da classe: `extends <Super>`, com `<Super>` um identificador de classe declarada	`class B extends A {}`

(continua)

(continuação)

Símbolo	Token	Significado	Forma de uso (montagem dos argumentos)	Exemplo
`new`	`NEW`	Cria objeto	`new <Classe>(<a1>, ..., <aN>)` — a lista de argumentos é de expressões e pode ser vazia; os parênteses são obrigatórios	`var q = new Quicksort();`
`this`	`THIS`	Objeto corrente, dentro de método	`this`, `this.<attr>` ou `this.<metodo>(<args>)`	`this.v[i] = x;`
Palavras reservadas: nomes de tipo				
`number`, `int`, `string`, `bool`, `list`, `dict`, `void`	`TYPE` (o lexema guarda o nome)	Tipo declarado	No lugar de `var`: `<tipo> <nome> [= <expr>];`; antes de um parâmetro: `func f(<tipo> p)`; no inicializador do `for` clássico. `int` e `number` são compatíveis entre si	`string s = "oi";`
Palavras reservadas: controle de fluxo				
`if`	`IF`	Desvio condicional	`if (<condicao>) <comando>` — a condição é uma expressão entre parênteses; o comando pode ser um bloco	`if (x > 0) print(x);`
`else`	`ELSE`	Ramo alternativo do `if`	`... else <comando>` — casa com o `if` mais próximo; `else if` é `else` seguido de outro `if`	`else { x = 0; }`
`while`	`WHILE`	Laço com teste no início; fecha o `do`	`while (<condicao>) <comando>`	`while (i < n) { i = i + 1; }`
`do`	`DO`	Laço com teste no fim	`do <comando> while (<condicao>) [;]`	`do { k = k - 1; } while (k > 0);`
`for`	`FOR`	Laço clássico ou de iteração	`for ([<init>]; [<cond>]; [<passo>]) <comando>` ou `for (var <x> in <expr>) <comando>` — `<init>` é declaração ou atribuição; `<passo>` é atribuição ou expressão	`for (var i = 0; i < 3; i = i + 1) ...`
`in`	`IN`	Separa variável e lista no `for` de iteração	`var <x> in <expressao>` — a expressão deve produzir uma lista; `var` é obrigatório	`for (var h in hs) ...`
`break`	`BREAK`	Sai do laço mais interno	`break [;]` — só dentro de laço	`break;`
`continue`	`CONTINUE`	Vai para a próxima iteração	`continue [;]` — só dentro de laço	`continue;`
Palavras reservadas: blocos e paralelismo				
`seq`	`SEQ`	Bloco sequencial	`[seq] { <comando1> ... <comandoN> }` — os comandos rodam em ordem; `seq` é opcional, pois um bloco entre chaves já é sequencial	`seq { a(); b(); }`
`par`	`PAR`	Bloco paralelo	`par { <comando1> ... <comandoN> }` — cada comando direto é um ramo, executado numa thread; o bloco termina quando todos os ramos terminam	`par { fat(5); fib(8); }`
`fork`	`FORK`	Dispara tarefa e devolve um *handle*	`fork <f>(<a1>, ..., <aN>) [on <destino>]` — `<f>` é o nome de uma `func` global; cada `<ai>` é uma expressão avaliada e copiada por valor; o resultado (tipo `task`) deve ser guardado	`var h = fork soma(1, 2);`

(continua)

(continuação)

Símbolo	Token	Significado	Forma de uso (montagem dos argumentos)	Exemplo
`on`	`ON`	Destino da tarefa	`on <expressao>` — a expressão deve valer, em execução, `"local"` (thread do processo) ou `"host:porta"` (trabalhador remoto); sem `on`, o destino é local	`fork f(x) on "local"`
`join`	`JOIN`	Espera a tarefa e devolve seu resultado	`join(<handle> [, <timeout>])` — `<handle>` é uma expressão de tipo `task`; `<timeout>`, opcional, é numérico, em milissegundos	`var r = join(h, 5000);`
Palavras reservadas: entrada, saída, canais e coleções				
`print`	`PRINT`	Escreve uma linha na saída	`print(<e1>, ..., <eN>);` — os valores são separados por espaço, e a linha é escrita de forma atômica; a lista pode ser vazia	`print("x =", x);`
`input`	`INPUT`	Lê uma linha da entrada (string)	`input([<prompt>])` — o *prompt*, opcional, é uma expressão impressa antes da leitura; é expressão, não comando	`var s = input();`
`c_channel`	`C_CHANNEL`	Cria canal de comunicação por *socket*	`c_channel(<ip>, <porta>)` — duas expressões: o endereço IP (string) e a porta (número)	`var c = c_channel(ip, 9000);`
`send`	`SEND`	Envia mensagem pelo canal	`send(<canal>, <mensagem>);` — é comando; exige `;`	`send(c, op);`
`receive`	`RECEIVE`	Recebe mensagem (string) do canal	`receive(<canal>)` — é expressão; deve ser usada como valor	`var m = receive(c);`
`matrix`	`MATRIX`	Cria matriz preenchida	`matrix(<linhas>, <colunas>, <valor_inicial>)` — três expressões; as duas primeiras numéricas	`var m = matrix(3, 3, 0);`
`true`, `false`	`BOOL_LITERAL`	Constantes lógicas	Usadas como expressão primária	`var ok = true;`
Identificadores e literais				
identificador	`IDENTIFIER`	Nome de variável, função, classe, atributo ou método	`(letra | _) {letra | dígito | _}`, desde que não seja palavra reservada	`nota_final`
número	`NUMBER`	Constante numérica	`dígitos [. dígitos]` — sem sinal (o `-` é operador unário) e sem notação científica	`42`, `3.14`
string	`STRING_LITERAL`	Constante de texto	`"` caracteres `"`, com os escapes `\n`, `\t`, `\"` e `\\`	`"linha\n"`
Operadores				
`+`	`PLUS`	Soma, concatenação de strings ou sinal unário	`<e1> + <e2>` ou `+<e>` — com uma string, o outro operando deve ser string (usa-se `to_string`)	`"n=" + to_string(n)`
`-`	`MINUS`	Subtração ou negação	`<e1> - <e2>` ou `-<e>` — operandos numéricos	`-x + 1`
`*`, `/`, `%`	`STAR`, `SLASH`, `MOD`	Multiplicação, divisão e resto	`<e1> op <e2>` — operandos numéricos	`i % 2`
`=`	`ASSIGN`	Atribuição (não é operador de expressão)	`<alvo> = <expressao>;` — `<alvo>` é variável, `<obj>.<attr>` ou `<lista>[<indice>]`	`v[i] = 0;`
`==`, `!=`	`EQ`, `NEQ`	Igualdade e diferença	`<e1> == <e2>` — produz `bool`	`op == "+"`
`<`, `>`, `<=`, `>=`	`LT`, `GT`, `LTE`, `GTE`	Comparações	`<e1> < <e2>` — produz `bool`; mesmo nível de precedência da igualdade	`i <= n`

(continua)

(continuação)

Símbolo	Token	Significado	Forma de uso (montagem dos argumentos)	Exemplo
`&&`, `||`	`AND`, `OR`	E e OU lógicos	`<e1> && <e2>` — `&` e `|` isolados são inválidos	`a > 0 && b > 0`
`!`	`NOT`	Negação lógica	`!<e>`	`!achou`
Delimitadores				
`( )`	`LPAREN`, `RPAREN`	Agrupamento, condições e listas de argumentos	`(<expr>)`, `if (<cond>)`, `f(<args>)`	`(a + b) * c`
`{ }`	`LBRACE`, `RBRACE`	Blocos e corpos	`{ <comandos> }` — corpo de função, classe, `seq` e `par`	`{ x = 1; }`
`[ ]`	`LBRACKET`, `RBRACKET`	Lista literal e indexação	`[<e1>, ..., <eN>]` ou `<lista>[<indice>]`	`[1, 2, 3]`, `v[0]`
`.`	`DOT`	Acesso a membro	`<obj>.<attr>` ou `<obj>.<metodo>(<args>)`	`p.nome`
`;`	`SEMICOLON`	Fim de comando simples	Obrigatório após declaração, atribuição, `print`, `send` e chamada; opcional após `return`, `break`, `continue` e `do-while`	`x = 1;`
`,`	`COMMA`	Separador de argumentos e elementos	`<e1>, <e2>`	`f(a, b)`
Funções nativas e métodos de lista (identificadores predefinidos)				
`len`	`IDENTIFIER`	Tamanho de lista, matriz ou string	`len(<colecao>)` — um argumento; devolve número	`len(v)`
`range`	`IDENTIFIER`	Lista de inteiros	`range(<fim>)` ou `range(<inicio>, <fim>)` — intervalo semiaberto	`range(0, 4)`
`substr`	`IDENTIFIER`	Trecho de string	`substr(<s>, <inicio>, <tamanho>)` — três argumentos: string, número, número	`substr(e, i, 1)`
`to_string`	`IDENTIFIER`	Número para string	`to_string(<numero>)` — um argumento numérico	`to_string(22)`
`to_number`	`IDENTIFIER`	String para número	`to_number(<s>)`	`to_number(s)`
`parse_ints`	`IDENTIFIER`	Números de uma linha	`parse_ints(<s>)` — devolve a lista dos números separados por espaço	`parse_ints("1 2 3")`
`exp`, `random`	`IDENTIFIER`	Exponencial e número aleatório em $[0, 1]$	`exp(<x>)` e `random()`	`1 / (1 + exp(-x))`
`append`, `push_back`, `pop`, `size`	`IDENTIFIER` após `.`	Métodos de lista	`<lista>.append(<v>)`, `<lista>.pop()`, `<lista>.size()`	`hs.append(h);`
Símbolos especiais				
`#`	— (descartado)	Comentário de linha	`# <texto até o fim da linha>`	`# soma parcial`
fim do texto	`END_OF_FILE`	Marca o fim do fluxo de *tokens*	Gerado sempre, uma vez, no fim	—
outro caractere	`UNKNOWN`	Caractere inválido	Gera o erro `LEX_UNKNOWN_CHAR`; string sem aspas de fechamento gera `LEX_UNTERMINATED_STRING`	`@`, `&`

Fonte: Elaboração própria (2026), a partir de `src/lexer.py`, `src/parser.py` e `src/semantic.py`.

O analisador léxico não interrompe a leitura no primeiro erro: produz *tokens* até o fim do texto e acumula os diagnósticos, ordenados por linha e coluna. Quando há qualquer erro léxico, o analisador sintático não é chamado.

3.3 Gramática

A gramática da MiniPar 2026.2 é LL(1) na prática: o analisador sintático é de descida recursiva e decide cada alternativa olhando um único *token* adiante. A notação usada está no Quadro 4. O Quadro 5 apresenta cada produção, com a forma em BNF, a montagem de cada argumento, o nó da árvore sintática abstrata produzido (com seus campos) e um exemplo. A gramática consolidada, em forma contínua, está no Apêndice A.

Quadro 4 — Notação da gramática

Símbolo	Significado
`<nao_terminal>`	Regra da gramática
`"texto"`	Terminal literal (lexema exato)
`<MAIUSCULO>`	Classe de *token* produzida pelo analisador léxico (Quadro 3)
`[ x ]`	x opcional (zero ou uma vez)
`{ x }`	x repetido zero ou mais vezes
`x | y`	Alternativa
`( x )`	Agrupamento

Fonte: Elaboração própria (2026).

Quadro 5 — Tabela da gramática da MiniPar 2026.2

Construção	Produção (BNF)	Montagem dos argumentos	Nó da AST e campos	Exemplo
Programa e comandos				
Programa	`<programa> ::= { <comando> } <END_OF_FILE>`	Zero ou mais comandos, até o fim do texto; não há função `main`	`Program(statements)`	—
Comando	`<comando> ::= <decl_classe> | <decl_funcao> | <cmd_return> | <cmd_break> | <cmd_continue> | <decl_variavel> | <bloco_seq> | <bloco_par> | <cmd_print> | <cmd_send> | <cmd_if> | <cmd_while> | <cmd_do_while> | <cmd_for> | <cmd_expressao>`	A alternativa é escolhida pelo primeiro *token*: palavra reservada, `<TYPE>`, `{`, ou início de expressão (`IDENTIFIER`, `this`, `new`, `receive`, `input`, `c_channel`, `matrix`, `[`, `fork`, `join`). `class` e `func` são comandos e podem aparecer em blocos	—	—
Declarações				
Variável	`<decl_variavel> ::= ( "var" | <TYPE> ) <IDENTIFIER> [ "=" <expressao> ] ";"`	`"var"` ou nome de tipo (`number`, `int`, `string`, `bool`, `list`, `dict`, `void`); um nome novo no escopo; valor inicial opcional, qualquer expressão	`VarDecl(name, type, init_value)`	`int n = 5;`

(continua)

(continuação)

Construção	Produção (BNF)	Montagem dos argumentos	Nó da AST e campos	Exemplo
Função	`<decl_funcao> ::= "func" <IDENTIFIER> "(" [ <parametro> { "," <parametro> } ] ")" <bloco_chaves> <parametro> ::= [ <TYPE> ] <IDENTIFIER>`	Nome; lista de parâmetros, possivelmente vazia, cada um com tipo opcional (descartado); corpo obrigatoriamente entre chaves	`FuncDecl(name, params, body)`	`func f(a, b) {...}`
Classe	`<decl_classe> ::= "class" <IDENTIFIER> [ "extends" <IDENTIFIER> ] "{" { <membro> } "}" <membro> ::= "var" <IDENTIFIER> [ "=" <expressao> ] ";" | "func" <IDENTIFIER> "(" [ <lista_param> ] ")" <bloco_chaves>`	Nome da classe; superclasse opcional; membros: atributos só com `var` (nome de tipo não é aceito) e métodos com `func`	`ClassDecl(name, superclass, attributes, methods)`	`class A { var x = 0; }`
Comandos simples				
Retorno	`<cmd_return> ::= "return" [ <expressao> ] [ ";" ]`	Valor opcional; sem valor, exige `;` logo após `return`	`ReturnStmt( value)`	`return x;`
Saída de laço	`<cmd_break> ::= "break" [ ";" ] <cmd_continue> ::= "continue" [ ";" ]`	Sem argumentos; o contexto de laço é verificado pelo semântico	`BreakStmt`, `ContinueStmt`	`break;`
Impressão	`<cmd_print> ::= "print" "(" [ <lista_ argumentos> ] ")" ";"`	Zero ou mais expressões separadas por vírgula	`PrintStmt( arguments)`	`print(a, b);`
Envio	`<cmd_send> ::= "send" "(" <expressao> "," <expressao> ")" ";"`	1º: canal criado por `c_ channel`; 2º: mensagem	`SendStmt( channel, message)`	`send(c, "5");`
Atribuição	`<cmd_expressao> ::= <expr_inicial> "=" <expressao> ";"`	Lado esquerdo: identificador, acesso a atributo (`x.p`) ou indexação (`x[i]`); outro alvo gera `SYN_INVALID_ ASSIGNMENT`. Lado direito: qualquer expressão	`Assignment(name, value)`, `PropertyAssign( object, property_name, value)` ou `IndexAssign( target, value)`	`this.v[i] = t;`
Expressão como comando	`<cmd_expressao> ::= <expr_inicial> ";"`	A expressão inteira deve ser chamada de função, chamada de método, `fork` ou `join`; senão, `SYN_INVALID_STATEMENT` (`x + 1;` e `input();` são rejeitados)	O próprio nó da expressão	`f(x);`
Blocos				
Bloco `seq`	`<bloco_seq> ::= [ "seq" ] <bloco_chaves> <bloco_chaves> ::= "{" { <comando> } "}"`	Lista de comandos, executados em ordem; `seq` opcional	`SeqBlock( statements)`	`seq { a(); b(); }`
Bloco `par`	`<bloco_par> ::= "par" "{" { <comando> } "}"`	Cada comando direto é um ramo paralelo (uma thread); o bloco espera todos os ramos	`ParBlock( statements)`	`par { f(); g(); }`
Controle de fluxo				
Condicional	`<cmd_if> ::= "if" "(" <expressao> ")" <comando> [ "else" <comando> ]`	Condição entre parênteses; ramo então; ramo senão opcional, ligado ao `if` mais próximo	`IfStmt( condition, then_ branch, else_ branch)`	`if (a) x(); else y();`

(continua)

(continuação)

Construção	Produção (BNF)	Montagem dos argumentos	Nó da AST e campos	Exemplo
Enquanto	`<cmd_while> ::= "while" "(" <expressao> ")" <comando>`	Condição; corpo (um comando ou bloco)	`WhileStmt(condition, body)`	`while (i < n) i = i + 1;`
Faça-enquanto	`<cmd_do_while> ::= "do" <comando> "while" "(" <expressao> ")" [ ";" ]`	Corpo executado ao menos uma vez; condição no fim	`DoWhileStmt(body, condition)`	`do {...} while (k > 0);`

Para (clássico)	`<cmd_for> ::= "for" "(" [ <init_for> ] ";" [ <expressao> ] ";" [ <passo_for> ] ")" <comando>`
	`<init_for> ::= ( "var" | <TYPE> ) <IDENTIFIER> [ "=" <expressao> ] | <expressao> [ "=" <expressao> ]`
	`<passo_for> ::= <expressao> [ "=" <expressao> ]`
		Três partes opcionais separadas por `;`: inicialização (declaração ou atribuição), condição e passo (atribuição ou expressão)	`ForStmt(initializer, condition, increment, body)`	`for (var i = 0; i < n; i = i + 1) ...`

Para (iteração)	`<cmd_for> ::= "for" "(" "var" <IDENTIFIER> "in" <expressao> ")" <comando>`	`var` obrigatório (`for (x in l)` é erro); variável de iteração; expressão que produz a lista	`ForStmt(iterator_name, iterable, body)`	`for (var h in hs) ...`
Expressões, da menor para a maior precedência				
Expressão	`<expressao> ::= <expressao_ou>`	Ponto de entrada de toda expressão	—	—
OU lógico	`<expressao_ou> ::= <expressao_e> { "||" <expressao_e> }`	Operandos do nível E; associa à esquerda	`BinaryExpr(op, left, right)`	`a || b`
E lógico	`<expressao_e> ::= <expressao_comp> { "&&" <expressao_comp> }`	Operandos do nível de comparação	`BinaryExpr`	`a && b`
Comparação	`<expressao_comp> ::= <expressao_termo> { ( "==" | "!=" | "<" | ">" | "<=" | ">=" ) <expressao_termo> }`	Igualdade e relacionais no mesmo nível: `1 < 2 == true` é `(1 < 2) == true`	`BinaryExpr`	`i <= n`
Termo	`<expressao_termo> ::= <expressao_fator> { ( "+" | "-" ) <expressao_fator> }`	Soma e subtração; associa à esquerda (`a - b - c` é `(a - b) - c`)	`BinaryExpr`	`a + b - c`
Fator	`<expressao_fator> ::= <expressao_unaria> { ( "*" | "/" | "%" ) <expressao_unaria> }`	Multiplicação, divisão e resto	`BinaryExpr`	`2 * x % 3`
Unária	`<expressao_unaria> ::= ( "-" | "+" | "!" ) <expressao_unaria> | <expressao_posfixa>`	Operador prefixo aplicado recursivamente; associa à direita	`UnaryExpr(op, operand)`	`-x`, `!!ok`
Pós-fixa	`<expressao_posfixa> ::= <expressao_primaria> { <sufixo> }` `<sufixo> ::= "." <IDENTIFIER> [ "(" [ <lista_argumentos> ] ")" ] | "[" <expressao> "]"`	Sufixos aplicados em laço, da esquerda para a direita: atributo, chamada de método ou índice	`PropertyAccess(object, property_name)`, `MethodCall(object, method_name, arguments)`, `IndexExpr(object, index)`	`a.b[0].c(1)`

(continua)

(continuação)

Construção	Produção (BNF)	Montagem dos argumentos	Nó da AST e campos	Exemplo
Argumentos	`<lista_argumentos> ::= <expressao> { "," <expressao> }`	Uma ou mais expressões; os parênteses externos admitem lista vazia	—	`f(1, x + 2)`
Expressões primárias				
Literais	`<NUMBER> | <STRING_LITERAL> | <BOOL_LITERAL>`	Constantes do analisador léxico	`NumberExpr`, `StringExpr`, `BoolExpr` (`value`)	`3.5`, `"a"`, `true`
Nome ou chamada	`<IDENTIFIER> [ "(" [ <lista_argumentos> ] ")" ]`	Sem parênteses, referência a variável; com parênteses, chamada de função do programa ou nativa	`IdentifierExpr(name)` ou `FuncCallExpr(name, arguments)`	`fat(n - 1)`
Objeto	`"new" <IDENTIFIER> "(" [ <lista_argumentos> ] ")" | "this"`	Nome da classe e argumentos; `this` só dentro de método	`NewExpr(class_name, arguments)`, `ThisExpr`	`new Pilha()`
Lista	`"[" [ <lista_argumentos> ] "]"`	Elementos separados por vírgula; pode ser vazia	`ListLiteral(elements)`	`[]`, `[1, 2]`
Matriz	`"matrix" "(" <expressao> "," <expressao> "," <expressao> ")"`	Linhas, colunas e valor inicial de todas as células	`MatrixCreateExpr(rows, cols, init_value)`	`matrix(2, 2, 0)`
Canal	`"c_channel" "(" <expressao> "," <expressao> ")"`	Endereço IP e porta	`CChannelExpr(ip, port)`	`c_channel(ip, 9000)`
Recepção	`"receive" "(" <expressao> ")"`	Canal de onde a mensagem é lida	`ReceiveExpr(channel)`	`receive(c)`
Entrada	`"input" "(" [ <expressao> ] ")"`	*Prompt* opcional	`InputExpr(prompt)`	`input("n: ")`
Agrupamento	`"(" <expressao> ")"`	Altera a precedência; não gera nó próprio	O nó interno	`(a + b) * c`
`fork`	`<expressao_fork> ::= "fork" <IDENTIFIER> "(" [ <lista_argumentos> ] ")" [ "on" <expressao> ]`	1º: nome de `func` global; depois, argumentos copiados por valor; por fim, destino opcional. `on` consome uma expressão completa: `fork f() on d + 1` usa `d + 1`	`ForkExpr(function_name, arguments, destination)`	`fork f(a) on d0`
`join`	`<expressao_join> ::= "join" "(" <expressao> [ "," <expressao> ] ")"`	1º: *handle* devolvido por `fork`; 2º, opcional: limite de tempo em milissegundos	`JoinExpr(handle, timeout)`	`join(h, 3000)`

Fonte: Elaboração própria (2026), a partir de `src/parser.py` e `src/ast_nodes.py`.

Como `fork` e `join` são expressões primárias, aceitam sufixos (`.` e `[ ]`); as regras semânticas decidem se o uso é válido (seção 3.6). O analisador não tem recuperação de erros: o primeiro erro sintático interrompe a análise.

3.4 Precedência e associatividade

O Quadro 6 lista os níveis de precedência, do maior para o menor. Não há operador ternário, atribuição composta (+=) nem incremento (++).

Quadro 6 — Precedência e associatividade dos operadores

Nível	Operadores	Associatividade	Método do analisador
7 (maior)	`. .m(...) [ ]` (sufixos)	esquerda	`_parse_primary`
6	`- + !` (unários)	direita	`_parse_unary`
5	`* / %`	esquerda	`_parse_factor`
4	`+ -`	esquerda	`_parse_term`
3	`== != < > <= >=`	esquerda	`_parse_comparison`
2	`&&`	esquerda	`_parse_and`
1 (menor)	`||`	esquerda	`_parse_or`

Fonte: Elaboração própria (2026), a partir de `src/parser.py`.

3.5 Orientação a objetos, blocos paralelos e canais

Uma classe declara atributos com `var` e métodos com `func`; `extends` define herança simples, `new` cria um objeto e `this` referencia o objeto corrente. O analisador semântico verifica, considerando a cadeia de herança, se os membros usados existem e se o número de argumentos corresponde ao de parâmetros, e detecta superclasses inexistentes e ciclos de herança. O Teste 6 (Código 50) é um exemplo completo, com recursão em métodos.

Um bloco `seq` executa seus comandos em ordem; um bloco `par` executa cada comando direto em uma thread e termina quando todos terminam, como no Teste 2 (Código 46). Os canais ligam dois pontos por *socket*: `c_channel(ip, porta)` cria o canal, `send` envia e `receive` recebe uma mensagem como string. No Teste 1 (Código 45), servidor e cliente de uma calculadora são dois ramos de um bloco `par` ligados por dois canais.

3.6 Tarefas com Fork e Join

O `fork` cria uma tarefa que executa uma função global com argumentos copiados por valor e devolve imediatamente um *handle*, sem esperar o término. O `join` espera a tarefa e devolve o valor de retorno da função. Sem `on`, ou com o destino `"local"`, a tarefa roda numa thread do mesmo processo; com `"host:porta"`, roda num processo trabalhador, possivelmente em outro computador. Assim, a mesma tarefa é alocada a um núcleo ou a um computador diferente apenas trocando o destino. No Teste 8 (Código 1), quatro tarefas são disparadas antes de qualquer `join`, cada uma com seu destino lido da entrada; depois, um laço coleta os resultados na ordem dos *handles*.

Código 1 — Fork e Join no Teste 8

```
var resultados = [];
```

```
if (modo == "par") {
    # Dispara as 4 tarefas antes de qualquer join.
    var hs = [];
    hs.append(fork avaliar_lote(0, 0 * fatia, 0 * fatia + fatia, carga) on d0);
    hs.append(fork avaliar_lote(1, 1 * fatia, 1 * fatia + fatia, carga) on d1);
    hs.append(fork avaliar_lote(2, 2 * fatia, 2 * fatia + fatia, carga) on d2);
    hs.append(fork avaliar_lote(3, 3 * fatia, 3 * fatia + fatia, carga) on d3);
    for (var h in hs) { resultados.append(join(h)); }
} else {
    var k = 0;
    while (k < num_avaliadores) {
```

Fonte: Elaboração própria (2026).

Como as tarefas podem rodar em outro processo, sem memória compartilhada, o analisador semântico impõe as regras do Quadro 7. Os tipos que podem cruzar tarefas são números, `bool`, `string`, `void` e listas desses tipos, inclusive aninhadas. Quando o tipo de um argumento não pode ser provado, o analisador registra uma nota em vez de um erro; quem garante a serialização no programa compilado é o gerador C++, que recusa o programa com `CPP_UNSUPPORTED_FEATURE` se não provar tipos concretos (seção 5.8). A tradução de `fork` e `join` está no Quadro 8.

Quadro 7 — Regras estáticas de Fork/Join

O que é rejeitado	Código
`fork` de um nome que não é `func` global (variável, método, local que sombreia a global, nome inexistente)	`SEM_FJ_01`
Número de argumentos diferente do número de parâmetros	`SEM_ARITY`
`join` de expressão de tipo conhecido diferente de `task`	`SEM_FJ_03`
Destino conhecido que não é string; limite de tempo conhecido que não é numérico	`SEM_FJ_04`
Tarefa que lê ou escreve variável global mutável, direta ou transitivamente; `this` em tarefa global; `input`, `c_channel`, `send` ou `receive` na tarefa	`SEM_FJ_05`
Argumento ou retorno que não pode cruzar tarefas (objeto, canal, *handle*, inclusive dentro de listas)	`SEM_FJ_06`
`fork` usado como comando, descartando o *handle*	`SEM_FJ_07`
Resultado vazio de `join` usado como valor (`join(h);` como comando é válido)	`SEM_FJ_08`

Fonte: Elaboração própria (2026), a partir de `src/semantic.py`.

Quadro 8 — Tradução de Fork/Join

Fase	**`fork f(a) on d`**	**`join(h, t)`**
TAC	`tK = capture a` … `tN = fork f(tK) on tD` (`"local"` sem `on`)	`tM = join h, timeout t`
C++	`minipar_rt::start("f", {to_value(a)}, Destination::parse(d))`	`value_cast<T>(__fj_join(h, __fj_timeout(t)))`

Fonte: Elaboração própria (2026), a partir de `src/tac.py` e `src/codegen.py`.

3.7 Restrições de contexto

A gramática aceita programas que o analisador semântico rejeita (Quadro 9). A compatibilidade de tipos é permissiva onde a inferência não tem informação: `var` e `unknown` são compatíveis com qualquer tipo, `int` e `number` são compatíveis entre si, e os demais tipos exigem igualdade.

Quadro 9 — Restrições de contexto verificadas pelo analisador semântico

Código	Regra
`SEM_UNDECLARED_VARIABLE`	Uso ou atribuição de nome não declarado em nenhum escopo visível
`SEM_REDECLARED`	Nome declarado duas vezes no mesmo escopo (sombrear em escopo interno é permitido)
`SEM_TYPE_MISMATCH`	Declaração ou atribuição com tipo incompatível; string com outro tipo em `+` ou em comparação; argumento de `substr`, `to_string` ou `len` de tipo errado
`SEM_NUMERIC_OPERANDS`	`-`, `*`, `/` ou `%` com operando de tipo conhecido não numérico
`SEM_OUTSIDE_LOOP`	`break` ou `continue` fora de laço
`SEM_UNDECLARED_FUNCTION`	Chamada a função não declarada nem nativa
`SEM_ARITY`	Número de argumentos diferente em função, método, nativa (`substr` 3, `to_string` 1, `len` 1) ou `fork`
`SEM_RESERVED_NAME`	`func` global com nome de função nativa
`SEM_UNDECLARED_CLASS`, `SEM_UNDECLARED_SUPERCLASS`	`new` ou `extends` de classe inexistente
`SEM_INHERITANCE_CYCLE`	Cadeia de herança cíclica
`SEM_UNDECLARED_ATTRIBUTE`, `SEM_UNDECLARED_METHOD`	Membro inexistente em objeto de classe conhecida, considerando a herança
`SEM_UNSUPPORTED_FEATURE`	Indexação de string (`s[i]`; usa-se `substr(s, i, 1)`) e lista de strings

Fonte: Elaboração própria (2026), a partir de `src/semantic.py`.

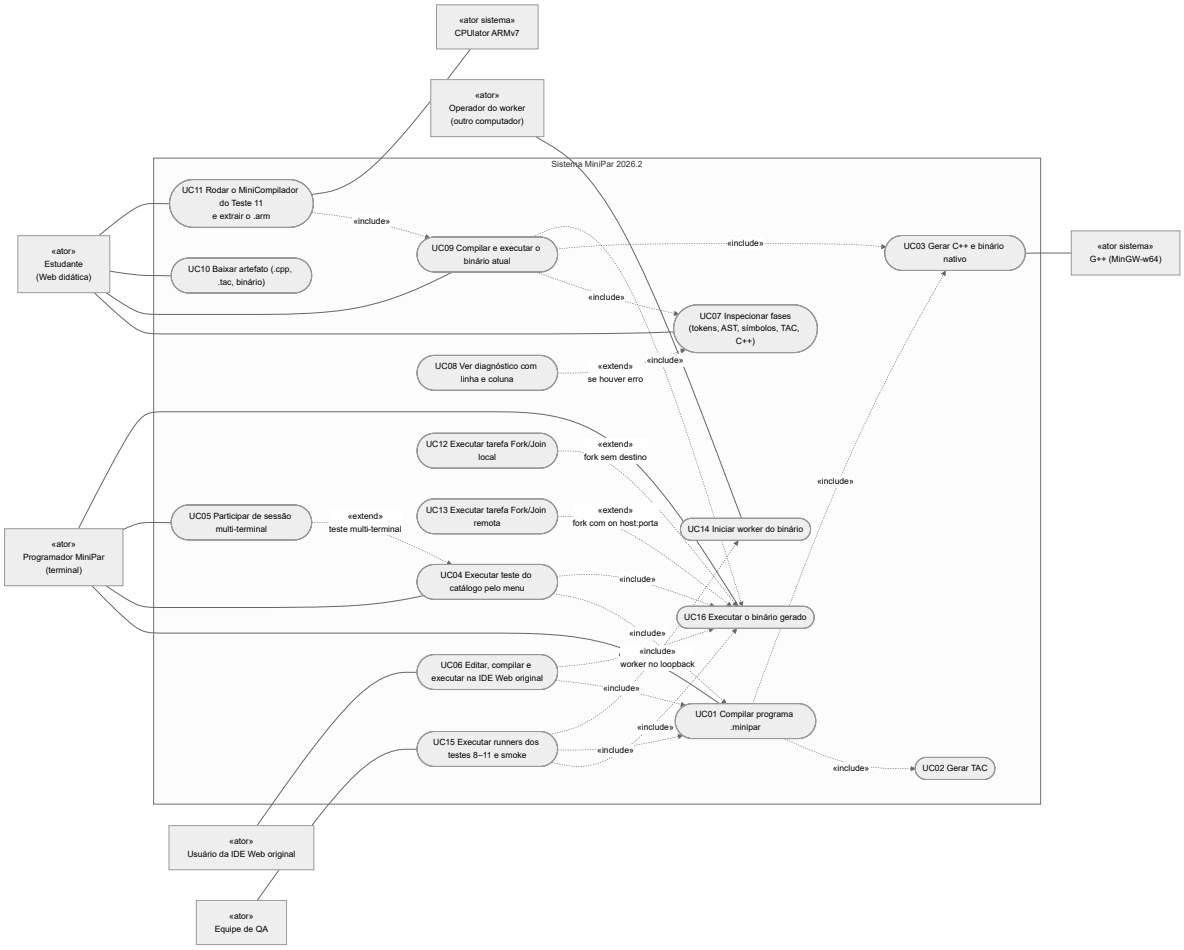
4 ARQUITETURA E MODELAGEM UML

Este capítulo apresenta a modelagem do sistema em UML: casos de uso, arquitetura de componentes, diagramas de classes e diagramas de sequência. Os diagramas modelam o sistema como ele está implementado: cada classe, atributo, método e mensagem citado existe no código. Eles foram escritos em Mermaid (Mermaid, 2026); os de casos de uso e de componentes usam fluxogramas com a semântica da UML, já que o Mermaid não tem esses dois tipos. Nos diagramas de classes, + indica membro público, - membro privado, e módulos sem classe aparecem com o estereótipo «module».

4.1 Diagrama de casos de uso

A Figura 1 mostra os atores e os casos de uso. Os atores principais são o programador, que usa o terminal; o usuário da IDE Web original; o estudante, que usa a Web didática; o operador do trabalhador remoto; e quem executa os verificadores automatizados. O GCC e o CPUlator são sistemas externos. Gerar (UC03) e executar (UC16) são casos separados: as tarefas locais e remotas (UC12 e UC13) estendem a execução, porque `fork` e `join` acontecem quando o programa chega a um `fork`, e iniciar o trabalhador (UC14) é pré-condição da tarefa remota. O Quadro 10 descreve cada caso de uso, com o ponto de entrada no código.

Figura 1 — Diagrama de casos de uso do MiniPar 2026.2



Fonte: Elaboração própria (2026).

Quadro 10 — Descrição dos casos de uso

ID	Ator	Ponto de entrada	Resultado
UC01	Programador	`python main.py <arquivo.minipar> [saida]` → `main.compile_file`	`.tac`, `.cpp` e executável, ou erro da fase
UC02	(incluído)	`TACGenerator.generate` / `generate_text`	Código de três endereços
UC03	(incluído)	`CppCodeGenerator.generate` e `compile_cpp`	`.cpp` e binário por `g++ -std=c++20 -O3`
UC04	Programador	`python main.py` → opção 1 → `menu.exibir_menu`	Compilação e execução dos programas de `tests/`
UC05	Programador (2º terminal)	`test_orchestrator.join_session`	Terminal participa de uma sessão ativa
UC06	Usuário da IDE	`python main.py` → opção 2 → `app.py`, `POST /api/compile`	Saída, `.cpp` e `.tac` na IDE
UC07	Estudante	`POST /api/v1/analyze` → `service.analyze`	Fases do léxico ao C++
UC08	(extensão de UC07)	`diagnostics[]` da resposta	Cartão de erro com fase, linha e coluna
UC09	Estudante	`POST /api/v1/compile` e `/api/v1/execute`	Binário executado com entrada e tempo limite
UC10	Estudante	`GET /api/v1/artifacts/<id>/<tipo>`	Download do `.cpp`, do `.tac` ou do binário
UC11	Estudante	Exemplo do Teste 11, depois UC09	Assembly extraído da saída, para o CPUlator
UC12	(extensão de UC16)	`fork f(args)` → `start(..., Destination::parse("local"))`	Tarefa numa thread do executor
UC13	(extensão de UC16)	`fork f(args) on "host:porta"`; pré-condição: UC14	Tarefa executada no trabalhador remoto
UC14	Operador	`<binario> --minipar-worker <porta> [--bind <ip>]`	Processo trabalhador aceitando conexões
UC15	Verificação	`run_t08.py`, `run_t09.py`, `run_t10.py`, `verificar_t11.py`	Oráculos dos testes
UC16	Programador	Executável gerado, `menu.executar_binario`, `service.execute`	Saída do programa; com Fork/Join, UC12 ou UC13

Fonte: Elaboração própria (2026).

4.2 Arquitetura de componentes

A Figura 2 mostra a arquitetura em camadas. A apresentação tem três interfaces (terminal, Web original e Web didática). O serviço de compilação expõe as fases à Web didática e usa dois auxiliares: o serializador da AST e o executor de processos com limite de tempo. O núcleo da linguagem contém os analisadores e a AST; o gerador TAC e o gerador C++ leem a AST em ramos independentes. O gerador C++ usa o contexto de tarefas do analisador semântico e chama o GCC, que produz o binário. Esse binário inclui o runtime `minipar_rt`, que fornece a API de tarefas e fala o protocolo TCP quando atua como coordenador ou como trabalhador. O Quadro 11 relaciona cada componente aos arquivos, à interface fornecida e às dependências.

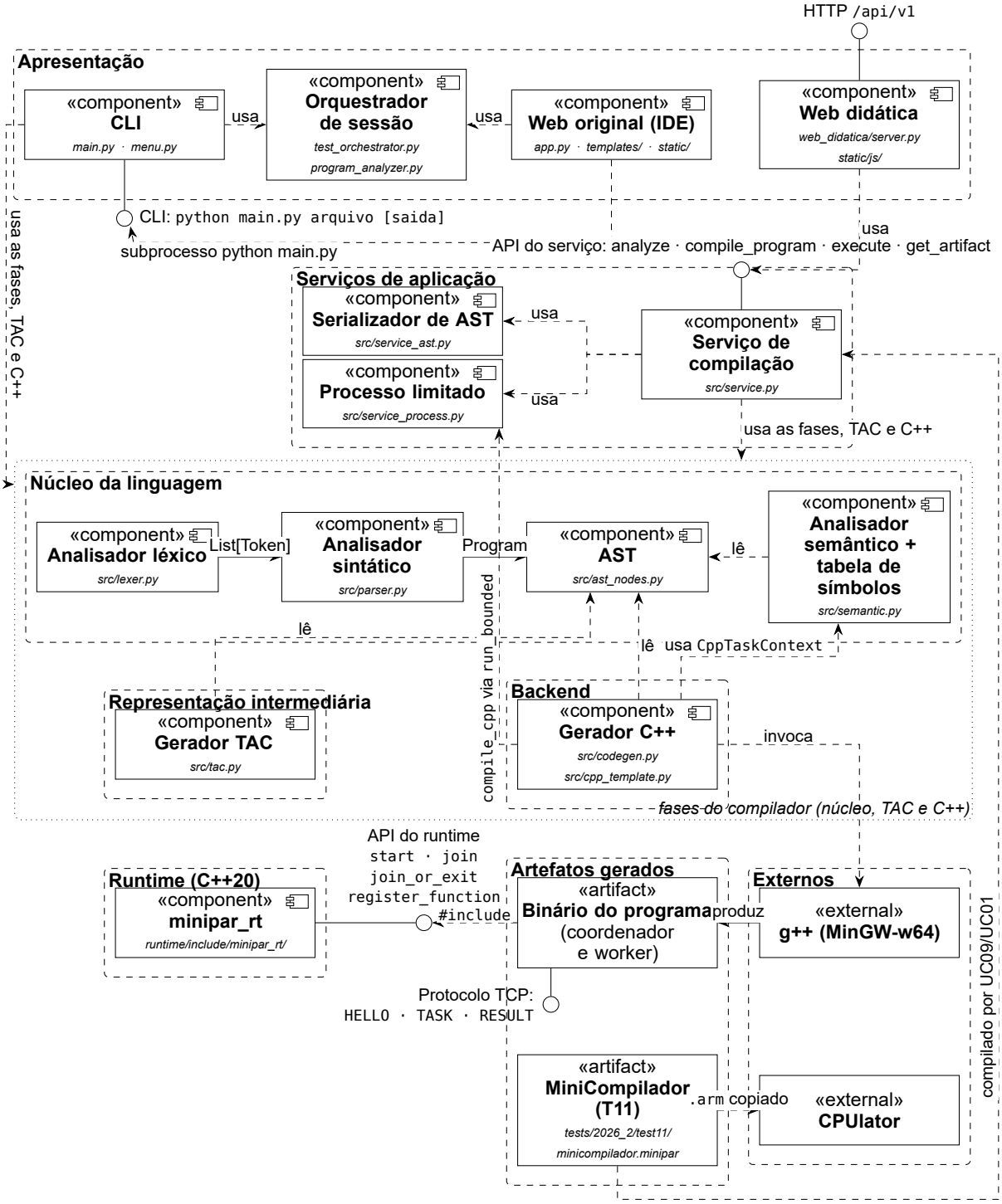
Quadro 11 — Componentes, arquivos e interfaces

Componente	Arquivos	Interface fornecida	Requer
CLI	`main.py`, `menu.py`	`compile_file(source_path, output_name) -> bool`	Núcleo, TAC, C++
Web original	`app.py`, `templates/`, `static/`	Rotas `/`, `/api/tests`, `/api/compile`	CLI por subprocesso
Web didática	`web_didatica/server.py`, `static/`	Rotas `/api/v1/analyze`, `compile`, `execute`, `artifacts`	Serviço de compilação
Serviço de compilação	`src/service.py`	`analyze`, `compile_program`, `execute`, `get_artifact`	Núcleo, TAC, C++, auxiliares
Serializador de AST	`src/service_ast.py`	`serialize_ast(program)`	AST
Processo limitado	`src/service_process.py`	`run_bounded(command, cwd, stdin, timeout)`	Sistema operacional
Analisador léxico	`src/lexer.py`	`Lexer(source).tokenize() -> List[Token]`	—
Analisador sintático	`src/parser.py`	`Parser(tokens).parse() -> Program`	Léxico, AST
AST	`src/ast_nodes.py`	37 classes de nó; `iter_child_nodes`, `find_fork_join`	—
Semântico e tabela de símbolos	`src/semantic.py`	`SemanticAnalyzer.analyze(program) -> bool`, `scopes`, `symbols`; `CppTaskContext`	AST
Gerador TAC	`src/tac.py`	`generate_text(program) -> str`	AST
Gerador C++	`src/codegen.py`, `src/cpp_template.py`	`generate_text`, `compile_cpp(cpp, exe)`	AST, semântico, GCC, runtime
Runtime `minipar_rt`	`runtime/include/minipar_rt/`	`register_function`, `start`, `join`, `join_or_exit`; protocolo TCP	Threads e *sockets*
MiniCompilador	`tests/2026_2/test11/`	Programa MiniPar que imprime *tokens*, AST, TAC, ARM e símbolos	Compilador MiniPar

Fonte: Elaboração própria (2026).

Não há interfaces abstratas entre as fases: elas são classes concretas chamadas em sequência, e o contrato entre elas é o tipo dos dados trocados (`List[Token]`, `Program`). A tabela de símbolos não é uma classe, mas uma pilha de dicionários mantida pelo analisador semântico (seção 5.5). O runtime entra no binário por `#include`, e os canais herdados (`c_channel`, `send`, `receive`) usam *sockets* próprios, por meio da classe `MiniParChannel` do prólogo C++.

Figura 2 — Diagrama de componentes do MiniPar 2026.2



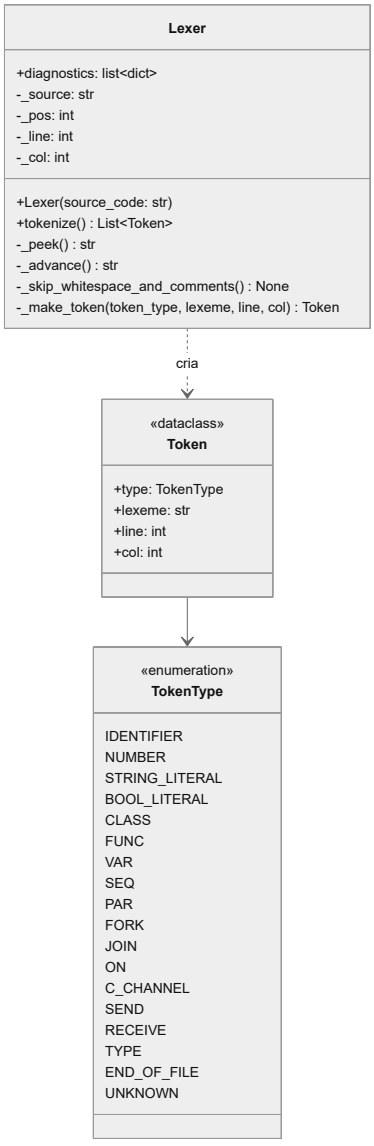
Fonte: Elaboração própria (2026).

Nota: As setas da CLI e do serviço para a moldura pontilhada indicam o uso de cada uma das fases nela contidas.

4.3 Diagramas de classes

As Figuras 3 a 15 mostram as classes de cada componente, com os membros públicos e os privados que explicam o funcionamento. No analisador léxico (Figura 3), `TokenType` tem 57 valores, e o diagrama omite os de operadores e pontuação, já listados no Quadro 3. A AST (Figuras 4 e 5) tem 37 classes de nó, todas *dataclasses* Python com o atributo `line`; `fork` e `join` são expressões (`ForkExpr`, `JoinExpr`), e `ForStmt` representa as duas formas de `for`.

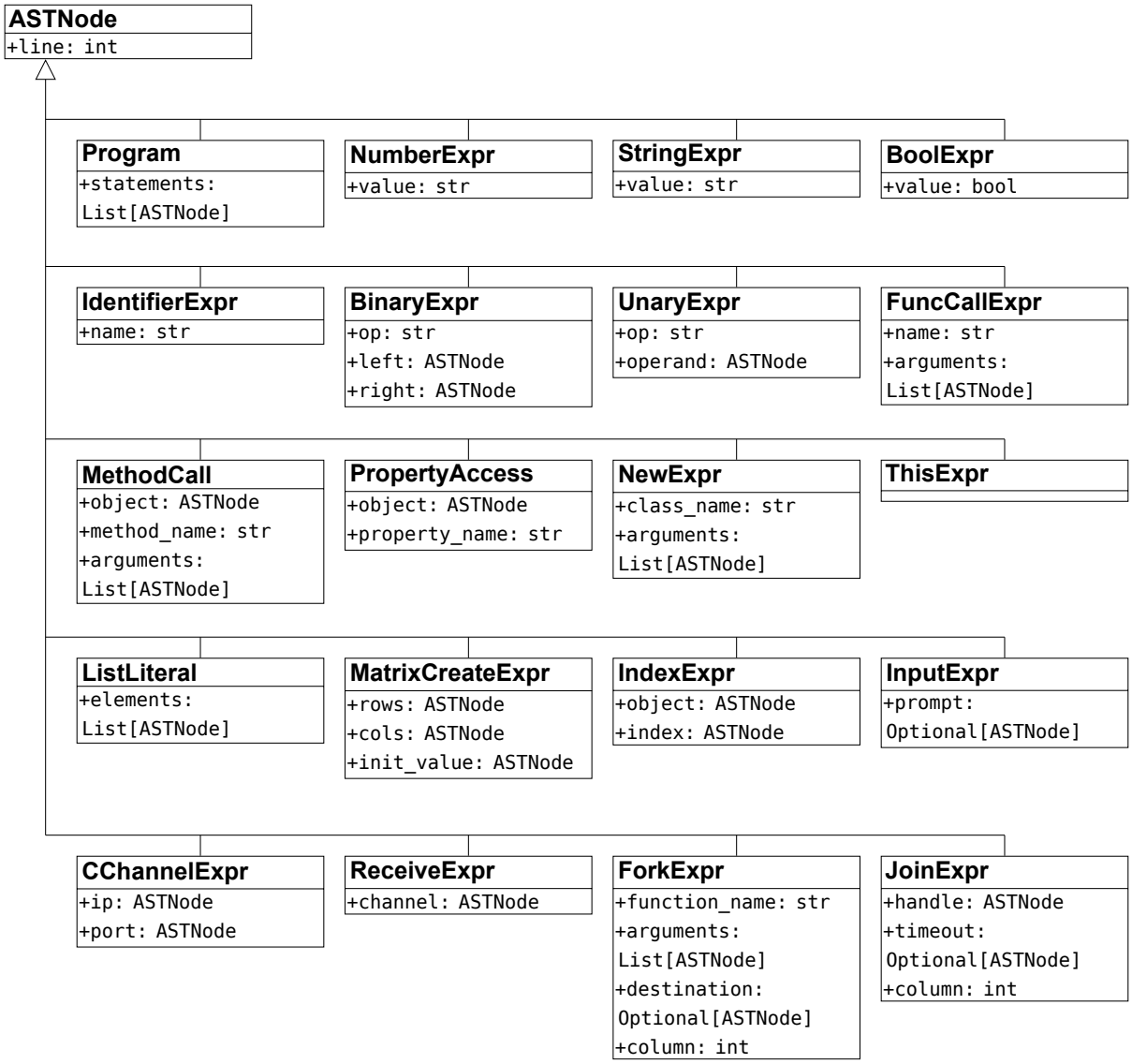
Figura 3 — Diagrama de classes do analisador léxico



Fonte: Elaboração própria (2026).

O analisador sintático (Figura 6) tem um método por nível de precedência. O analisador semântico (Figura 7) mantém a pilha `_scopes`, os tipos estruturais (`ValueType`) usados na inferência de listas e tarefas e o contexto de tarefas (`CppTaskContext`) con-

Figura 4 — Diagrama de classes da AST: raiz e nós de expressão

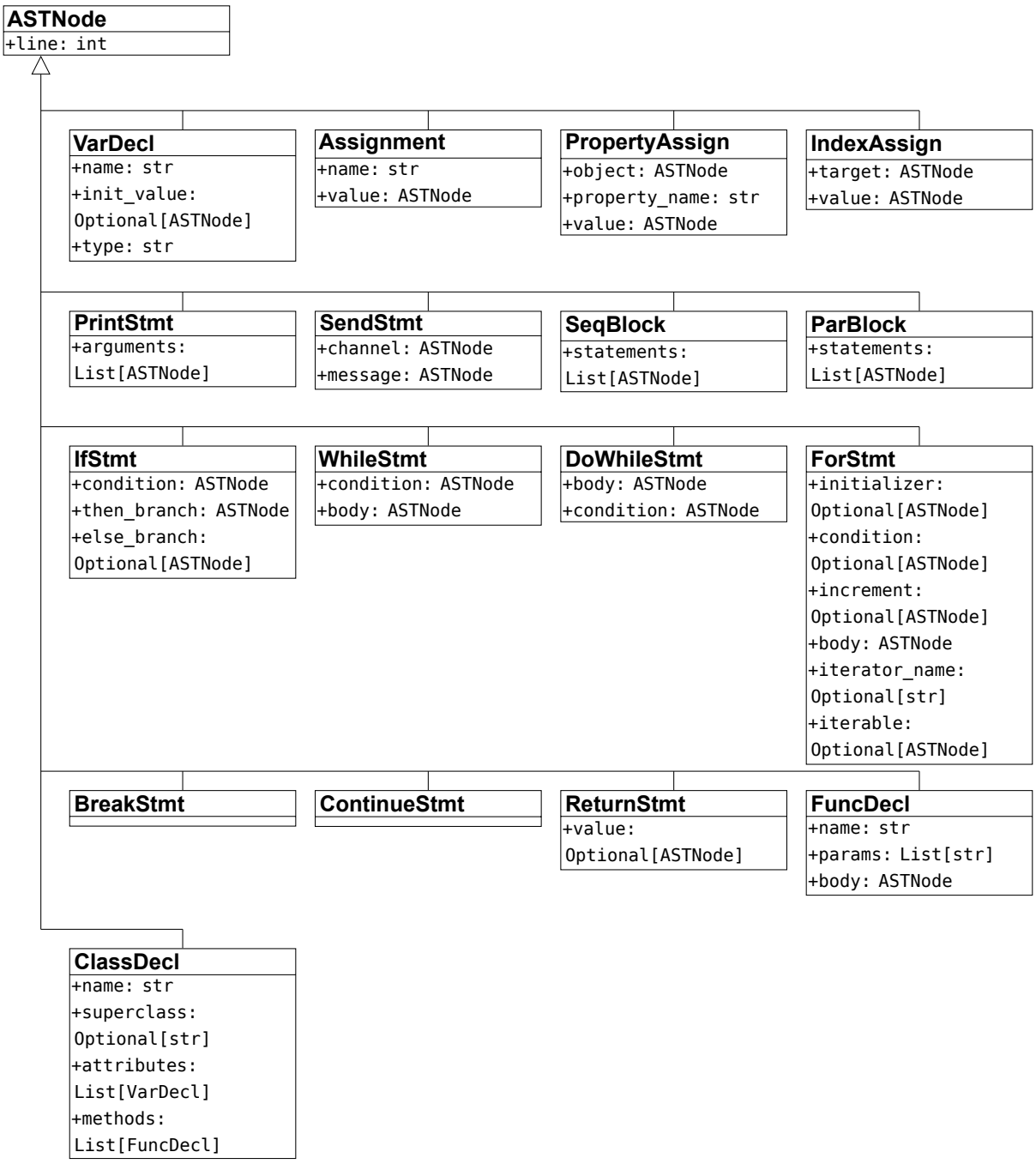


Fonte: Elaboração própria (2026).

sumido pelo gerador C++. Os geradores TAC e C++ aparecem nas Figuras 8 e 9, e o serviço de compilação, que é um módulo de funções, na Figura 10.

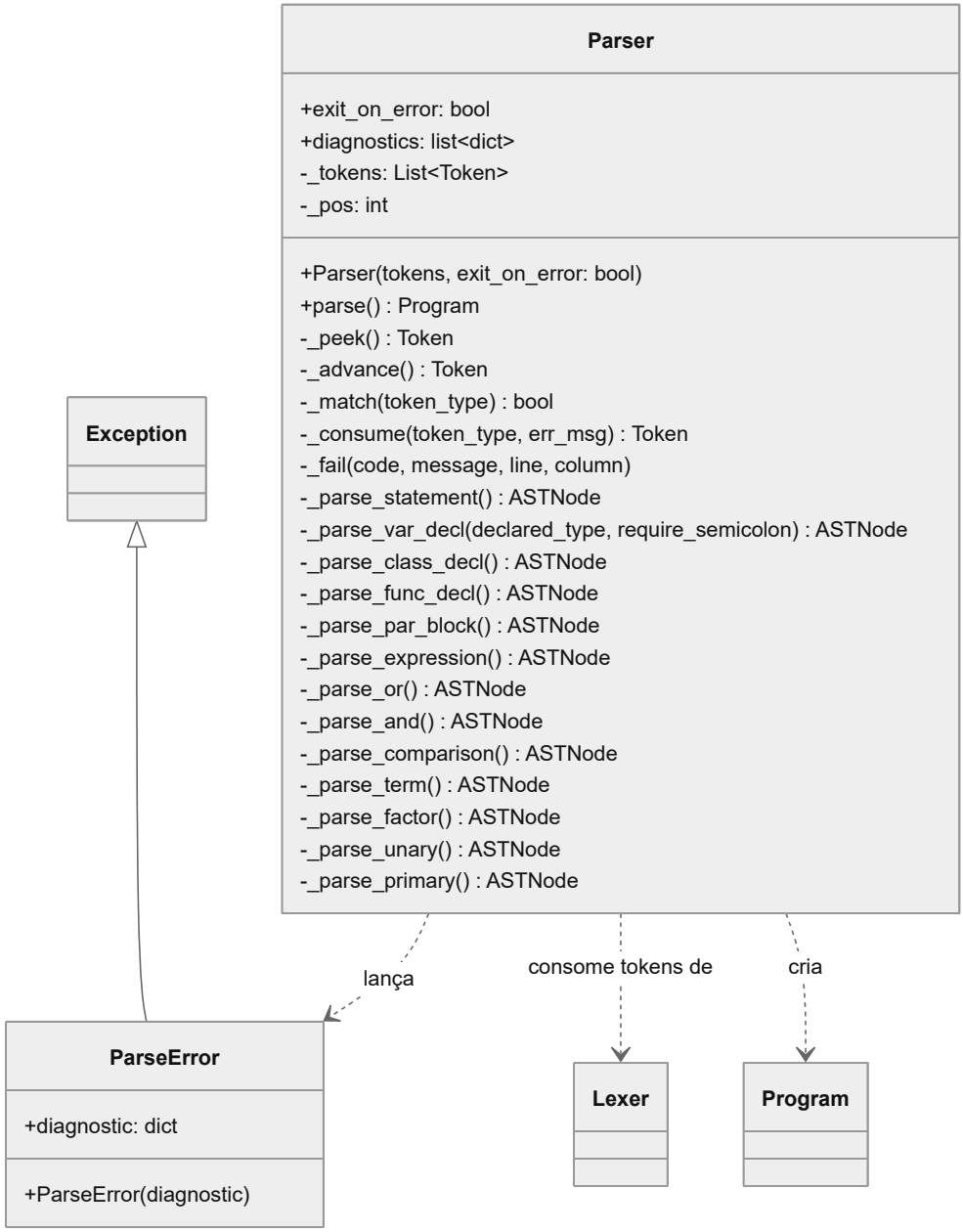
As interfaces aparecem nas Figuras 11 e 12; os módulos JavaScript da Web didática escolhem entre a fonte de dados real, que chama o servidor, e uma simulada, que não compila. O runtime (Figuras 13 e 14) reúne o tipo de valor que cruza tarefas (`Value`), os resultados e erros, os destinos, o registro de funções, o executor de threads e, na parte remota, as conexões, o trabalhador e a camada de *sockets*. Uma conexão passa pelos estados `Connecting`, `Ready` e `Dead`, e uma tarefa submetida a uma conexão morta termina com o erro que a derrubou. O MiniCompilador do Teste 11 (Figura 15) é uma classe escrita em MiniPar.

Figura 5 — Diagrama de classes da AST: declarações e comandos



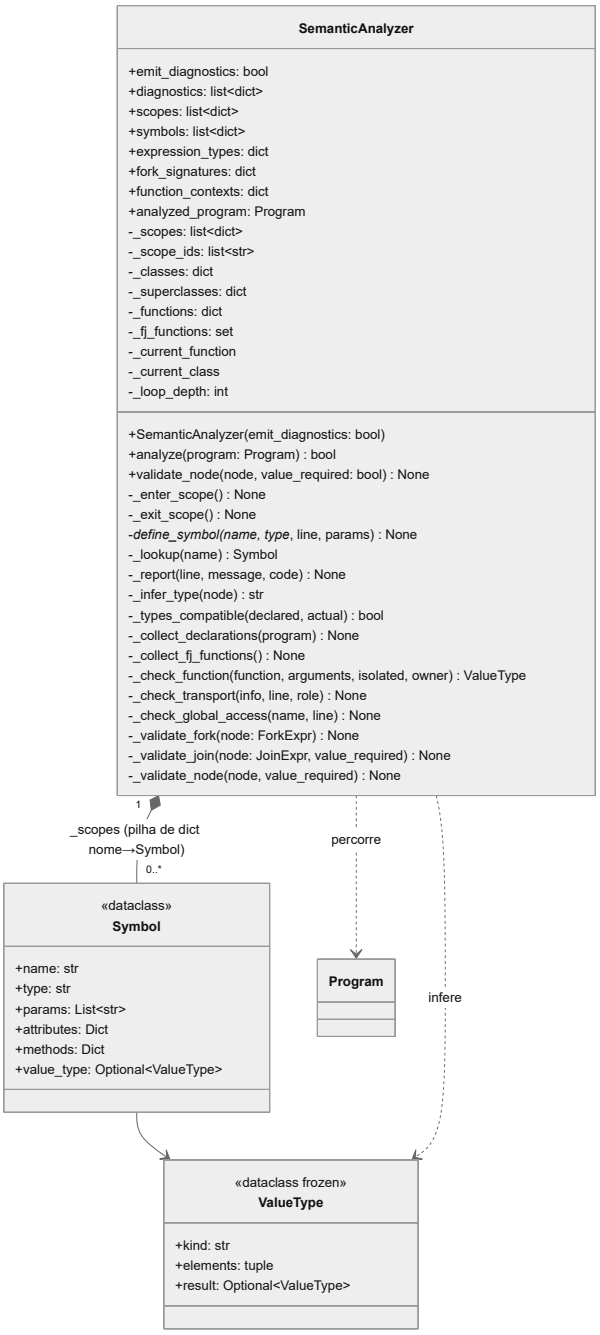
Fonte: Elaboração própria (2026).

Figura 6 — Diagrama de classes do analisador sintático



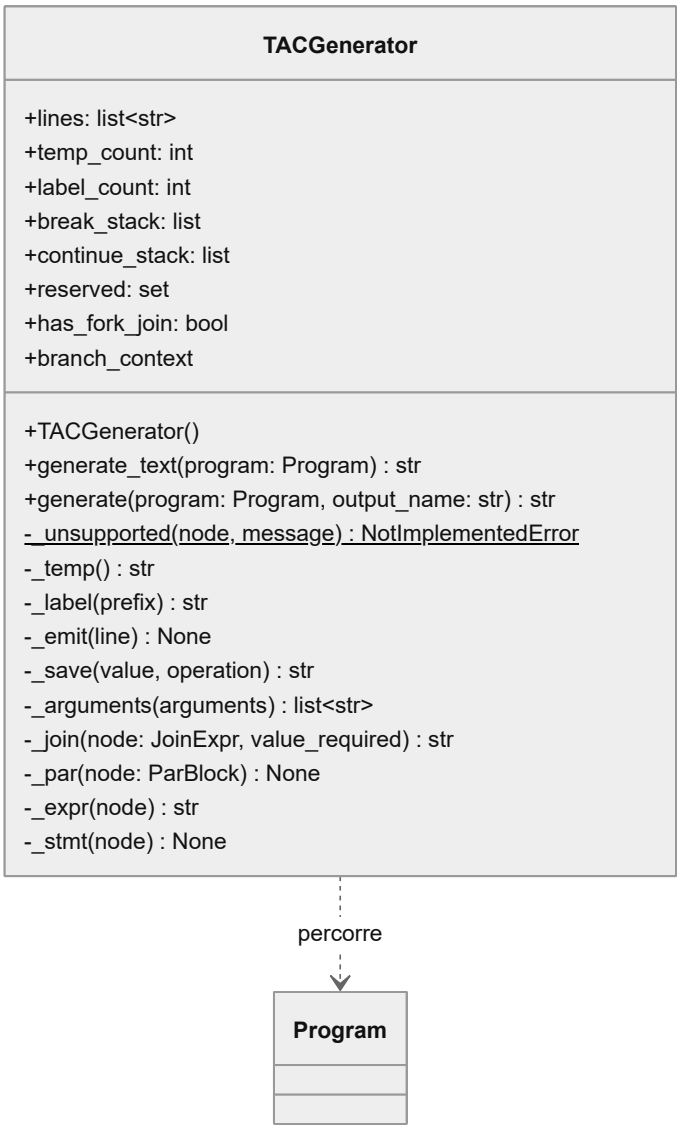
Fonte: Elaboração própria (2026).

Figura 7 — Diagrama de classes do analisador semântico e da tabela de símbolos



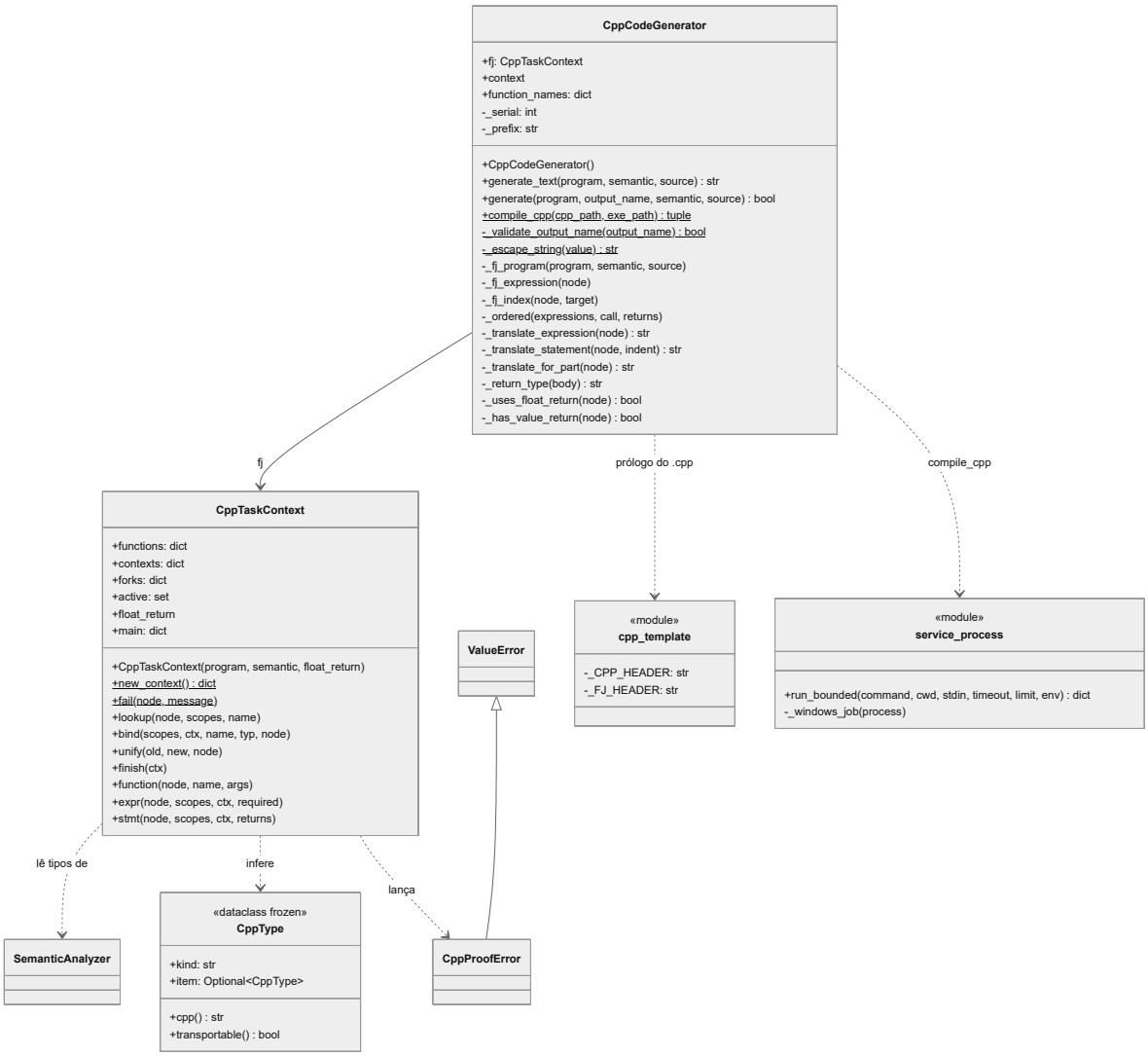
Fonte: Elaboração própria (2026).

Figura 8 — Diagrama de classes do gerador TAC



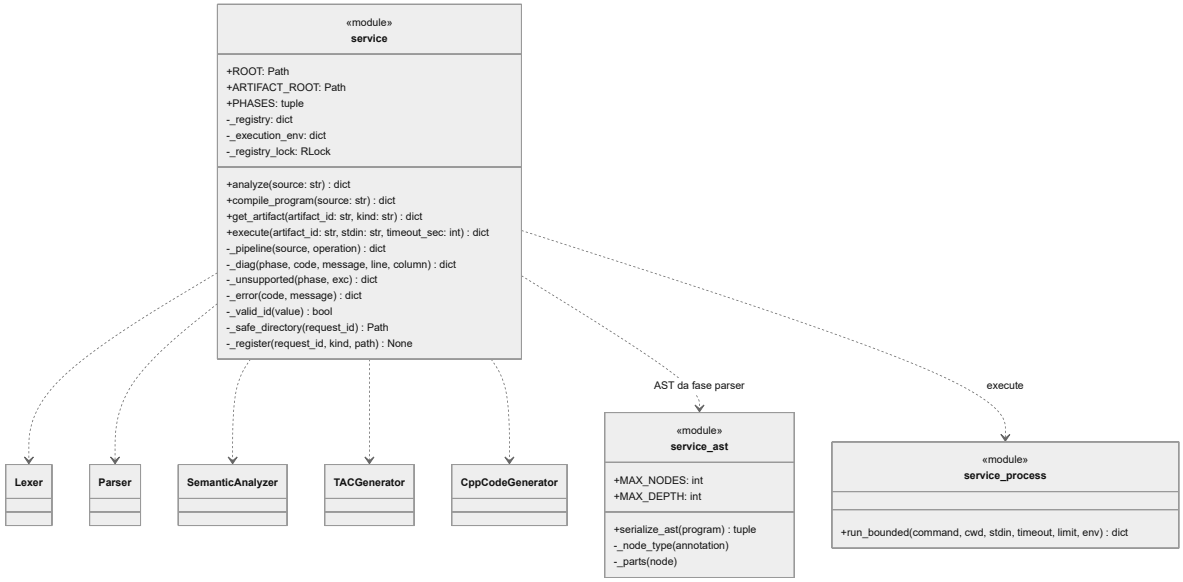
Fonte: Elaboração própria (2026).

Figura 9 — Diagrama de classes do gerador C++



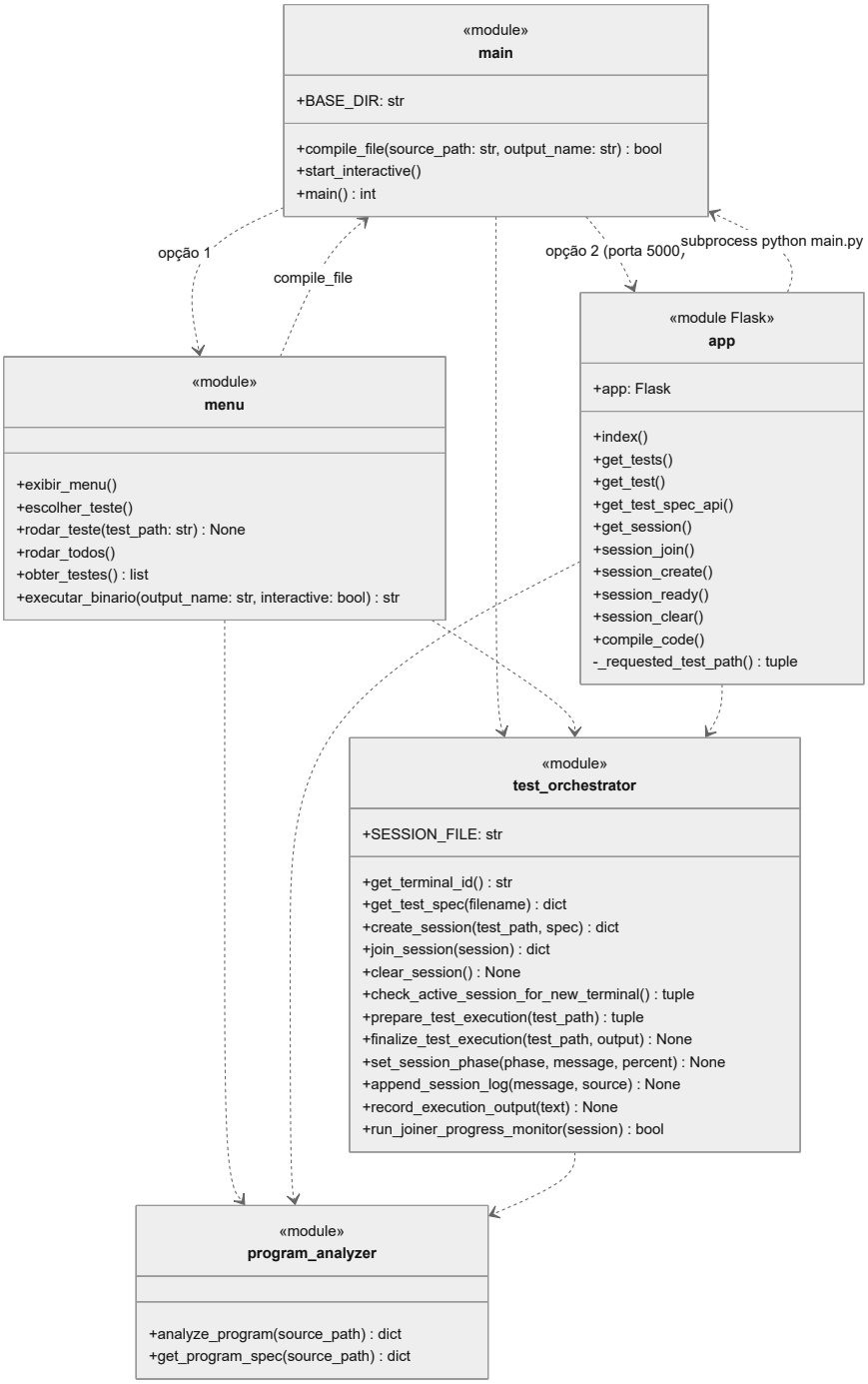
Fonte: Elaboração própria (2026).

Figura 10 — Diagrama de classes do serviço de compilação



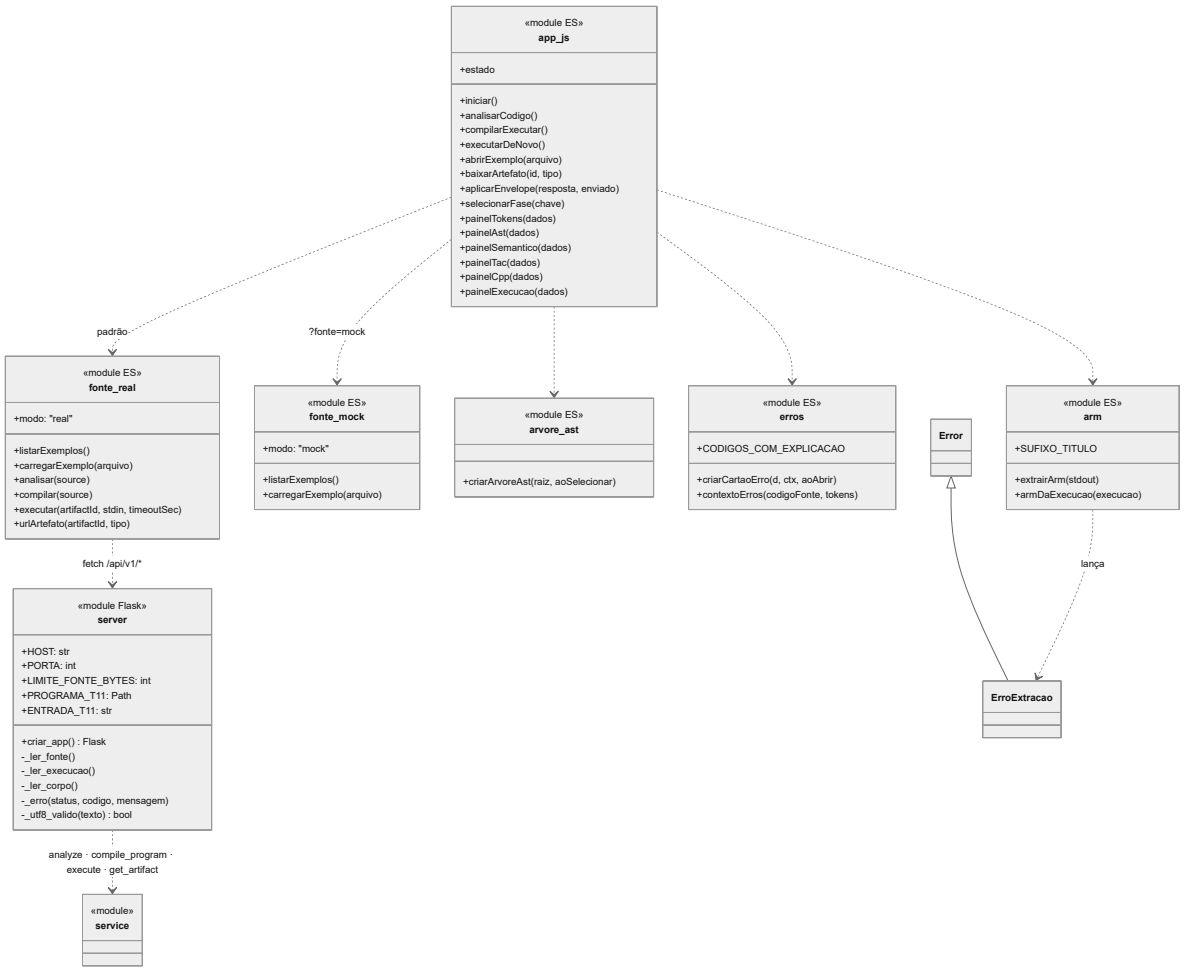
Fonte: Elaboração própria (2026).

Figura 11 — Diagrama de classes da CLI e da Web original



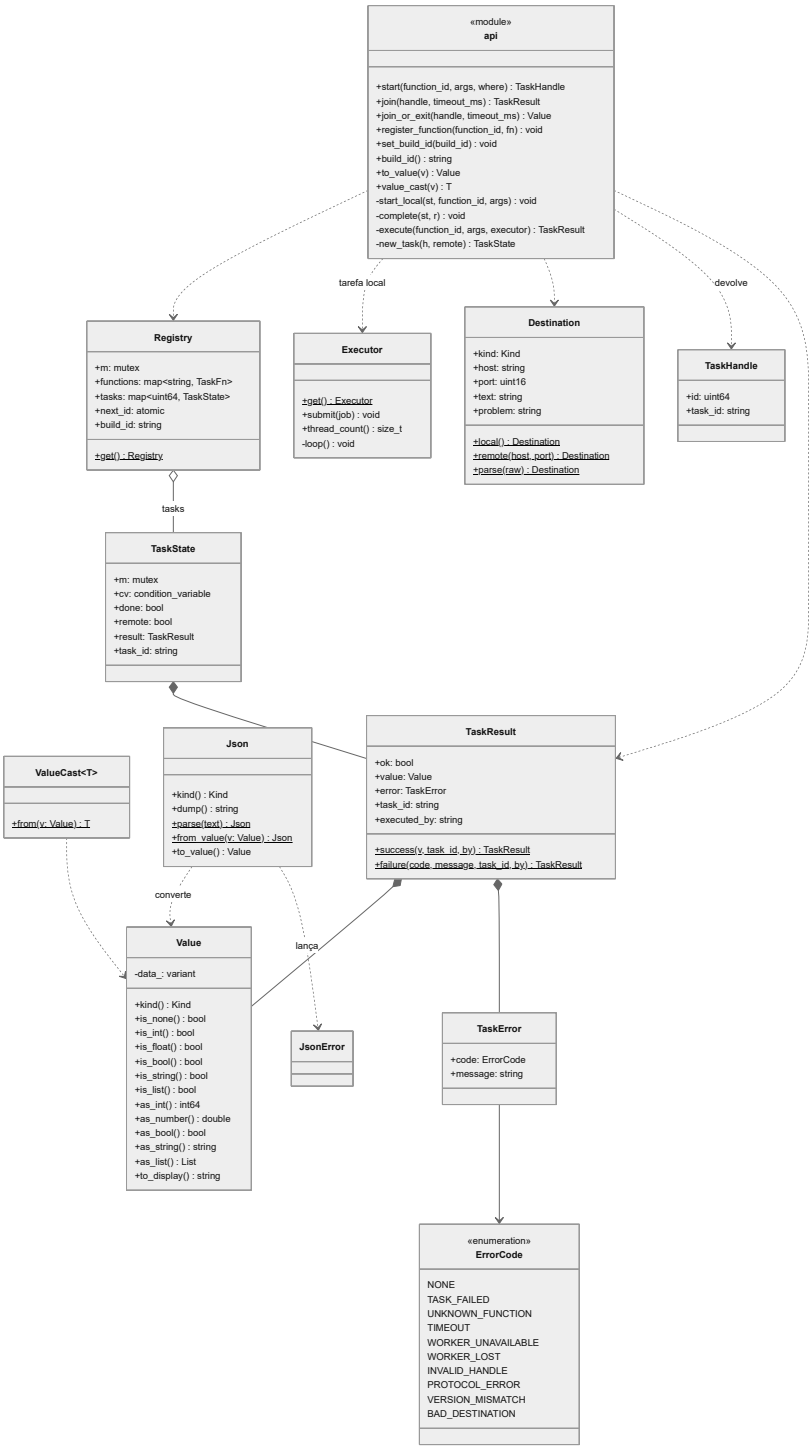
Fonte: Elaboração própria (2026).

Figura 12 — Diagrama de classes da Web didática



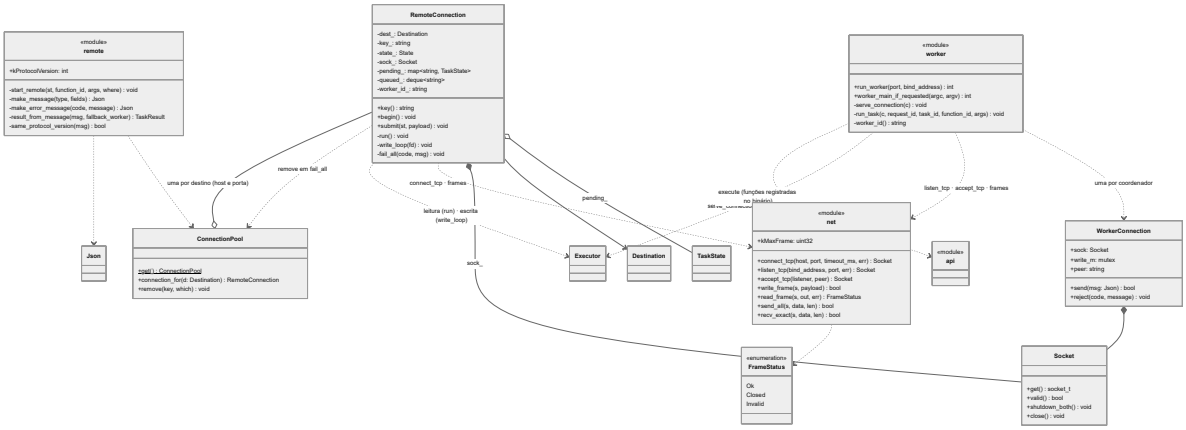
Fonte: Elaboração própria (2026).

Figura 13 — Diagrama de classes do runtime: núcleo



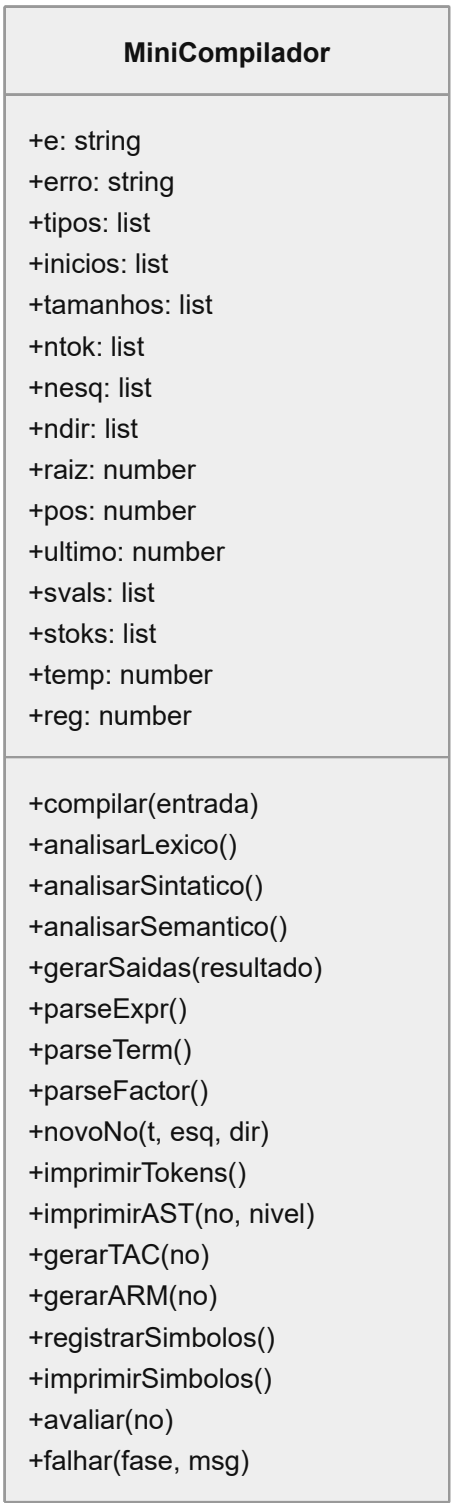
Fonte: Elaboração própria (2026).

Figura 14 — Diagrama de classes do runtime: execução remota



Fonte: Elaboração própria (2026).

Figura 15 — Diagrama de classes do MiniCompilador do Teste 11

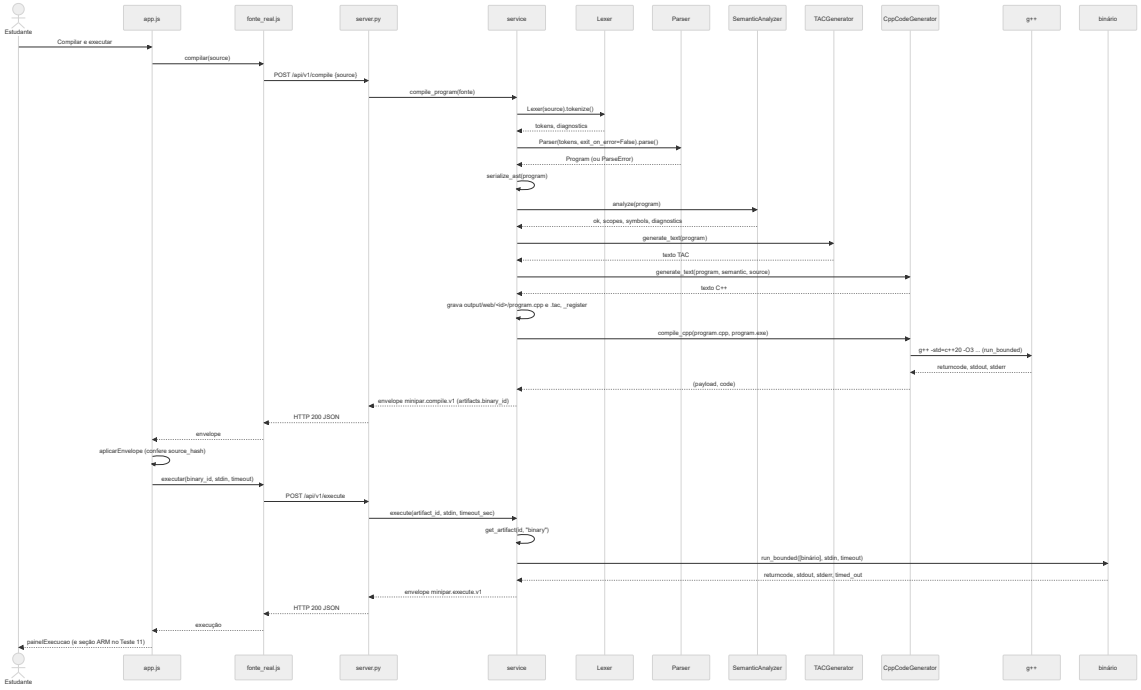


Fonte: Elaboração própria (2026).

4.4 Diagramas de sequência

A Figura 16 mostra uma compilação seguida de execução pela Web didática. Em qualquer fase com erro, o serviço para, devolve o diagnóstico e marca as fases seguintes como não executadas. A Figura 17 mostra um `fork` local e um remoto no binário gerado. A conexão com um destino é criada no primeiro `start` para ele; o `start` devolve o *handle* sem esperar a rede, e as tarefas ficam numa fila até a conexão ficar pronta. No fluxo principal, o `join` gerado usa `join_or_exit`, que encerra o programa com código 1 em caso de erro; dentro de uma tarefa, usa `join` e lança uma exceção, que vira erro da tarefa mãe.

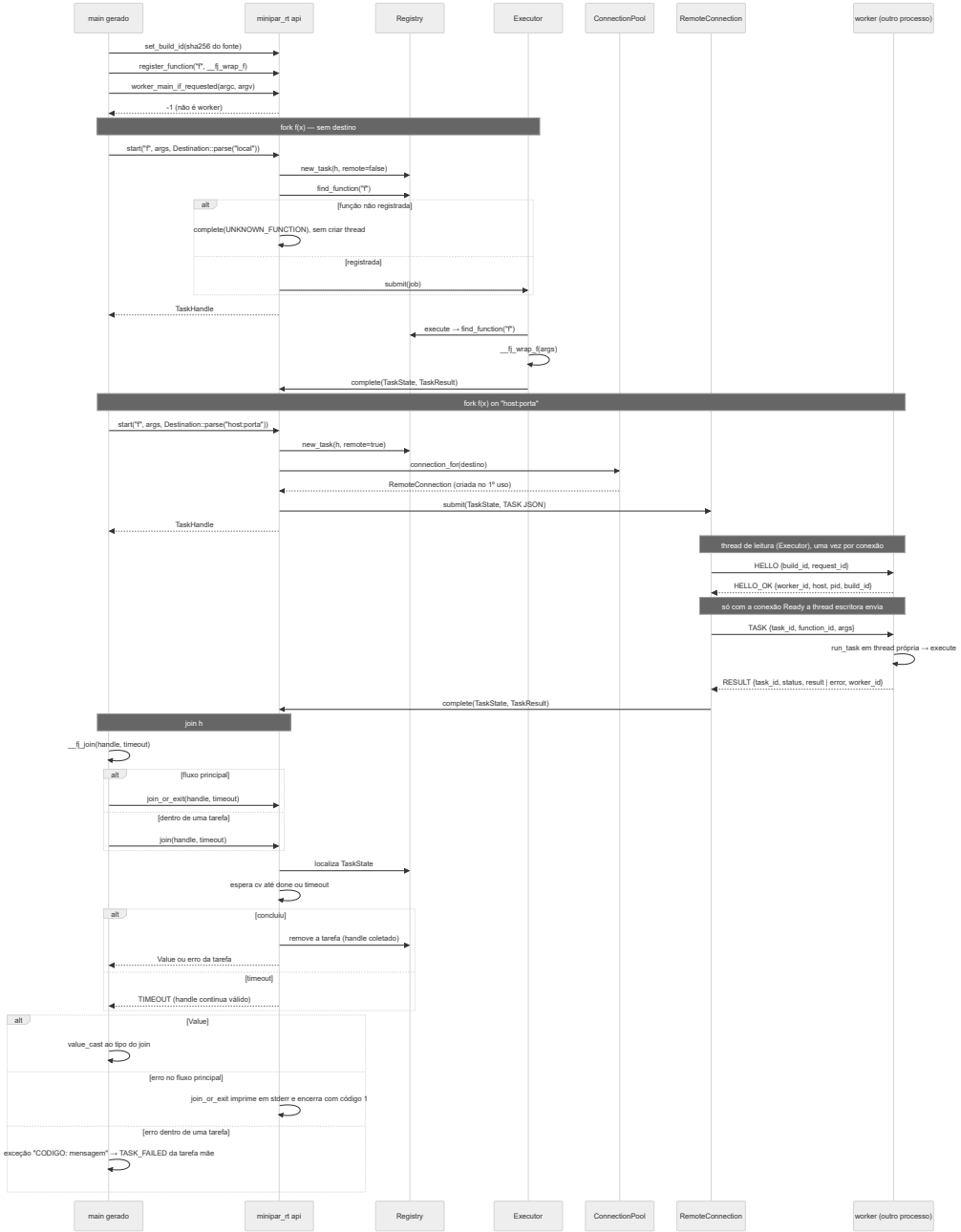
Figura 16 — Sequência de compilação e execução pela Web didática



Fonte: Elaboração própria (2026).

O Quadro 12 indica onde cada componente pedido pelo enunciado foi modelado.

Figura 17 — Sequência de Fork/Join local e remoto no binário gerado



Fonte: Elaboração própria (2026).

Quadro 12 — Componentes exigidos e onde estão modelados

Componente exigido	Componente ou classe	Figuras
Analisador léxico	`Lexer`, `Token`, `TokenType`	2, 3
Analisador sintático	`Parser`, `ParseError`; AST em `ast_nodes`	2, 4, 5, 6
Analisador semântico	`SemanticAnalyzer`, `ValueType`	2, 7
Tabela de símbolos	`Symbol` e a pilha `_scopes`; listas `scopes` e `symbols`	7
Gerador de código de três endereços	`TACGenerator`	2, 8
Gerador de código C++	`CppCodeGenerator`, `CppTaskContext`, `cpp_template`	2, 9
Controle do compilador	`main.compile_file` e `service._pipeline`	10, 11, 16
Execução	`compile_cpp`, `service.execute`, `run_bounded`, `Executor`	9, 10, 13
Comunicação	`MiniParChannel`, `RemoteConnection`, `ConnectionPool`, `WorkerConnection`	9, 14, 17
Interfaces	CLI, Web original e Web didática	1, 2, 11, 12

Fonte: Elaboração própria (2026).

5 IMPLEMENTAÇÃO

Este capítulo descreve a implementação de cada componente. Os pseudocódigos, que seguem o código função por função e citam entre colchetes o método correspondente, estão no Apêndice B. Neles, ← indica atribuição, `nó.campo` o acesso a um campo da AST e `relatar(...)` o registro de um diagnóstico sem interromper a análise.

5.1 Tecnologias e ambiente

O compilador é escrito em Python, com programação orientada a objetos. O código gerado e o runtime de tarefas são escritos em C++20 e compilados pelo GCC. As interfaces Web usam Flask no servidor e HTML, CSS e JavaScript sem *frameworks* no navegador (Quadro 13). O código é comentado nos pontos em que a regra não é evidente pela leitura, como as decisões de tipo do gerador C++, o isolamento das tarefas e o tratamento de falhas do runtime. A Tabela 3 mostra o tamanho de cada parte do código.

Quadro 13 — Tecnologias e ambiente

Aspecto	**Escolha**
Linguagem do compilador	Python 3.12, orientado a objetos
Analisador sintático	Descida recursiva, LL(1) na prática
Representação intermediária	Código de três endereços, gerado da AST
Código-alvo	C++20, compilado pelo GCC 16.2.0 com `-std=c++20 -O3 -pthread` (e `-lws2_32` no Windows)
Runtime de tarefas	`minipar_rt`, C++20 somente com cabeçalhos; threads da biblioteca-padrão e *sockets* Winsock/POSIX
Comunicação distribuída	TCP com mensagens enquadradas (4 bytes de tamanho e JSON)
Interfaces	Terminal (Python), Web original e Web didática (Flask 3.0.0, HTML, CSS e JavaScript)
Teste 11	Programa MiniPar; Assembly ARMv7 executado no CPUlator (Wong, 2026)
Sistema operacional	Windows 11; runtime também testado em Linux

Fonte: Elaboração própria (2026).

Tabela 3 — Tamanho do código por parte do sistema

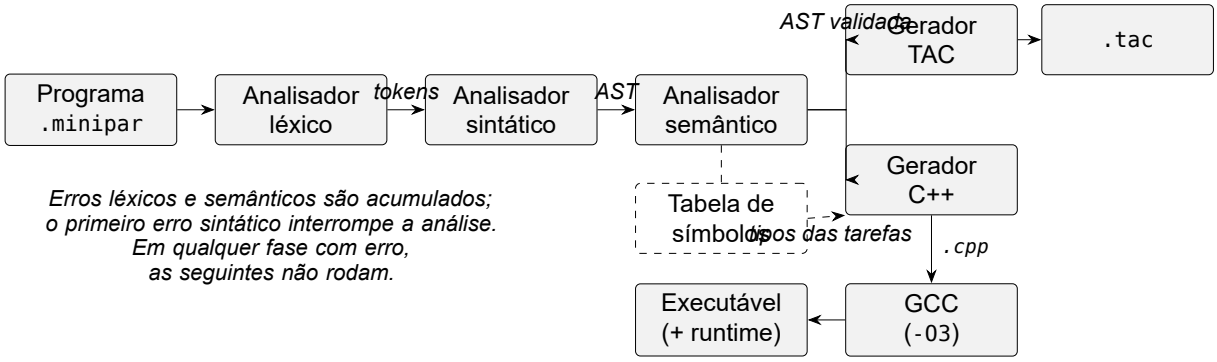
Parte	Linhas
Compilador (`src/`: léxico, sintático, AST, semântico, TAC, C++, serviço)	3 583
Interfaces herdadas (`main.py`, `menu.py`, `app.py` e auxiliares)	1 223
Runtime `minipar_rt` (cabeçalhos C++)	1 985
Testes do runtime (C++ e Python)	1 984
Web didática: servidor e testes em Python	2 121
Web didática: JavaScript, HTML e CSS	1 910
Verificadores automatizados (Python)	5 336
Programas de teste em MiniPar	1 617

Fonte: Elaboração própria (2026). A contagem inclui comentários e linhas em branco.

5.2 Fluxo de compilação

A Figura 18 mostra as fases do compilador e os artefatos que passam entre elas. Cada fase só roda se a anterior terminou sem erro; a partir da AST validada, o gerador TAC e o gerador C++ trabalham em ramos independentes, e o C++ é entregue ao GCC (Código 9). A interface de terminal segue a mesma ordem, mas imprime `[ERRO SINTATICO]` e encerra no primeiro erro.

Figura 18 — Fases do compilador MiniPar 2026.2



Fonte: Elaboração própria (2026).

5.3 Analisador léxico

O analisador léxico (`src/lexer.py`, Código 10) percorre o texto uma vez, mantendo posição, linha e coluna. Em cada passo, pula espaços e comentários e decide a classe do próximo *token* pelo primeiro caractere: letra ou sublinhado inicia identificador ou palavra reservada; dígito inicia número; aspas iniciam string; os demais caracteres são operadores ou delimitadores, com preferência pelo de dois caracteres. O tratamento de comentários exigido pelo enunciado está no procedimento que pula espaços: ao encontrar #, o analisador avança até o fim da linha sem produzir *tokens*. O

analisador nunca aborta: caracteres desconhecidos viram `UNKNOWN` com diagnóstico, e uma string sem fechamento é emitida mesmo assim; quem decide parar é o chamador.

5.4 Analisador sintático e construção da AST

O analisador sintático (`src/parser.py`) é de descida recursiva, com um *token* de antecipação. Suas primitivas consultam o *token* atual, consomem um *token* opcional ou obrigatório e registram erros (Código 11). Cada comando é reconhecido pelo primeiro *token* (Código 12): um comando que começa por identificador, `this`, `new`, `[`, `fork`, `join` ou outra expressão primária é analisado como expressão e, se vier seguido de `=`, vira atribuição; senão, precisa ser chamada, `fork` ou `join`. As declarações, os blocos e o controle de fluxo seguem o Código 13, e as expressões são analisadas por níveis de precedência (Código 14): cada nível chama o superior e, enquanto encontra um operador do seu nível, monta um nó binário com o resultado acumulado à esquerda. Cada produção e o nó que ela constrói estão no Quadro 5.

5.5 Tabela de símbolos

A tabela de símbolos de cada programa é mantida pelo analisador semântico como uma pilha de escopos, cada um um dicionário de nome para símbolo; o escopo de índice zero é o global (Código 15). Cada símbolo guarda nome, categoria (variável, parâmetro, função, classe, atributo ou método), tipo inferido, escopo, linha e, para funções e métodos, a lista de parâmetros. A busca segue o escopo léxico, do mais interno ao global (Código 16). Uma função cria dois escopos: o da função, com os parâmetros, e o do corpo, com as variáveis locais. Sombrear um nome em escopo interno é permitido; redeclará-lo no mesmo escopo, não. Funções e classes globais são coletadas antes da validação, por isso podem ser chamadas antes da declaração. Para as interfaces, o analisador publica a lista de escopos (identificador, tipo, nome, escopo pai e linha) e a de símbolos, que a Web didática desenha como árvore (Figura 24).

5.6 Analisador semântico e tratamento de erros

A análise semântica (`src/semantic.py`) tem dois passos (Código 17): o primeiro coleta as funções e classes globais, verifica nomes reservados e ciclos de herança e marca as funções que contêm `fork` ou `join`; o segundo valida cada comando na ordem do texto, com uma regra por tipo de nó (Código 20). A inferência de tipos é conservadora (Código 19): literais e construtores têm tipo conhecido, comparações produzem `bool`, `+` com string produz string, e o que não pode ser provado fica `unknown`.

Para validar `fork f(...)`, o analisador reanalisa o corpo de `f` e de cada função, método e inicializador alcançado a partir dela, com os tipos dos argumentos e sem

acesso às variáveis locais do chamador; assim, um acesso a global mutável numa função chamada indiretamente também é relatado (Código 21).

O analisador não para no primeiro erro: percorre a AST inteira e acumula os diagnósticos (Código 18), cada um com fase, severidade, código, mensagem, linha e coluna. Os diagnósticos de todas as fases seguem o catálogo do Quadro 14. A interface de terminal os imprime como `[ERRO SEMANTICO] Linha L: mensagem`; a Web didática os exibe como cartões.

Quadro 14 — Catálogo de diagnósticos por fase

Fase	Códigos	Situação
Léxica	`LEX_UNTERMINATED_STRING`, `LEX_UNKNOWN_CHAR`	String sem fechamento; caractere desconhecido
Sintática	`SYN_EXPECTED_TOKEN`, `SYN_EXPECTED_EXPRESSION`, `SYN_INVALID_STATEMENT`, `SYN_UNRECOGNIZED_STATEMENT`, `SYN_INVALID_ASSIGNMENT`	*Token* esperado ausente; expressão ausente; expressão isolada inválida; comando não reconhecido; alvo de atribuição inválido
Semântica	`SEM_*` (Quadro 9) e `SEM_FJ_01` a `SEM_FJ_08` (Quadro 7)	Restrições de contexto e regras de Fork/Join
TAC	`TAC_UNSUPPORTED_FEATURE`	Construção sem tradução para TAC
C++	`CPP_UNSUPPORTED_FEATURE`	Construção sem tradução, ou tipos das tarefas não provados
GCC	`GPP_FAILED`, `GPP_TIMEOUT`, `GPP_NO_BINARY`, `GPP_NOT_FOUND`	Compilação com erro, acima de 120 s, sem binário ou sem compilador
Qualquer	`INTERNAL_ERROR`	Exceção inesperada na fase corrente

Fonte: Elaboração própria (2026).

5.7 Gerador de código intermediário

O gerador TAC (`src/tac.py`) percorre a AST com um método para expressões, que devolve o endereço do resultado, e outro para comandos, que emite instruções. Temporários são criados em sequência (`t0`, `t1`, …) e rótulos recebem o nome da construção e um número (`WHILE_1`, `ENDIF_3`). Atribuições, operações, acessos a lista e chamadas (`param` seguido de `call nome, n`) seguem a forma clássica; `if`, `while`, `do-while` e `for` viram desvios `ifFalse` e `goto`. Os argumentos de um `fork` são copiados com `capture`; um bloco `par` vira ramos rotulados disparados por `par_spawn` e sincronizados por `par_barrier` (Código 22). O Código 2 mostra o TAC dos `fork` e `join` do Teste 8; o TAC completo do Teste 7 está no Código 36.

Código 2 — TAC gerado para os `fork` e `join` do Teste 8

```
t51 = modo == "par"
ifFalse t51 goto ELSE_21
t52 = []
hs = t52
t53 = capture 0
t54 = 0 * fatia
t55 = capture t54
t56 = 0 * fatia
t57 = t56 + fatia
```

```
t58 = capture t57
t59 = capture carga
t60 = d0
t61 = fork avaliar_lote(t53, t55, t58, t59) on t60
param t61
t62 = call hs.append, 1
... (forks dos avaliadores 1 e 2: mesmo padrão)
t91 = fork avaliar_lote(t83, t85, t88, t89) on t90
param t91
t92 = call hs.append, 1
t93 = iter_begin hs
FOR_IN_22:
t94 = iter_has_next t93
ifFalse t94 goto ENDFOR_24
t95 = iter_value t93
h = t95
t96 = join h
param t96
t97 = call resultados.append, 1
FOR_STEP_23:
iter_next t93
goto FOR_IN_22
ENDFOR_24:
goto ENDIF_20
```

Fonte: Elaboração própria (2026).

5.8 Gerador de código C++ e compilação nativa

O gerador C++ (`src/codegen.py` e `src/cpp_template.py`) traduz a AST para C++20. As variáveis MiniPar viram variáveis C++ declaradas com `auto`. O prólogo comum define as funções nativas, a classe `MiniParChannel` dos canais e a impressão atômica: cada `print` monta a linha inteira e a escreve sob um *mutex*, para que linhas de threads diferentes não se misturem. Um bloco `par` vira um vetor de `std::thread`, aguardadas no fim do bloco.

Programas com `fork`/`join` seguem outra rota. Primeiro, `CppTaskContext` prova o tipo C++ concreto de cada função de tarefa, de seus argumentos e de seu retorno; se não conseguir, o programa é recusado com `CPP_UNSUPPORTED_FEATURE`, em vez de gerar conversões silenciosas. Depois, para cada função de tarefa, o gerador produz:

a) uma função C++ tipada com o corpo traduzido;

b) um invólucro que confere a aridade e converte argumentos e retorno;

c) o registro do invólucro no runtime (`register_function`), no início do `main`;

d) o identificador da compilação (SHA-256 do fonte MiniPar), que impede um coordenador de falar com um trabalhador compilado de outro fonte;

e) a chamada que faz o mesmo binário atuar como trabalhador quando recebe `--minipar-worker <porta>`.

Cada `fork` vira `start(id, args, Destination::parse(destino))`, e cada `join` vira uma espera seguida da conversão para o tipo esperado (Código 3). A compilação executa `g++ -std=c++20 -O3 programa.cpp -o programa.exe -pthread -I runtime/include`, com limite de 120 segundos; quando o limite estoura, a árvore de processos inteira é encerrada.

Código 3 — Trechos do C++ gerado para o Teste 8

```
// No coordenador usa a API contratada; dentro de tarefa, preserva o worker.
inline minipar_rt::Value __fj_join(const minipar_rt::TaskHandle& h, int timeout) {
    if (!__fj_in_task) return minipar_rt::join_or_exit(h, timeout);
    auto r = minipar_rt::join(h, timeout);
    if (!r.ok) throw std::runtime_error(std::string(minipar_rt::to_string(r.error.code)) + ": " +
    ↪  r.error.message);
    return std::move(r.value);
}
    // ...
minipar_rt::Value __fj_wrap_avaliar_lote(const minipar_rt::TaskArgs& a) {
    if (a.size() != 4) throw std::invalid_argument("aridade de avaliar_lote");
    __fj_scope scope(true);
    return minipar_rt::to_value(__fj_fn_4(minipar_rt::value_cast<int>(a.at(0)),
    ↪  minipar_rt::value_cast<int>(a.at(1)), minipar_rt::value_cast<int>(a.at(2)),
    ↪  minipar_rt::value_cast<int>(a.at(3))));
}
int main(int argc, char** argv) {
    minipar_rt::set_build_id("69d0f55f9f4d9e149f97818b34ca455aecb1515e1567660c8044069f843759b1");
    minipar_rt::register_function("avaliar_lote", __fj_wrap_avaliar_lote);
    if (int rc = minipar_rt::worker_main_if_requested(argc, argv); rc >= 0) return rc;
    // ...
        hs.push_back(([&]() { auto __fj_45_0 = minipar_rt::to_value(0); auto __fj_45_1 =
        ↪  minipar_rt::to_value(([&]() { auto __fj_42_0 = 0; auto __fj_42_1 = fatia; return
        ↪  (__fj_42_0 * __fj_42_1); })())); auto __fj_45_2 = minipar_rt::to_value(([&]() { auto
        ↪  __fj_44_0 = ([&]() { auto __fj_43_0 = 0; auto __fj_43_1 = fatia; return (__fj_43_0 *
        ↪  __fj_43_1); })()); auto __fj_44_1 = fatia; return (__fj_44_0 + __fj_44_1); }())); auto
        ↪  __fj_45_3 = minipar_rt::to_value(carga); auto __fj_45_4 = d0; return
        ↪  minipar_rt::start("avaliar_lote", {__fj_45_0, __fj_45_1, __fj_45_2, __fj_45_3},
        ↪  minipar_rt::Destination::parse(__fj_45_4)); })()));
    // ...
            resultados.push_back(([&]() { auto __fj_58_0 = h; auto __fj_58_1 = -1; return
            ↪  minipar_rt::value_cast<std::vector<double>>(__fj_join(__fj_58_0, __fj_58_1));
            ↪  })()));
```

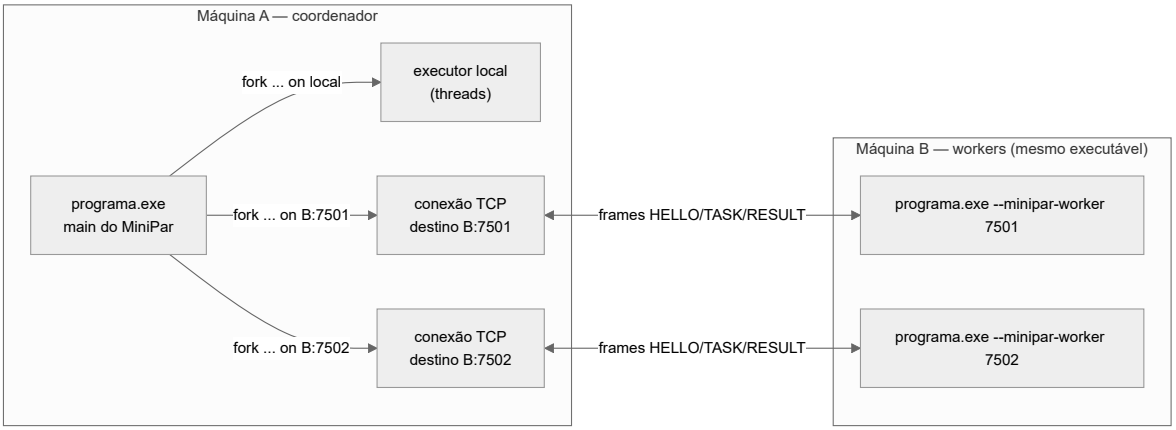
Fonte: Elaboração própria (2026).

5.9 Runtime de tarefas e execução distribuída

O runtime `minipar_rt` é uma biblioteca C++20 somente de cabeçalhos, incluída no binário do programa. Sua API tem quatro operações: `register_function` registra uma função executável como tarefa; `start` dispara uma tarefa e devolve um *handle* sem esperar o cálculo nem a rede; `join` espera o resultado, com limite de tempo opcional, e devolve sucesso ou erro; e `join_or_exit` faz o mesmo, mas encerra o programa em caso de erro. No destino `local`, a tarefa vai para um executor que cresce sob demanda, de modo que tarefas simultâneas recebem threads distintas e rodam de fato em paralelo.

O programa compilado é um único executável com dois papéis (Figura 19): sem argumentos, é o coordenador, que roda o programa MiniPar e chama `start` a cada `fork`; com `--minipar-worker <porta>`, vira trabalhador, escuta a porta e só executa as funções registradas nele. Não há servidor central nem código enviado pela rede.

Figura 19 — Topologia da execução distribuída



Fonte: Elaboração própria (2026).

Cada destino `host:porta` corresponde a uma conexão TCP persistente, aberta no primeiro `start` para ele e reaproveitada. Cada mensagem é um quadro com quatro bytes de tamanho seguidos de um JSON em UTF-8, de até 8 MiB (Quadro 15). Uma thread escritora por conexão envia as tarefas em ordem, e uma thread leitora entrega cada resultado ao *handle* correspondente, em qualquer ordem de término. Todas as falhas são explícitas e nenhuma trava o programa (Quadro 16). O trabalhador não tem autenticação nem criptografia e é destinado a redes locais de laboratório.

Quadro 15 — Mensagens do protocolo do runtime

Mensagem	Campos principais	Função
`HELLO`	`build_id`, `request_id`	Abre o diálogo com o identificador da compilação
`HELLO_OK`	`worker_id`, `build_id`	O trabalhador aceita; o coordenador espera até 5 s
`TASK`	`task_id`, `function_id`, `args`	Pede a execução de uma função registrada
`RESULT`	`task_id`, `status`, resultado	Devolve o resultado da tarefa
`ERROR`	código e mensagem	Informa erro de protocolo e fecha a conexão

Fonte: Elaboração própria (2026).

Quadro 16 — Erros que o `join` pode devolver

Código	Quando acontece	Exemplo de causa
`BAD_DESTINATION`	Destino que não é `local` nem `IPv4:porta`	Erro de digitação no destino
`WORKER_UNAVAILABLE`	Conexão recusada ou *handshake* sem resposta	Trabalhador não iniciado ou porta errada
`VERSION_MISMATCH`	Identificador da compilação ou versão do protocolo diferente	Trabalhador compilado de outro fonte
`WORKER_LOST`	A conexão caiu com a tarefa pendente	Trabalhador encerrado durante a tarefa
`TIMEOUT`	O `join` passou do limite (60 s por padrão no remoto)	Trabalhador vivo que não responde
`TASK_FAILED`	A função lançou exceção ou o resultado passou de 8 MiB	Divisão por zero na tarefa
`UNKNOWN_FUNCTION`	Função não registrada no trabalhador	Binários diferentes
`PROTOCOL_ERROR`	Quadro ou JSON inválido	Outro serviço na mesma porta
`INVALID_HANDLE`	`join` de *handle* já coletado ou desconhecido	Erro de programa

Fonte: Elaboração própria (2026).

5.10 Serviço de compilação

O serviço de compilação (`src/service.py`) liga o compilador à Web didática. Ele oferece `analyze`, que roda as fases do léxico ao C++ sem o GCC; `compile_program`, que roda todas as fases e produz o binário; `execute`, que executa o binário com entrada e limite de tempo; e `get_artifact`, que devolve o `.cpp`, o `.tac` ou o binário. A resposta é um JSON com o estado de cada fase, os *tokens*, a AST, os escopos e símbolos, o TAC, o C++, os diagnósticos e o SHA-256 do código analisado, que a Web usa para descartar respostas de um código já alterado. Cada compilação usa uma pasta própria, para que compilações simultâneas não se misturem.

5.11 MiniCompilador do Teste 11

O Teste 11 pede um compilador de expressões aritméticas escrito na própria MiniPar, que mostre *tokens*, AST, TAC, tabela de símbolos, resultado e um programa Assembly ARMv7 que rode no CPUlator e deixe o resultado num registrador. A classe `MiniCompilador` é compilada pelo compilador hospedeiro, e o binário faz todo o trabalho; nenhuma fase do Teste 11 é feita em Python. Como listas de strings não são suportadas, *tokens* e nós ficam em listas paralelas de números, e todo texto é recuperado da entrada com `substr` (Quadro 17).

O programa executa as fases em ordem e para na primeira falha (Código 23). O léxico aceita inteiros não negativos sem zero à esquerda, `+`, `-`, `*`, parênteses, espaços e comentários (Código 24); a análise sintática é de descida recursiva (Códigos 25 e 26). A análise semântica garante um Assembly correto: constantes e resultados intermediários cabem em 32 bits com sinal, e a AST cabe nos treze registradores `r0` a `r12` (Código 27); a tabela de símbolos guarda as constantes distintas (Código 28). Por fim,

Quadro 17 — Estruturas de dados do MiniCompilador

Campo	Conteúdo
`e`, `erro`	A expressão lida e a primeira mensagem de erro
`tipos[t]`, `inicios[t]`, `tamanhos[t]`	*Token* `t`: tipo (0 número, 1 operador, 2 parêntese), posição e tamanho do lexema
`ntok[n]`, `nesq[n]`, `ndir[n]`	Nó `n` da AST: *token* e filhos esquerdo e direito (`-1` indica ausência)
`raiz`, `pos`, `ultimo`	Nó raiz, *token* atual e último nó construído
`svals[j]`, `stoks[j]`	Tabela de símbolos: valor da constante e *token* da primeira ocorrência
`temp`, `reg`	Próximo temporário do TAC e próximo registrador

Fonte: Elaboração própria (2026).

o programa gera o TAC e o Assembly em pós-ordem (Código 29): constantes até 255 usam `MOV rN, #k`, as maiores usam `LDR rN, =k`, e o programa termina num laço `b .`, onde o simulador é parado.

6 INTERFACES DO COMPILADOR

O enunciado pede que as interfaces de terminal e Web existentes sejam mantidas e que uma segunda interface Web, de caráter didático, seja acrescentada, com uma representação clara de cada fase e, em especial, da AST (Hernández Domínguez, 2026). As saídas e telas deste capítulo foram produzidas pelo compilador real.

6.1 Interface de terminal

A interface de terminal é iniciada por `python main.py`. O programa pergunta o modo de uso: terminal, com um menu dos programas de teste, ou Web, que inicia a IDE original. Também é possível compilar um arquivo diretamente, com `python main.py <arquivo> <saída>`: o compilador informa cada fase, grava o TAC e o C++ em `output/` e chama o GCC (Código 4). Em caso de erro, a CLI imprime a fase e a linha, como `[ERRO LEXICO] Linha 1: String sem fechamento`, e não chama as fases seguintes. O terminal também permite rodar o Teste 1 com dois terminais, um como servidor e outro como cliente.

Código 4 — Compilação e execução do Teste 7 pela linha de comando

```
> python main.py tests/teste7_quicksort_iterativo_tac.minipar t07
```

```
=== Compilador MiniPar ===

=== Compilando: tests/teste7_quicksort_iterativo_tac.minipar ===
[Fase 1] Analise Lexica e Sintatica...
[Fase 2] Analise Semantica e Tabela de Simbolos...
[Fase 3] Geracao de Codigo Intermediario (TAC - 3 enderecos)...
[TAC] Arquivo gerado: output\t07.tac
[Fase 4] Transpilacao e Chamada do GCC (-O3)...
[CODEGEN] Comando: ['g++', '-std=c++20', '-O3', '<repo>\\output\\t07.cpp', '-o',
↪  '<repo>\\output\\t07.exe', '-pthread', '-I', '<repo>\\runtime\\include', '-lws2_32']
[SUCESSO] Executavel: output\t07.exe
=== Processo Finalizado ===
```

```
> output\t07.exe
```

```
Array ordenado: [5, 12, 33, 61, 98]
```

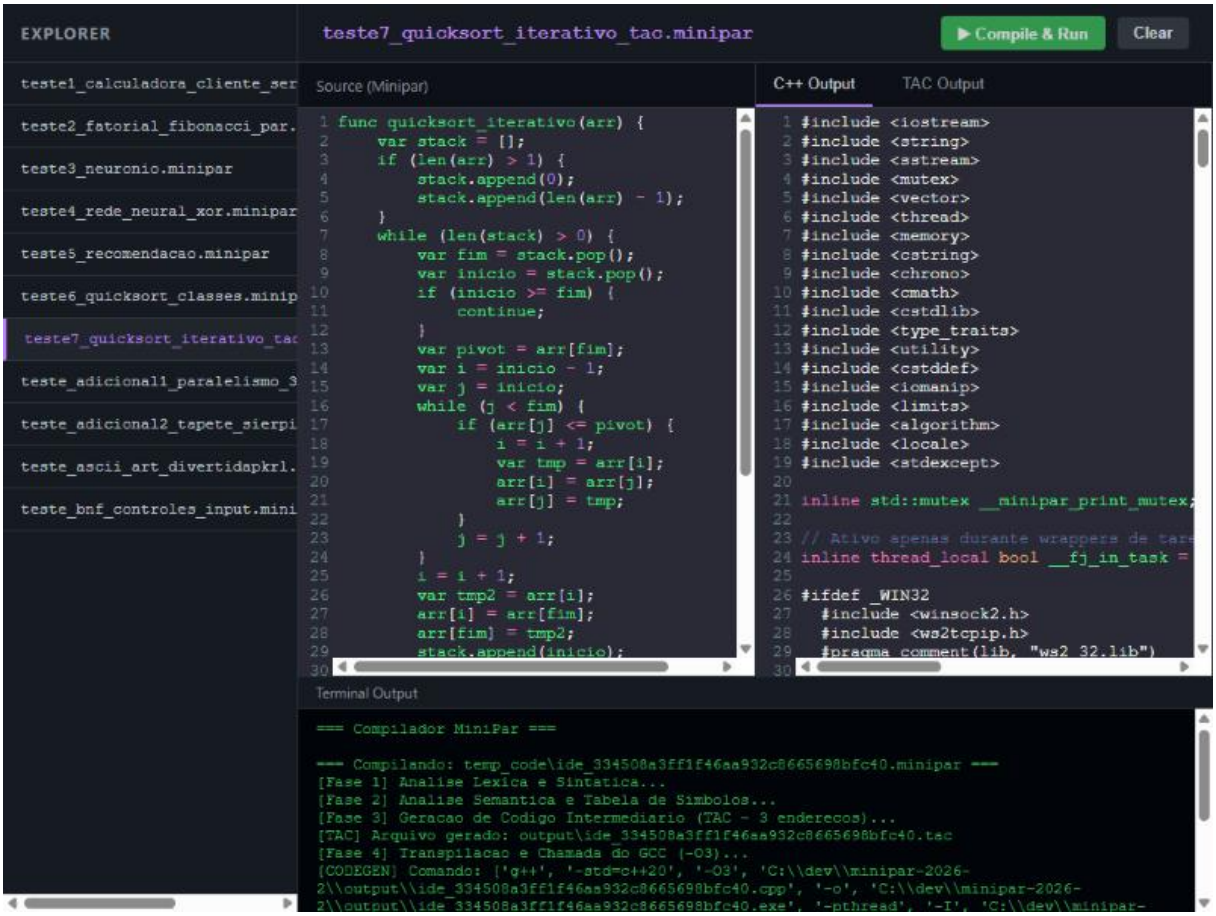
Fonte: Elaboração própria (2026); o caminho do repositório foi substituído por `<repo>`.

6.2 Interface Web original

A Web original é a IDE herdada da versão 2026.1, em http://127.0.0.1:5000. Ela tem um explorador com os programas de teste, um editor, um botão para compilar

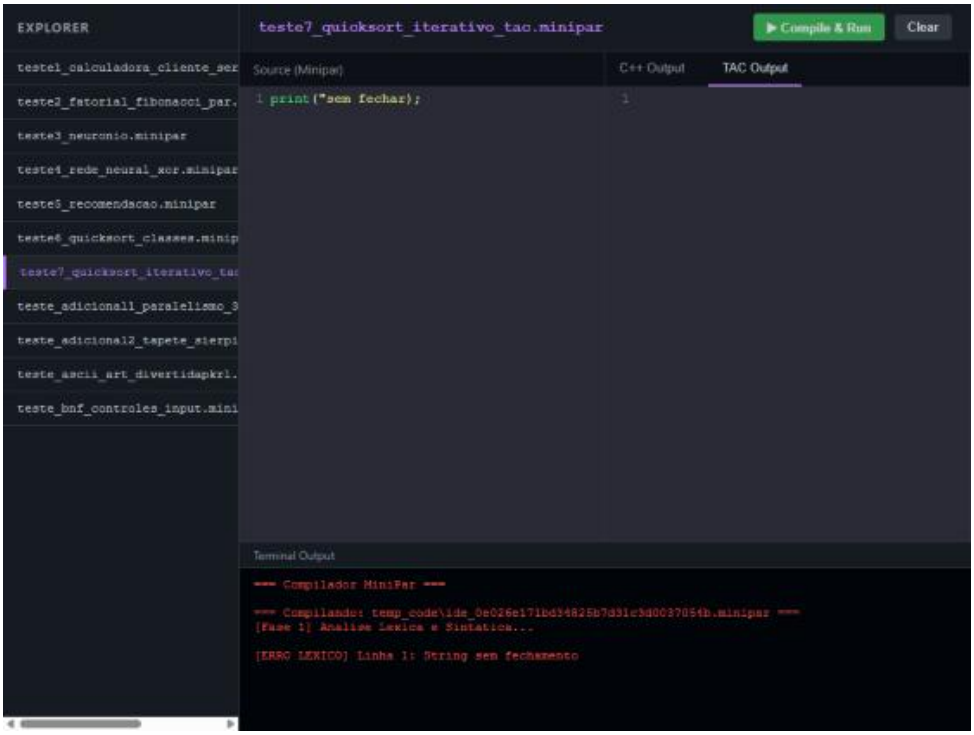
e executar e abas com a saída, o C++ e o TAC (Figura 20). Para compilar, o servidor grava o código num arquivo temporário e chama a CLI como subprocesso. Um código com erro mostra a mensagem do compilador e deixa as abas de C++ e TAC vazias (Figura 21).

Figura 20 — Web original: Teste 7 compilado e executado



Fonte: Elaboração própria (2026).

Figura 21 — Web original: erro léxico de string sem fechamento



Fonte: Elaboração própria (2026).

6.3 Interface Web didática

A Web didática mostra cada fase da compilação real de um programa: *tokens*, AST, escopos e símbolos, TAC, C++ e, sob pedido, a compilação pelo GCC e a execução do binário com entrada e limite de tempo. A página conversa por JSON com um servidor Flask (`web_didatica/server.py`), que escuta só no próprio computador, na porta 5001, e chama o serviço de compilação (Quadro 18). Para iniciá-la, executa-se `python web_didatica/server.py` e abre-se http://127.0.0.1:5001/.

Quadro 18 — Rotas da Web didática e seu uso na tela

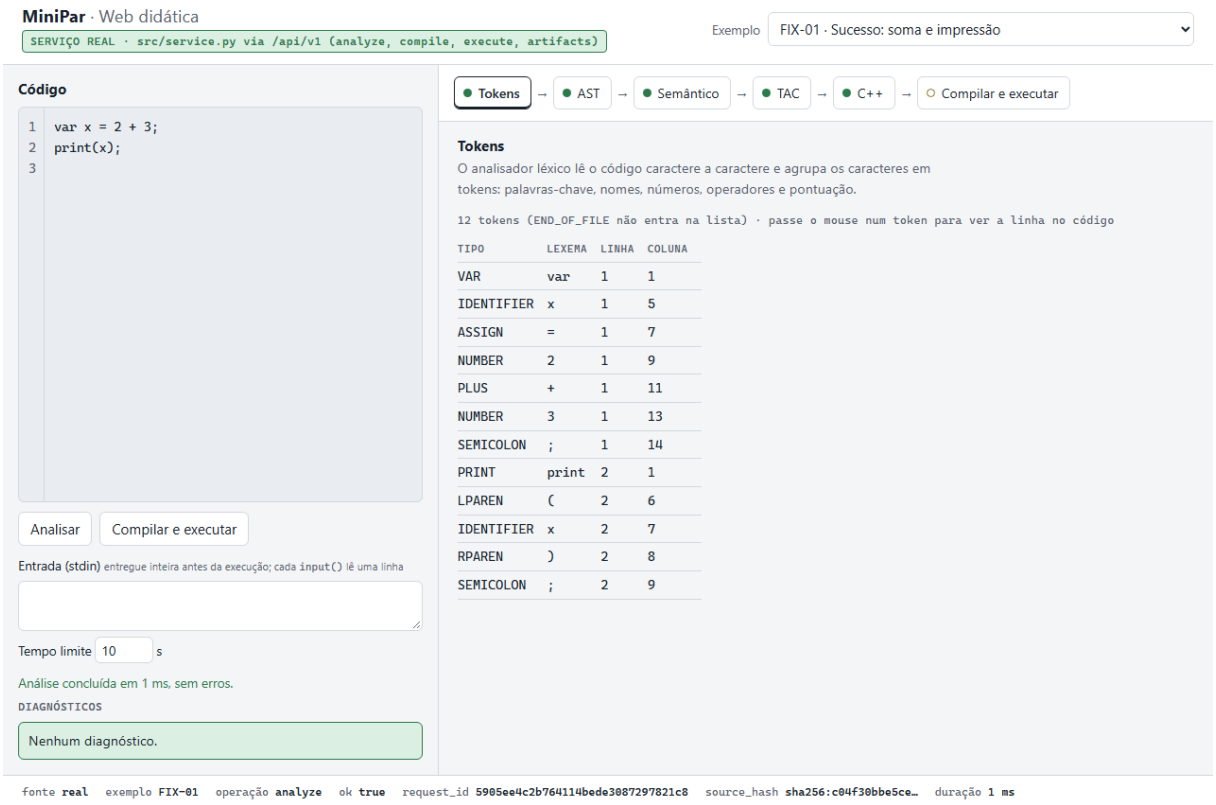
Rota	Uso na tela
`POST /api/v1/analyze`	Ao abrir um exemplo, no botão Analisar ou com Ctrl+Enter: fases do léxico ao C++
`POST /api/v1/compile`	Botão Compilar e executar: todas as fases, com o GCC
`POST /api/v1/execute`	Logo após uma compilação bem-sucedida e no botão Executar de novo
`GET /api/v1/artifacts/<id>/<tipo>`	Botões Baixar `.cpp`, `.tac` e binário

Fonte: Elaboração própria (2026).

À esquerda, a tela tem o editor, os botões, a entrada do programa, o tempo limite e os diagnósticos; à direita, a trilha de fases, cada uma com seu estado, e o painel da

fase escolhida. As Figuras 22 a 26 mostram a análise de `var x = 2 + 3; print(x);`. A primeira fase lista os *tokens* com tipo, lexema, linha e coluna. A AST é uma árvore que se abre e fecha por nó; ao clicar num nó, um quadro mostra tipo, papel no pai, linha, filhos, atributos e caminho até a raiz, e a linha fica destacada no editor, o que atende ao pedido de melhorar a visualização da AST. A fase semântica mostra os escopos aninhados com seus símbolos, e as fases TAC e C++ mostram o código gerado.

Figura 22 — Web didática: tabela de *tokens*

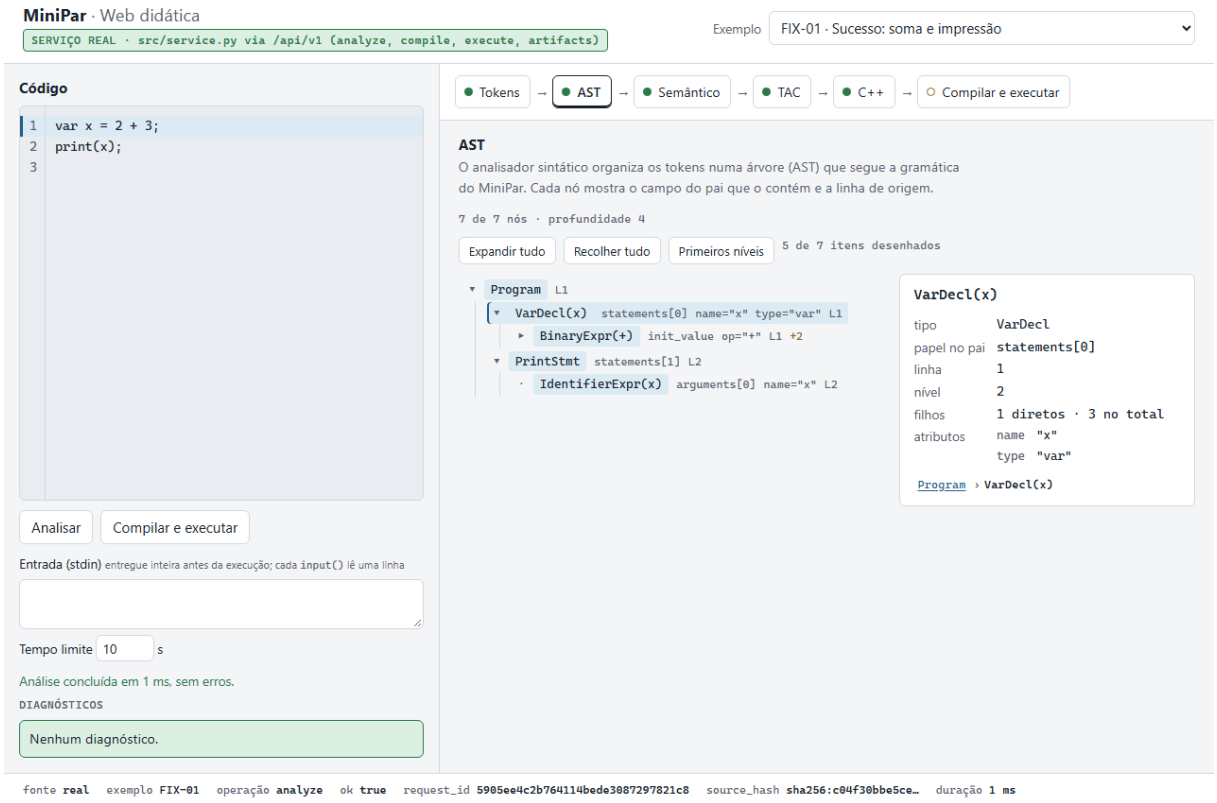


Fonte: Elaboração própria (2026).

O botão Compilar e executar envia o código ao serviço e, com sucesso, executa o binário daquela compilação com a entrada e o tempo limite escolhidos (Figura 27); abaixo ficam a saída do programa e os botões de download. Cada erro aparece como um cartão com fase, linha e coluna, o trecho do código com a coluna marcada, uma explicação em português e o código do catálogo (Figuras 28 a 30). No Teste 11, a saída mostra as fases geradas pelo MiniCompilador, e o botão Baixar `.arm` recorta da saída o bloco de Assembly que será aberto no CPUlator (Figura 31).

O Quadro 19 compara as três interfaces.

Figura 23 — Web didática: AST com o nó `VarDecl(x)` selecionado



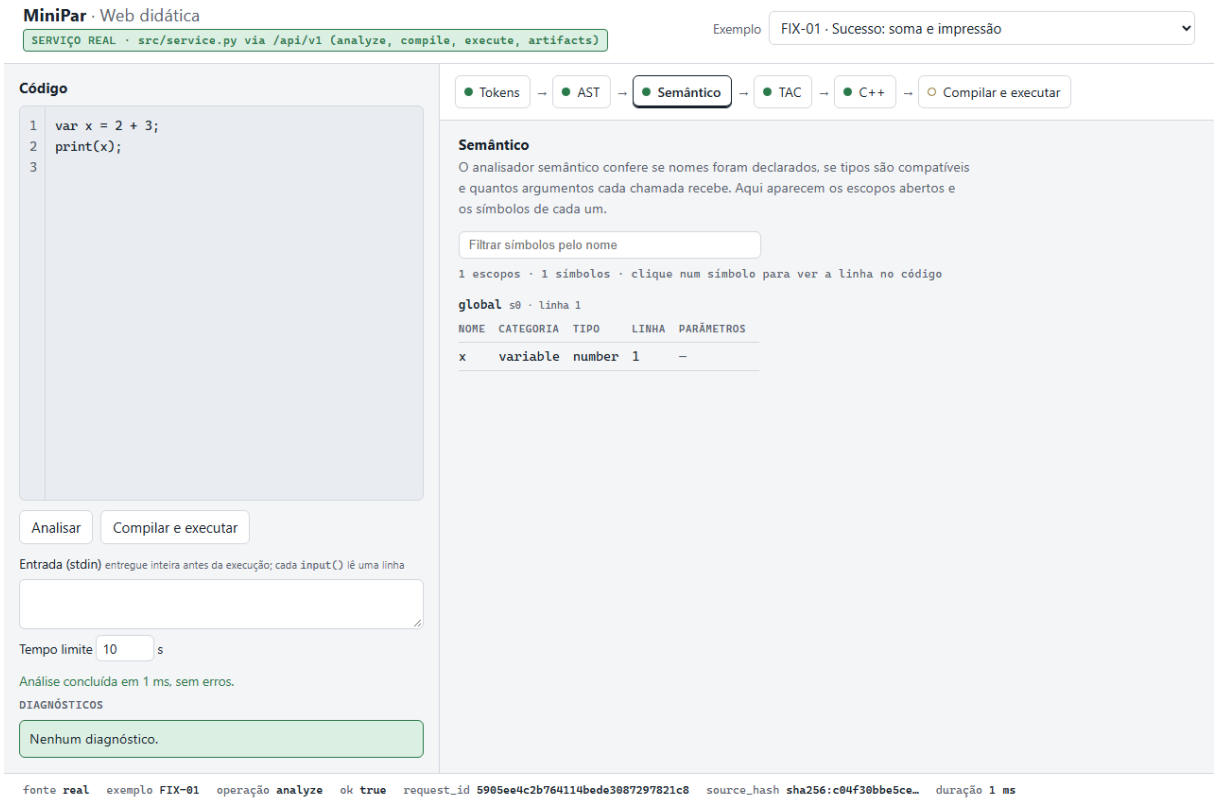
Fonte: Elaboração própria (2026).

Quadro 19 — Comparação entre as interfaces

Aspecto	Terminal	Web original	Web didática
Início	`python main.py`, opção 1, ou CLI direta	`python main.py`, opção 2 (porta 5000)	`python web_didatica/server.py` (porta 5001)
Como compila	Chama as fases diretamente	Chama a CLI por subprocesso	Chama o serviço de compilação
O que mostra	Mensagens das fases e saída; TAC e C++ em arquivos	Saída, C++ e TAC em abas	*Tokens*, AST interativa, escopos e símbolos, TAC, C++, execução e downloads
Erros	Mensagem com fase e linha	Mensagem do compilador	Cartão com fase, linha, coluna, trecho e explicação

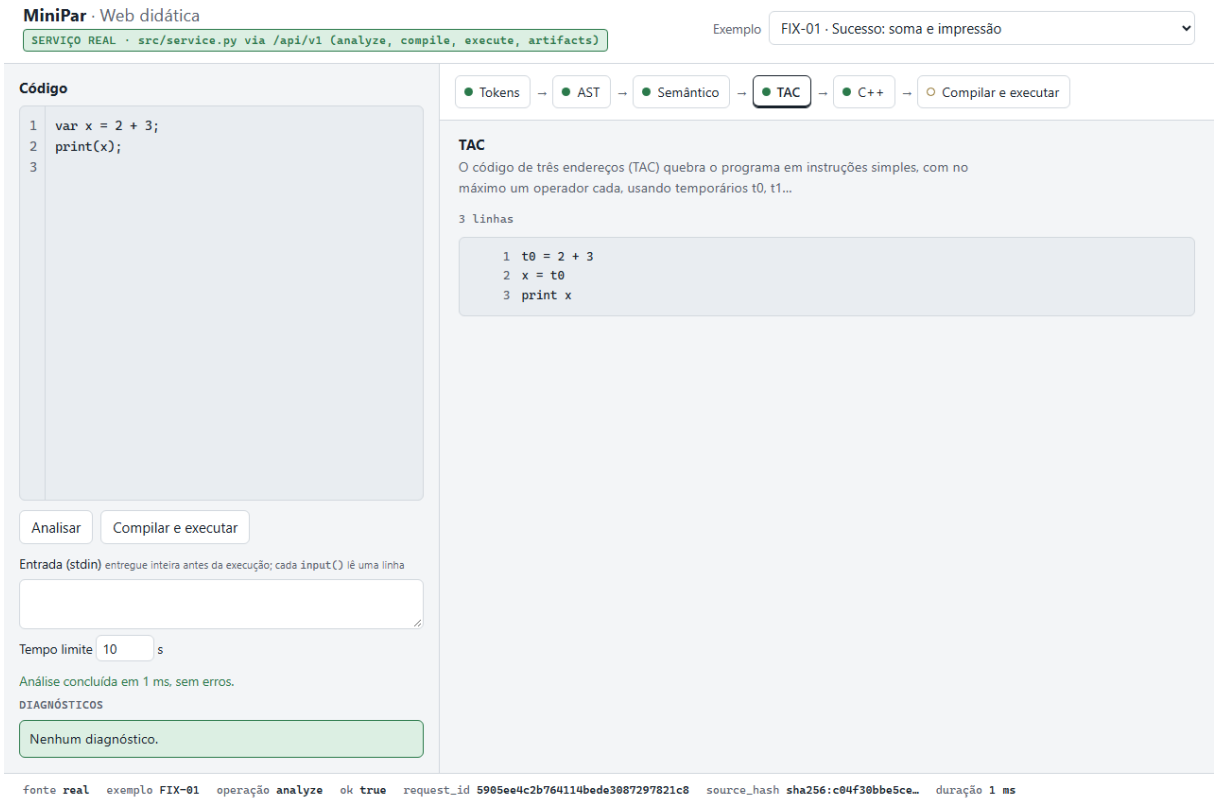
Fonte: Elaboração própria (2026).

Figura 24 — Web didática: escopos e símbolos



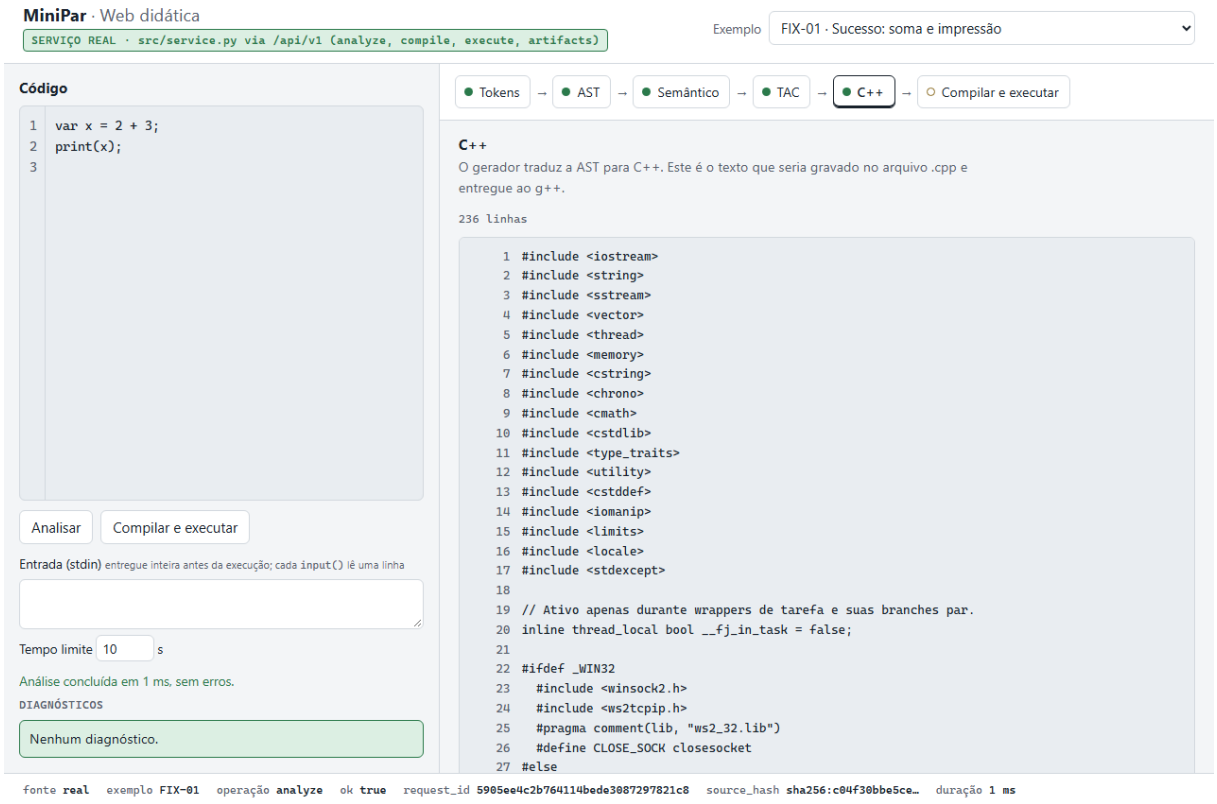
Fonte: Elaboração própria (2026).

Figura 25 — Web didática: código de três endereços



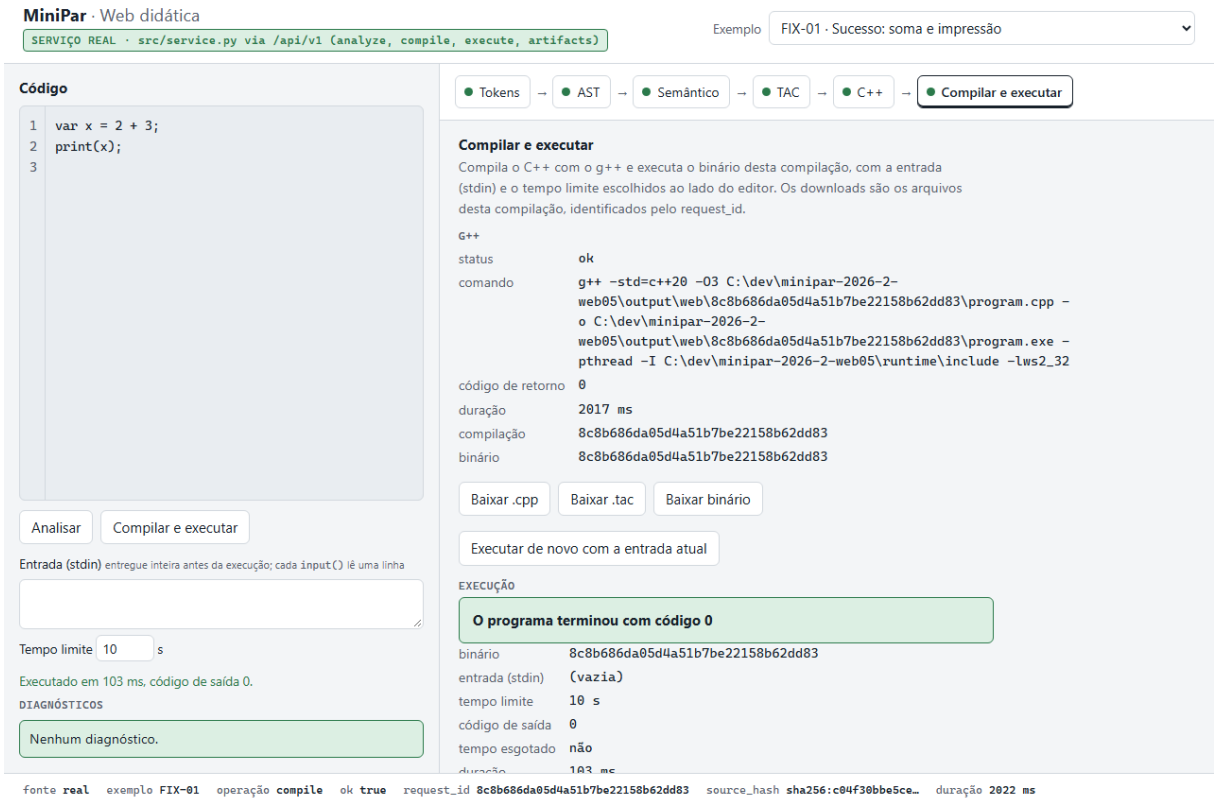
Fonte: Elaboração própria (2026).

Figura 26 — Web didática: C++ gerado



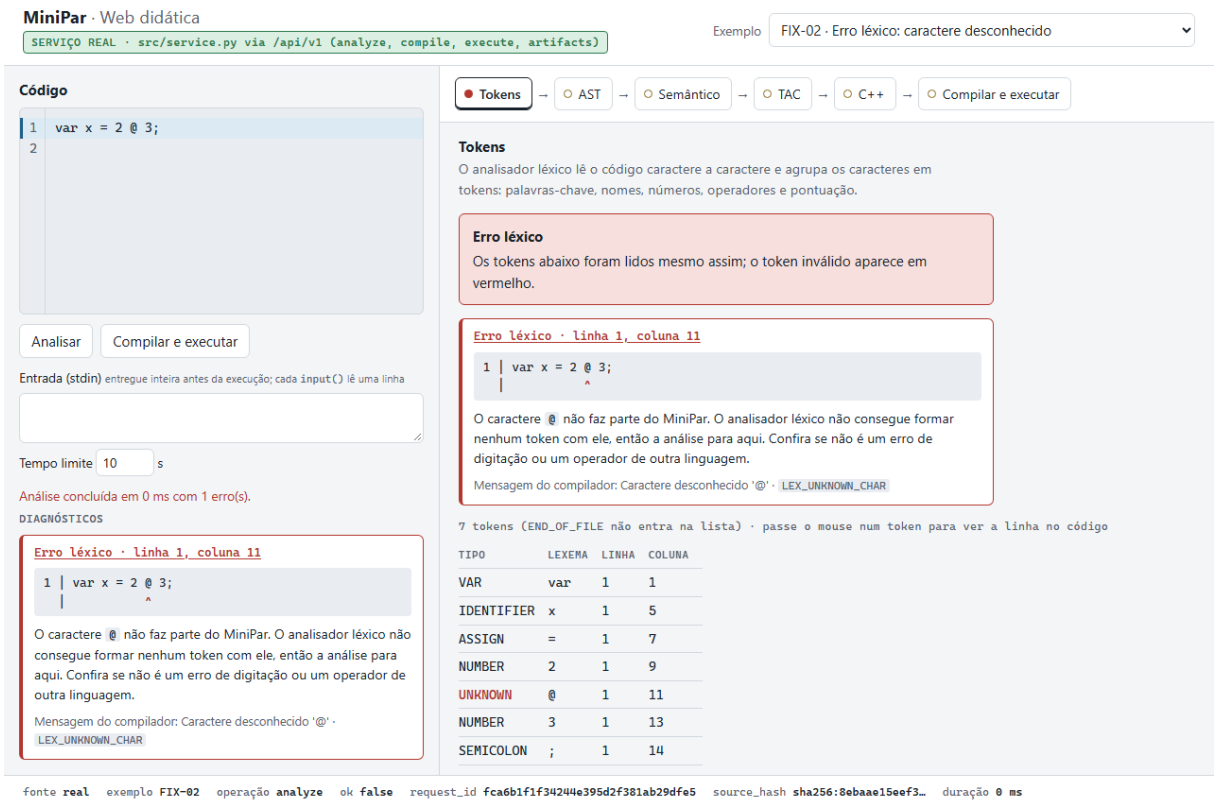
Fonte: Elaboração própria (2026).

Figura 27 — Web didática: compilação pelo GCC e execução



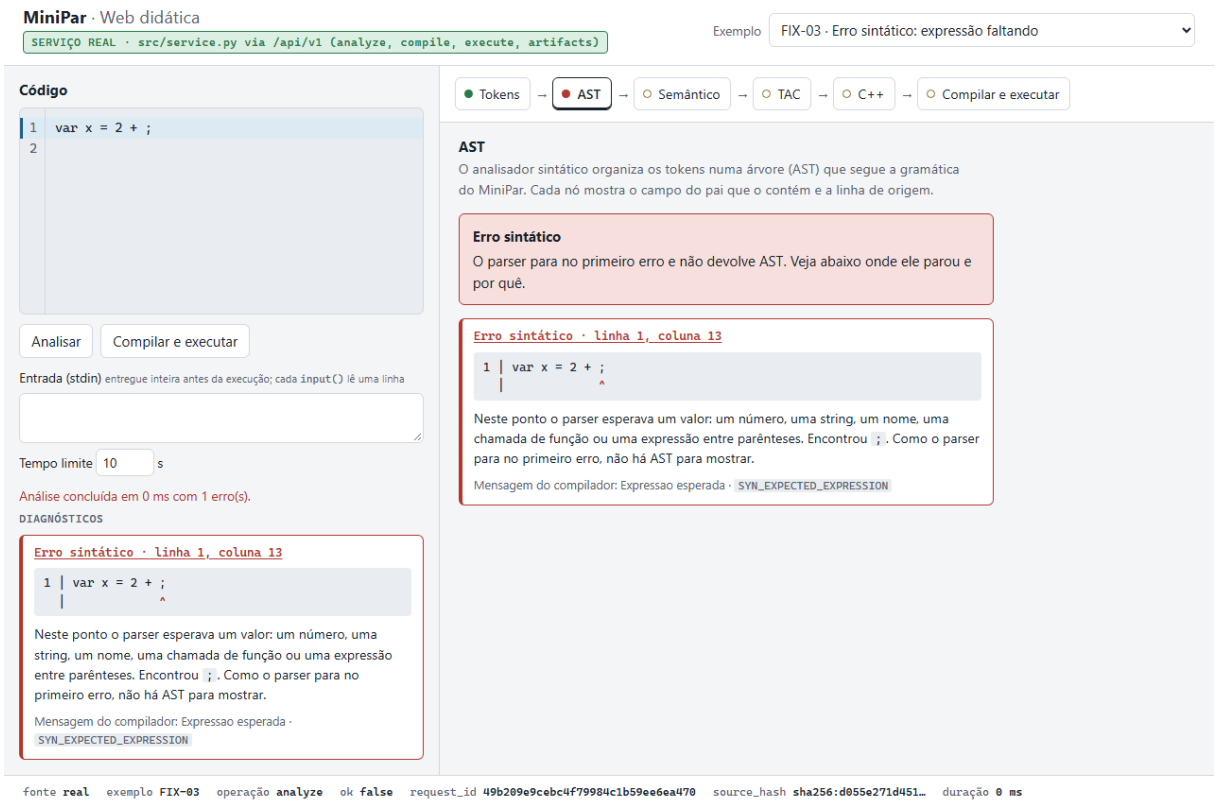
Fonte: Elaboração própria (2026).

Figura 28 — Web didática: erro léxico (`var x = 2 @ 3;`)



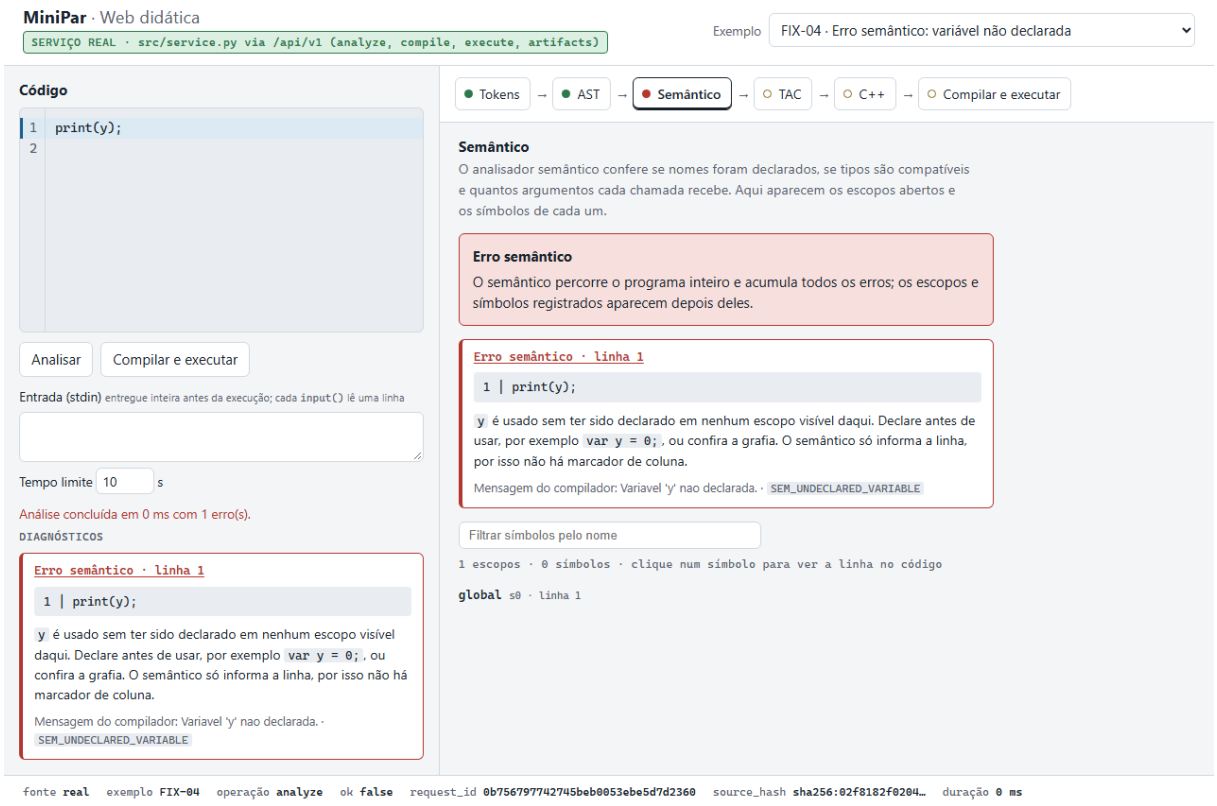
Fonte: Elaboração própria (2026).

Figura 29 — Web didática: erro sintático (`var x = 2 + ;`)



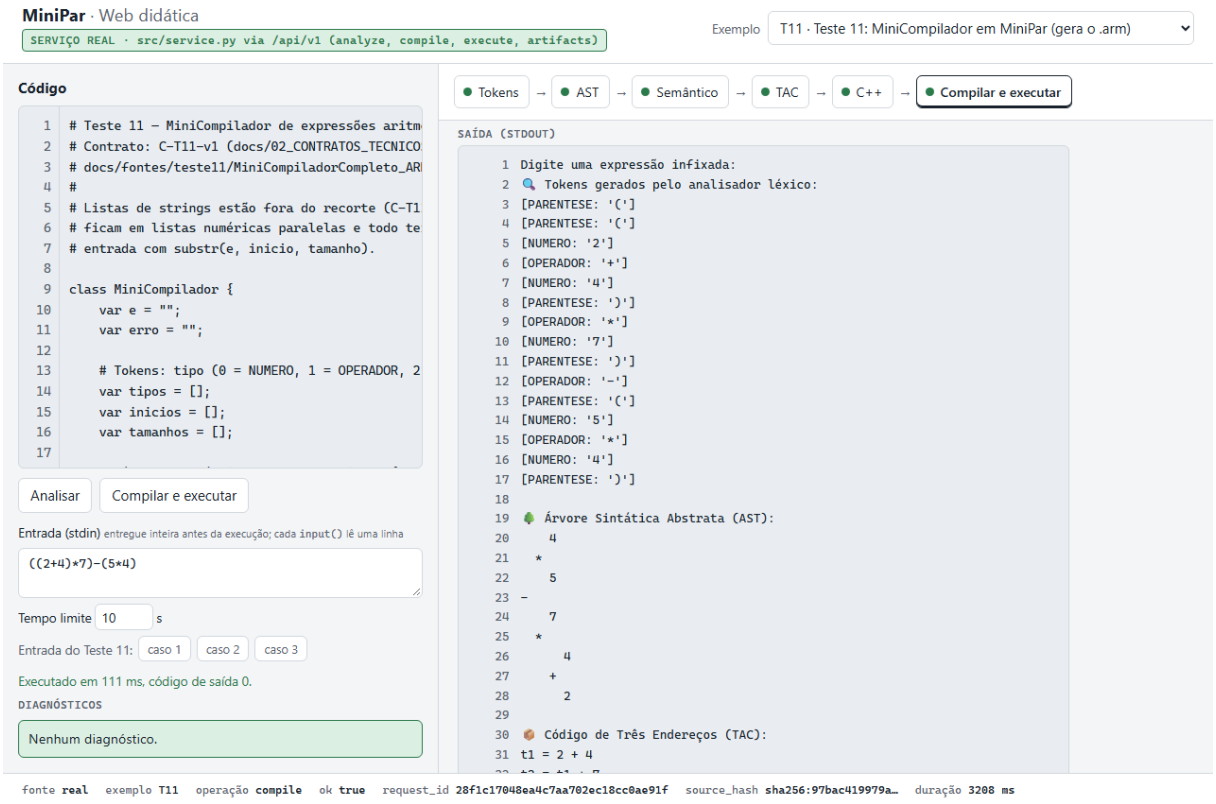
Fonte: Elaboração própria (2026).

Figura 30 — Web didática: erro semântico (`print(y);`)



Fonte: Elaboração própria (2026).

Figura 31 — Web didática: Teste 11, caso 1, `((2+4)*7)-(5*4)`



Fonte: Elaboração própria (2026).

7 TESTES E VALIDAÇÃO

Este capítulo relata a verificação do compilador: os testes de integração dos componentes, os onze programas de teste, a execução entre dois computadores e as medições de desempenho e de falhas. As saídas completas estão no Apêndice C.

7.1 Estratégia de verificação

Cada programa de teste tem um oráculo independente do compilador: um verificador em Python recalcula o resultado esperado (por exemplo, as 160 notas do Teste 8 ou o fractal do Teste 9) ou o compara com a saída de referência do enunciado. Um teste só é aprovado quando foi executado e o oráculo conferiu a saída; código de saída zero, sozinho, não basta. Testes com concorrência foram repetidos e só são aprovados se todas as repetições passarem. Nenhuma saída esperada está embutida no compilador: o TAC do Teste 7, por exemplo, é verificado também num programa alterado, para provar que é gerado a partir do código. O ambiente foi Windows 11 com 14 CPUs lógicas, Python 3.12.10, Flask 3.0.0 e GCC 16.2.0.

7.2 Testes de integração dos componentes

Os testes de integração exercitam os componentes juntos, pelas mesmas interfaces que o usuário ou o código gerado usam (Tabela 4). A suíte do runtime cobre o que mais facilmente falharia entre computadores: mensagens fragmentadas byte a byte, mensagens maiores que uma leitura de *socket*, quadros inválidos, trabalhador indisponível, tarefas que terminam fora de ordem na mesma conexão e 160 tarefas distribuídas em dois trabalhadores sem perda nem duplicação. Em cada caso, o resultado esperado é um valor correto ou um erro explícito, nunca um travamento.

Tabela 4 — Suítes de testes de integração

Suíte	O que integra e verifica	Casos	Resultado
Runtime	API local, quadros do protocolo, trabalhadores em processos separados, falhas de rede e API usada pelo C++ gerado	6 etapas	Aprovado
Compilador e runtime	MiniPar → CLI → C++ gerado → runtime: destino local, trabalhador local e queda do trabalhador	3	Aprovado
TAC do Teste 7	TAC derivado da AST, com programa alterado	5	Aprovado
Gerador TAC	`fork`, `join`, `par` e `for-in`	54	Aprovado
Gerador C++	C++ com o runtime e impressão atômica	19	Aprovado
Serviço de compilação	Fases, diagnósticos, isolamento e artefatos	36	Aprovado
Canal TCP	Canal do Teste 1, inclusive entrada após 7 s	2	Aprovado
Interface de terminal	Processo `main.py`, menu, compilação, execução, erro sintático e Fork/Join	14	Aprovado
Web original	Servidor na porta 5000, compilação e execução	1	Aprovado
Web didática	Rotas com o serviço e o GCC reais, entrada, tempo limite, downloads e Teste 11	37	Aprovado
Integração ponta a ponta	CLI com Fork/Join e runtime, serviço de fases e Web didática	25	Aprovado
Conformidade da gramática	Analisadores léxico e sintático contra cada produção do Quadro 5	114	Aprovado
Conformidade da UML	Classes, membros, heranças e cobertura dos diagramas	862	Aprovado

Fonte: Elaboração própria (2026).

7.3 Programas de teste

A Tabela 5 resume os onze programas de teste, cujo código-fonte está no Apêndice D. Dez foram aprovados; o Teste 4 ficou parcial.

Tabela 5 — Resultado dos programas de teste

Teste	Oráculo conferido	Resultado
T01 Calculadora cliente-servidor	Servidor recebe `+ 2 3` pelo canal; cliente recebe 5; 30 de 30 execuções	Aprovado
T02 Fatorial e Fibonacci em `par`	Fatorial de 5 igual a 120 e sequência 0, 1, 1, 2, 3, 5, 8, 13, sem linhas misturadas; 30 de 30	Aprovado
T03 Neurônio	Iterações 1, 50 e 51 iguais às da referência (quatro casas); convergência na 51	Aprovado
T04 Rede neural XOR	Classificação correta nas quatro entradas; valores diferem da referência em até 0,0151	Parcial
T05 Recomendação	Os 12 produtos da referência, com probabilidade entre 0,55 e 1	Aprovado
T06 Quicksort com classes	`10 -3 -40 80 70 -100` ordenado como `[-100, -40, -3, 10, 70, 80]`	Aprovado
T07 Quicksort iterativo e TAC	`[33, 12, 98, 5, 61]` ordenado; TAC muda com o programa; C++ sem violação de limites	Aprovado
T08 Quatro avaliadores	160 avaliações, sem perda nem duplicação, notas e soma iguais às do oráculo, nos modos sequencial, paralelo local e remoto	Aprovado
T09 Cinco trabalhos	Teste 8 mais o fractal de Mandelbrot, recalculado em dois tamanhos	Aprovado
T10 Seis trabalhos	Teste 9 mais o tapete de Sierpinski, com 8^g células cheias, num `fork` próprio	Aprovado
T11 MiniCompilador	Casos 1 e 3 iguais a 22 e caso 2 igual a −84; saídas e `.arm` iguais aos de referência	Aprovado

Fonte: Elaboração própria (2026).

Os Testes 1 a 7 vêm das versões anteriores. No Teste 1, servidor e cliente são ramos de um bloco `par` ligados por canais (Código 30). No Teste 2, os dois cálculos rodam em threads, e a ordem das linhas varia entre execuções, mas cada linha é impressa inteira (Código 31). O Teste 3 implementa um neurônio que converge na iteração 51, como a referência (Código 32); o Teste 5 recomenda os mesmos 12 produtos da referência (Código 34); o Teste 6 ordena o vetor com classes (Código 35); e o Teste 7 ordena sem recursão e tem seu TAC comparável ao do enunciado (Código 36).

O Teste 4 treina uma rede com dois neurônios de entrada, três ocultos e um de saída, por 20 mil épocas. A classificação da função XOR está correta, mas os valores diferem da referência (Tabela 6). Uma reprodução independente em Python, com os mesmos pesos iniciais e as mesmas atualizações, obteve os valores do binário até a sexta casa, o que descarta erro do compilador: o programa usa pesos iniciais fixos e taxa de aprendizado 0,5, enquanto a referência sorteia os pesos sem informar a semente e usa taxa 0,2. Por isso o teste é registrado como parcial.

Tabela 6 — Teste 4: saídas do compilador e da referência

Entrada	MiniPar	Referência	Diferença absoluta
`[0, 0]`	0,006844	0,0089	0,0021
`[0, 1]`	0,986493	0,9769	0,0096
`[1, 0]`	0,989250	0,9758	0,0135
`[1, 1]`	0,014030	0,0291	0,0151

Fonte: Elaboração própria (2026); referência de Hernández Domínguez (2026).

Os Testes 8 a 10 exercitam o Fork/Join. O Teste 8 avalia 160 atividades com quatro avaliadores: cada avaliador recebe uma fatia de 40 atividades e calcula cada nota com um gerador congruencial repetido um número de vezes proporcional à carga, o que dá à tarefa um custo de CPU controlado. No modo paralelo, quatro `fork` disparam as fatias antes de qualquer `join`, com destinos lidos da entrada (Código 1). O oráculo recalcula as 160 notas e confere que cada atividade foi avaliada uma única vez (Código 37). O Teste 9 acrescenta o fractal de Mandelbrot em ASCII, idêntico, caractere a caractere, ao da referência (Código 38), e o Teste 10 acrescenta o tapete de Sierpinski num `fork` próprio, impresso em ASCII e convertido em SVG pelo verificador (Código 39).

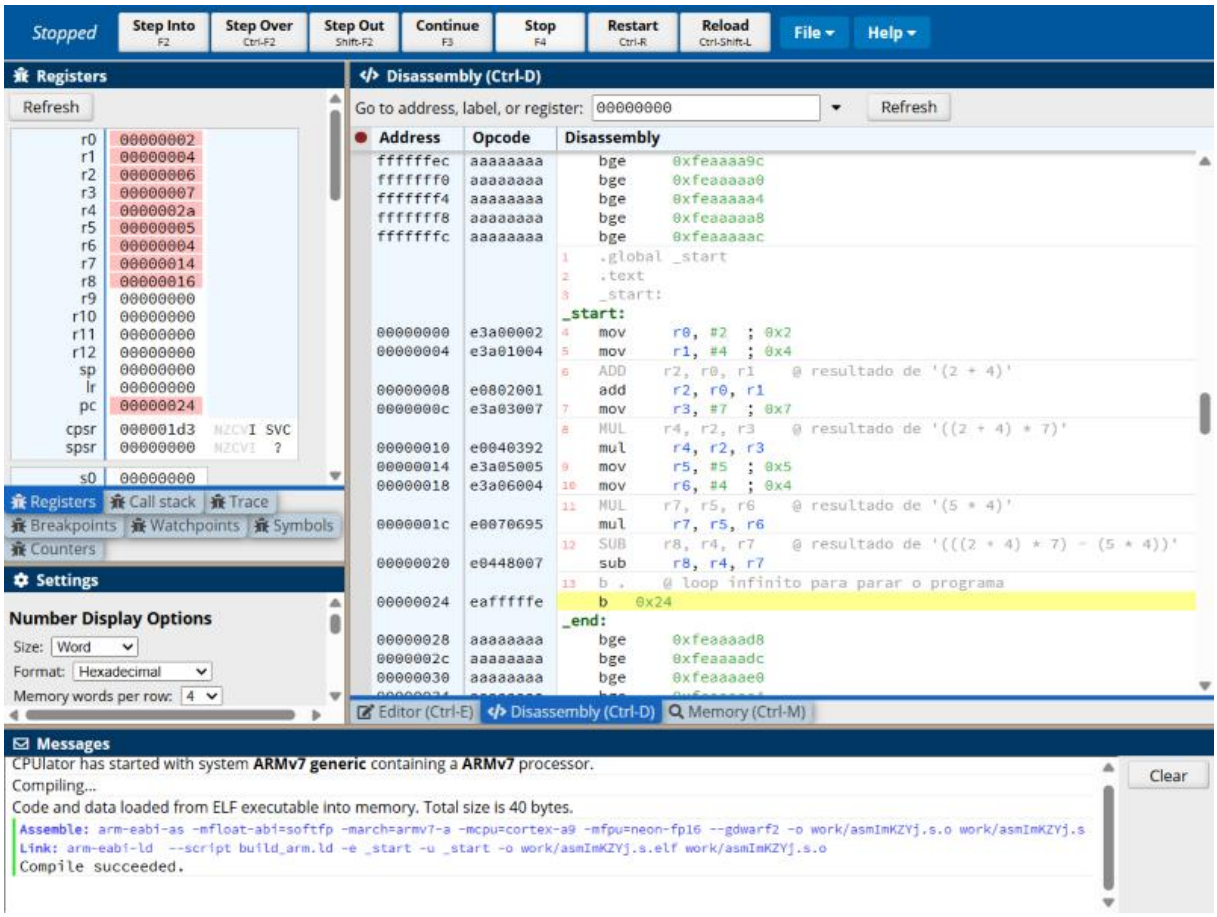
O MiniCompilador do Teste 11 foi executado nos três casos obrigatórios (Tabela 7); a saída do caso 1 (Código 40) e o `.arm` gerado são idênticos aos de referência. O `.arm` de cada caso foi aberto no CPUlator, montado, executado até a instrução `b .` e conferido pelos registradores (Figuras 32 e 33). O verificador conferiu ainda nove entradas extras, entre elas `2*3+4*5` = 26 e `2-3*4` = −10, que exercitam precedência e associatividade, e 21 entradas inválidas, rejeitadas com mensagem (Quadro 20).

Tabela 7 — Teste 11: casos obrigatórios

Caso	Entrada	Resultado	CPUlator: registrador final	Situação
1	`((2+4)*7)-(5*4)`	22,0	`r8 = 0x00000016 = 22`	Aprovado
2	`((10 -8*7)+4)*2`	−84,0	`r8 = 0xFFFFFFAC = −84`	Aprovado
3	`((2+4)*7)-(5*4)`	22,0	`r8 = 0x00000016 = 22`	Aprovado

Fonte: Elaboração própria (2026).

Figura 32 — CPUlator: caso 1 do Teste 11



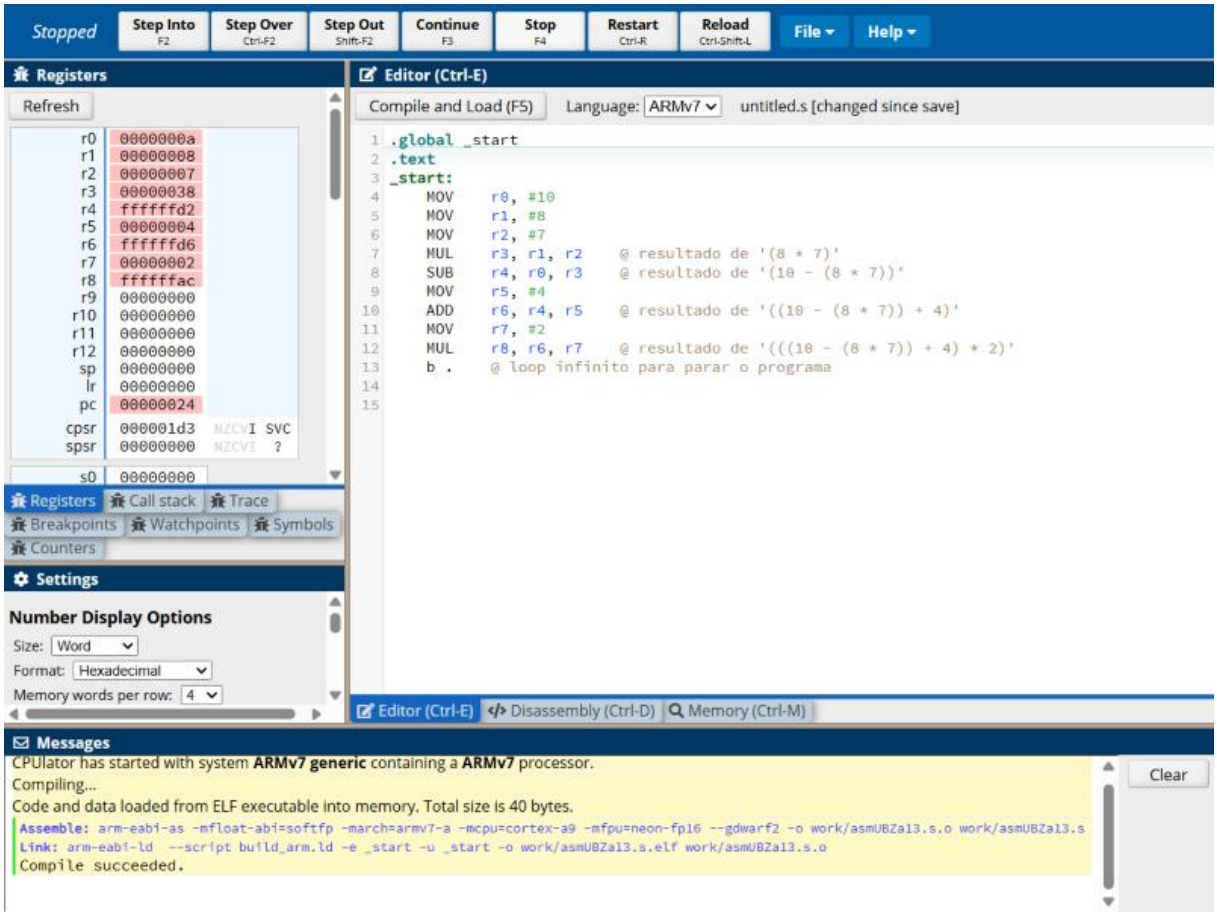
Fonte: Elaboração própria (2026).

Quadro 20 — Teste 11: entradas rejeitadas e mensagens

Entrada	Saída do MiniCompilador
`-2+3`	Erro sintático: menos unário não suportado na coluna 1
`2.5*4`	Erro léxico: número decimal não suportado na coluna 1
`x+1`	Erro léxico: identificador não suportado na coluna 1
`8/2`	Erro léxico: divisão `'/'` não suportada na coluna 2
`((2+3)`	Erro sintático: `'('` da coluna 1 sem `')'` correspondente
`2147483648`	Erro léxico: constante fora do intervalo 0..2147483647 na coluna 1
`1+2+3+4+5+6+7+8`	Erro semântico: a AST tem 15 nós e o Assembly comporta no máximo 13
`65536*65536`	Erro semântico: o resultado não cabe em 32 bits com sinal

Fonte: Elaboração própria (2026).

Figura 33 — CPUlator: caso 2 do Teste 11



Fonte: Elaboração própria (2026).

7.4 Execução em dois computadores

O enunciado pede que o paralelismo seja testado com computadores diferentes. Execuções no próprio computador provam o protocolo, mas não a rede; por isso, o Teste 8 rodou com o coordenador na máquina A e quatro trabalhadores na máquina B, ligadas por uma rede local. O mesmo código-fonte foi compilado nas duas máquinas, e os trabalhadores foram iniciados com `--minipar-worker <porta>` nas portas 7201 a 7204. Cada trabalhador registrou a conexão vinda de A e uma tarefa concluída (Código 41), e o coordenador produziu o mesmo resultado da execução sequencial, com soma das notas 7630 (Código 42). Num segundo cenário, um trabalhador estava encerrado quando o coordenador iniciou: o `join` terminou com `WORKER_UNAVAILABLE` em 3 segundos, sem travar (Código 43). Num programa de demonstração do runtime, o trabalhador em outro computador foi encerrado durante uma tarefa longa, e o `join` do coordenador terminou com erro, sem travar (Código 44).

7.5 Desempenho e falhas

O *benchmark* mede o Teste 8 com a mesma carga em quatro cenários, que só mudam quem executa as quatro fatias: `seq` (chamadas diretas, em sequência), `par_local` (quatro `fork` locais, quatro threads), `par_loopback` (quatro trabalhadores em processos separados na mesma máquina) e `par_rede` (trabalhadores em outro computador). Cada execução só entra na estatística se a saída bater com o oráculo; há uma rodada de aquecimento descartada, e os cenários são intercalados. O tempo é o de parede do processo coordenador, e a aceleração e a redução seguem as Equações 1 e 2, sobre as medianas. A máquina A tem processador Intel Core Ultra 5 135U, com 12 núcleos físicos e 14 threads lógicas.

Tabela 8 — Tempos medianos na máquina A (carga de 5 milhões, 7 medições por cenário)

Rodada	`seq` (s)	`par_local` (s)	Acel. local	`par_loopback` (s)	Acel. *loopback*	Desvio (s)
1	2,581	0,747	3,46	0,738	3,50	0,056
2	2,507	0,720	3,48	1,144	2,19	0,211
3	2,582	0,766	3,37	1,121	2,30	0,262

Fonte: Elaboração própria (2026).

Nota: O desvio refere-se ao cenário `par_loopback`.

Tabela 9 — Rodada com trabalhadores em outro computador

Cenário	Mediana (s)	Mín. (s)	Máx. (s)	Desvio (s)	Aceleração	Redução
`seq` (A)	2,493	2,480	2,506	0,010	—	—
`par_local` (A)	0,738	0,726	0,785	0,020	3,38	70,4 %
`par_loopback` (A)	0,747	0,742	0,759	0,006	3,34	70,0 %
`par_rede` (A → B)	1,068	0,978	1,380	0,135	2,33	57,2 %

Fonte: Elaboração própria (2026).

As Tabelas 8 e 9 mostram os resultados. Nos cenários locais, os quatro trabalhadores dividem os núcleos da mesma máquina; o teto teórico é 4, e a diferença para os valores medidos vem da partida do processo, incluída no tempo, e das fatias de custo desigual, coerentemente com a lei de Amdahl (Equação 3). O cenário `par_loopback` variou entre rodadas (de 2,19 a 3,50) porque os processos disputam os núcleos e pagam a serialização e o TCP local. O cenário `par_rede`, o único que usa a CPU de outra máquina, foi mais lento e mais variável, porque inclui a conexão pela rede e depende da CPU de B. As falhas do runtime foram repetidas de forma controlada, com processos novos a cada repetição (Tabela 10).

Tabela 10 — Falhas repetíveis do runtime

ID	Montagem	Resultado esperado e observado	Execuções
F1	Quatro destinos em portas sem trabalhador	`WORKER_UNAVAILABLE`, código 1, em cerca de 2 s	15/15
F2	Quatro trabalhadores; o primeiro é encerrado após receber a tarefa	`WORKER_LOST`, código 1, de 7 a 15 ms após o encerramento	15/15
F3	Servidor que aceita as tarefas e nunca responde, com limite de 3 s	`TIMEOUT`, código 1, entre 3,05 e 3,08 s	15/15

Fonte: Elaboração própria (2026).

7.6 Limitações

As limitações conhecidas são:

a) o Teste 4 não reproduz os valores numéricos da referência, que dependem de pesos sorteados sem semente informada;

b) entre dois computadores foram executados o Teste 8 e a demonstração do runtime; os Testes 1, 9 e 10 foram executados com trabalhadores na mesma máquina;

c) o tapete de Sierpinski do Teste 10 é impresso em ASCII e convertido em SVG, e não desenhado numa janela gráfica como na referência;

d) a validação completa foi feita em Windows 11; em Linux, só a suíte do runtime;

e) o trabalhador remoto não tem autenticação nem criptografia;

f) o analisador sintático relata apenas o primeiro erro, sem recuperação.

8 CONCLUSÃO

O compilador da linguagem MiniPar 2026.2 atinge o objetivo geral: programas orientados a objetos, com blocos sequenciais, blocos paralelos e tarefas `fork`/`join`, são traduzidos para executáveis nativos que distribuem trabalho entre núcleos de um computador ou entre computadores diferentes, apenas trocando o destino de cada tarefa.

A linguagem está especificada por uma tabela de símbolos, que descreve cada palavra reservada, operador, delimitador e função nativa com a forma de uso, e por uma gramática BNF detalhada produção por produção, ambas derivadas do código; a gramática foi conferida por sondas que executam os analisadores reais. Os analisadores léxico, sintático e semântico, a tabela de símbolos e o tratamento de erros formam componentes independentes. O código de três endereços é gerado da AST para qualquer programa, inclusive `fork`, `join` e `par`. O gerador C++ produz código C++20 compilado pelo GCC com otimização e, para programas com tarefas, só gera código quando prova os tipos que cruzam as tarefas. O runtime executa tarefas em threads ou em trabalhadores remotos, com um protocolo enquadrado e erros explícitos, e nenhum dos cenários de falha testados travou o programa. As interfaces herdadas foram preservadas, e a Web didática mostra cada fase de uma compilação real, com uma AST navegável e erros explicados.

Na validação, dez dos onze programas de teste foram aprovados com oráculos independentes, e o Teste 4 ficou parcial por uma diferença entre o programa e a referência, e não por erro do compilador. O Teste 8 rodou entre dois computadores, com o mesmo resultado da execução sequencial e aceleração de 2,33 vezes. O MiniCompilador do Teste 11, escrito na própria MiniPar, gerou Assembly ARMv7 que rodou no CPUlator com os resultados esperados, o que mostra que a linguagem é expressiva o bastante para escrever outro compilador.

Como trabalhos futuros, ficam: acrescentar recuperação de erros ao analisador sintático; executar os Testes 1, 9 e 10 entre dois computadores; definir a tolerância numérica do Teste 4; validar o compilador em Linux; acrescentar autenticação e criptografia ao protocolo dos trabalhadores; e aplicar otimizações sobre o código de três endereços, hoje delegadas ao GCC.

REFERÊNCIAS

AHO, Alfred V.; LAM, Monica S.; SETHI, Ravi; ULLMAN, Jeffrey D. **Compiladores**: princípios, técnicas e ferramentas. 2. ed. São Paulo: Pearson Addison-Wesley, 2008.

AMDAHL, Gene M. Validity of the single processor approach to achieving large scale computing capabilities. *In*: AFIPS SPRING JOINT COMPUTER CONFERENCE, 1967, Atlantic City. **Proceedings** [...]. New York: Association for Computing Machinery, 1967. p. 483-485.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 6023**: informação e documentação: referências: elaboração. Rio de Janeiro: ABNT, 2018.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 10520**: informação e documentação: citações em documentos: apresentação. Rio de Janeiro: ABNT, 2023.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 14724**: informação e documentação: trabalhos acadêmicos: apresentação. Rio de Janeiro: ABNT, 2024.

BOOCH, Grady; RUMBAUGH, James; JACOBSON, Ivar. **UML**: guia do usuário. 2. ed. Rio de Janeiro: Elsevier, 2006.

BRASILEIRO, Victor André Lopes; SÁ JUNIOR, Erasmo da Silva; SILVA, José Riquelme Teixeira da. **minipar-2026-2**: compilador da linguagem MiniPar 2026.2. [*S. l.*]: GitHub, 2026. Repositório de código-fonte. Disponível em: https://github.com/VictorBrasileiroo/minipar-2026-2. Acesso em: 9 nov. 2026.

CONWAY, Melvin E. A multiprocessor system design. *In*: AFIPS FALL JOINT COMPUTER CONFERENCE, 1963, Las Vegas. **Proceedings** [...]. New York: Association for Computing Machinery, 1963. p. 139-146.

COOPER, Keith D.; TORCZON, Linda. **Construindo compiladores**. 2. ed. Rio de Janeiro: Elsevier, 2014.

COULOURIS, George; DOLLIMORE, Jean; KINDBERG, Tim; BLAIR, Gordon. **Sistemas distribuídos**: conceitos e projeto. 5. ed. Porto Alegre: Bookman, 2013.

FREE SOFTWARE FOUNDATION. **Using the GNU Compiler Collection (GCC)**. Boston: Free Software Foundation, 2026. Disponível em: https://gcc.gnu.org/onlinedocs/. Acesso em: 9 nov. 2026.

HERNÁNDEZ DOMÍNGUEZ, Arturo. **Desenvolvimento de um compilador para a linguagem MiniPar 2026.2 orientada a objetos utilizando real paralelismo com execução em computadores diferentes**. Maceió: Universidade Federal de Alagoas, Instituto de Computação, 2026. Enunciado do projeto da disciplina Compiladores.

INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. **ISO/IEC 14882:2020**: programming languages: C++. 6. ed. Geneva: ISO, 2020.

LEA, Doug. A Java fork/join framework. *In*: ACM CONFERENCE ON JAVA GRANDE, 2000, San Francisco. **Proceedings** [...]. New York: Association for Computing Machinery, 2000. p. 36-43.

MERMAID. **Mermaid**: diagramming and charting tool. Versão 11. [*S. l.*], 2026. Disponível em: https://mermaid.js.org/. Acesso em: 9 nov. 2026.

MINIPAR improvement: compilador da linguagem MiniPar 2026.1. [*S. l.*]: GitHub, 2026. Repositório de código-fonte. Disponível em: https://github.com/luizwhirl/minipar-improvement. Acesso em: 15 out. 2026.

OBJECT MANAGEMENT GROUP. **OMG Unified Modeling Language (OMG UML)**: version 2.5.1. Needham: OMG, 2017. Disponível em: https://www.omg.org/spec/UML/2.5.1/PDF. Acesso em: 9 nov. 2026.

RÊGO, Hélio José Ribeiro; SILVA, Antônio Guilherme da; CALIXTO, Gabriel Nathan Telaroli Dias. **Relatório técnico**: desenvolvimento do compilador para a linguagem MiniPar v2025.1. Maceió: Universidade Federal de Alagoas, 2025. Relatório da disciplina Compiladores.

SOMMERVILLE, Ian. **Engenharia de software**. 9. ed. São Paulo: Pearson Prentice Hall, 2011.

SZYPERSKI, Clemens. **Component software**: beyond object-oriented programming. 2. ed. London: Addison-Wesley, 2002.

TANENBAUM, Andrew S.; VAN STEEN, Maarten. **Sistemas distribuídos**: princípios e paradigmas. 2. ed. São Paulo: Pearson Prentice Hall, 2007.

VALE, Helena Cristina Pimentel do; LENZI, Lívia Aparecida Ferreira (org.). **Manual para normalização de trabalhos acadêmicos da UFAL**. 2. ed. rev. e atual. Maceió: UFAL, 2025. Disponível em: http://www.repositorio.ufal.br/jspui/handle/123456789/10627. Acesso em: 9 nov. 2026.

WONG, Henry. **CPUlator computer system simulator**. [*S. l.*], 2026. Disponível em: https://cpulator.01xz.net/. Acesso em: 9 nov. 2026.

Apêndices

APÊNDICE A — GRAMÁTICA BNF CONSOLIDADA

Este apêndice reúne, em forma contínua, a gramática detalhada no Quadro 5 e as regras léxicas do Quadro 3. Cada regra sintática cita, em comentário, o método do analisador que a implementa.

Código 5 — Regras léxicas: espaços e comentários

```
<ignorado>   ::= <espaco> | <comentario>
<espaco>     ::= qualquer caractere c com c.isspace()          (* inclui \n, \t, \r *)
<comentario> ::= "#" { qualquer caractere exceto "\n" }        (* até o fim da linha *)
```

Fonte: Elaboração própria (2026).

Código 6 — Regras léxicas: identificadores

```
<IDENTIFIER> ::= ( <letra> | "_" ) { <letra> | <digito_u> | "_" }
<letra>      ::= qualquer caractere c com c.isalpha()          (* Unicode: "ação" é aceito *)
<digito_u>   ::= qualquer caractere c com c.isalnum() e não letra
```

Fonte: Elaboração própria (2026).

Código 7 — Regras léxicas: literais

```
<NUMBER>         ::= <digito> { <digito> } [ "." { <digito> } ]  (* "1." é aceito; ".5" não *)
<digito>         ::= qualquer caractere c com c.isdigit()

<STRING_LITERAL> ::= '"' { <caractere_string> } '"'
<caractere_string> ::= qualquer caractere exceto '"' e "\"     (* pode conter quebra de linha *)
                     | "\" <caractere>
<BOOL_LITERAL>   ::= "true" | "false"
```

Fonte: Elaboração própria (2026).

Código 8 — Gramática sintática consolidada da MiniPar 2026.2

```
(* ====================================================================== *)
(*  MiniPar 2026.2 — derivada de src/parser.py                            *)
(* ====================================================================== *)

(* ─── PROGRAMA ──────────────────────────────────── Parser.parse ─── *)
<programa> ::= { <comando> } <END_OF_FILE>

(* ─── COMANDOS ──────────────────────────────── _parse_statement ─── *)
<comando> ::= <declaracao_classe>
            | <declaracao_funcao>
            | <comando_return>
            | <comando_break>
            | <comando_continue>
            | <declaracao_variavel>
            | <bloco_seq>
            | <bloco_par>
```

```
            | <comando_print>
            | <comando_send>
            | <comando_if>
            | <comando_while>
            | <comando_do_while>
            | <comando_for>
            | <comando_expressao>

(* class e func são comandos: podem aparecer dentro de blocos. *)

(* ─── DECLARAÇÕES ──────────────────────────────────────────────── *)
<declaracao_variavel> ::= ( "var" | <TYPE> ) <IDENTIFIER> [ "=" <expressao> ] ";"
                                                  (* _parse_var_decl *)

<declaracao_funcao>   ::= "func" <IDENTIFIER> "(" [ <lista_parametros> ] ")" <bloco_chaves>
<lista_parametros>    ::= <parametro> { "," <parametro> }
<parametro>           ::= [ <TYPE> ] <IDENTIFIER>    (* o tipo é lido e descartado *)
                                                   (* _parse_func_decl *)

<declaracao_classe>   ::= "class" <IDENTIFIER> [ "extends" <IDENTIFIER> ]
                          "{" { <membro_classe> } "}"
<membro_classe>       ::= "var" <IDENTIFIER> [ "=" <expressao> ] ";"
                        | "func" <IDENTIFIER> "(" [ <lista_parametros> ] ")" <bloco_chaves>
                                                  (* _parse_class_decl *)

(* ─── COMANDOS SIMPLES ─────────────────────────────────────────── *)
<comando_return>   ::= "return" [ <expressao> ] [ ";" ]     (* _parse_return_stmt *)
<comando_break>    ::= "break" [ ";" ]
<comando_continue> ::= "continue" [ ";" ]
<comando_print>    ::= "print" "(" [ <lista_argumentos> ] ")" ";"
<comando_send>     ::= "send" "(" <expressao> "," <expressao> ")" ";"

<comando_expressao> ::= <expressao_inicial> "=" <expressao> ";"   (* atribuição *)
                      | <expressao_inicial> ";"                    (* chamada/fork/join *)

(* <expressao_inicial> é uma <expressao> cujo primeiro token é um de:
   IDENTIFIER this new receive input c_channel matrix "[" fork join.
   Restrições verificadas pelo parser depois de analisar a expressão:
   - com "=": o lado esquerdo deve ser IDENTIFIER, acesso a propriedade
     (x.p) ou indexação (x[i]); senão SYN_INVALID_ASSIGNMENT.
     Produz Assignment, PropertyAssign ou IndexAssign.
   - sem "=": a expressão inteira deve ser chamada de função, chamada de
     método, fork ou join; senão SYN_INVALID_STATEMENT.              *)

(* ─── BLOCOS ──────────────────────────── _parse_seq/_parse_par_block ─── *)
<bloco_seq>    ::= [ "seq" ] <bloco_chaves>
<bloco_chaves> ::= "{" { <comando> } "}"
<bloco_par>    ::= "par" "{" { <comando> } "}"   (* cada comando é um ramo paralelo *)

(* ─── CONTROLE ─────────────────────────────────────────────────── *)
<comando_if>       ::= "if" "(" <expressao> ")" <comando> [ "else" <comando> ]
                       (* "else" casa com o if mais próximo; else-if = else + <comando_if> *)
<comando_while>    ::= "while" "(" <expressao> ")" <comando>
<comando_do_while> ::= "do" <comando> "while" "(" <expressao> ")" [ ";" ]

<comando_for>      ::= "for" "(" ( <for_iterador> | <for_classico> ) ")" <comando>
<for_iterador>     ::= "var" <IDENTIFIER> "in" <expressao>
<for_classico>     ::= [ <inicializador_for> ] ";" [ <expressao> ] ";" [ <passo_for> ]
<inicializador_for>::= ( "var" | <TYPE> ) <IDENTIFIER> [ "=" <expressao> ]
                     | <expressao> [ "=" <expressao> ]
<passo_for>        ::= <expressao> [ "=" <expressao> ]
                                                  (* _parse_for_stmt *)

(* ─── EXPRESSÕES (da menor para a maior precedência) ──────────── *)
<expressao>      ::= <expressao_ou>                              (* _parse_expression *)
<expressao_ou>   ::= <expressao_e> { "||" <expressao_e> }         (* _parse_or *)
<expressao_e>    ::= <expressao_comp> { "&&" <expressao_comp> }   (* _parse_and *)
<expressao_comp> ::= <expressao_termo>
                     { ( "==" | "!=" | "<" | ">" | "<=" | ">=" ) <expressao_termo> }
                                                                 (* _parse_comparison *)
```

```
<expressao_termo>::= <expressao_fator> { ( "+" | "-" ) <expressao_fator> }   (* _parse_term *)
<expressao_fator>::= <expressao_unaria> { ( "*" | "/" | "%" ) <expressao_unaria> }
                                                                  (* _parse_factor *)
<expressao_unaria>::= ( "-" | "+" | "!" ) <expressao_unaria>
                    | <expressao_posfixa>                         (* _parse_unary *)

<expressao_posfixa>  ::= <expressao_primaria> { <sufixo> }         (* _parse_primary, laço final
↪  *)
<sufixo>             ::= "." <IDENTIFIER> [ "(" [ <lista_argumentos> ] ")" ]
                       | "[" <expressao> "]"

<expressao_primaria> ::= <expressao_fork>
                       | <expressao_join>
                       | "matrix" "(" <expressao> "," <expressao> "," <expressao> ")"
                       | "[" [ <lista_argumentos> ] "]"
                       | "c_channel" "(" <expressao> "," <expressao> ")"
                       | "receive" "(" <expressao> ")"
                       | "input" "(" [ <expressao> ] ")"
                       | "new" <IDENTIFIER> "(" [ <lista_argumentos> ] ")"
                       | "this"
                       | <NUMBER> | <STRING_LITERAL> | <BOOL_LITERAL>
                       | <IDENTIFIER> [ "(" [ <lista_argumentos> ] ")" ]
                       | "(" <expressao> ")"

<lista_argumentos>   ::= <expressao> { "," <expressao> }

(* ─── FORK/JOIN (2026.2) ──────────────────────────────────────── *)
<expressao_fork> ::= "fork" <IDENTIFIER> "(" [ <lista_argumentos> ] ")" [ "on" <expressao> ]
<expressao_join> ::= "join" "(" <expressao> [ "," <expressao> ] ")"
```

Fonte: Elaboração própria (2026), derivada de `src/parser.py`.

APÊNDICE B — PSEUDOCÓDIGOS DOS COMPONENTES

Este apêndice traz os pseudocódigos do compilador e do MiniCompilador do Teste 11, descritos no capítulo 5.

B.1 Fluxo de compilação e analisador léxico

Código 9 — Pseudocódigo do fluxo de compilação [`service._pipeline`]

```
função PIPELINE(fonte, operação ∈ {analyze, compile}):
    resultado ← envelope minipar.compile.v1 com todas as fases "skipped"
    // 1. Léxico
    lexer ← LEXER(fonte); tokens ← lexer.TOKENIZAR()
    publicar fase lexer (tokens sem END_OF_FILE)
    se lexer.diagnósticos ≠ ∅: retornar resultado            // não chama o parser
    // 2. Sintaxe
    tentar programa ← PARSER(tokens, sair_no_erro=falso).PARSE()
    se ParseError e: anexar e.diagnóstico; retornar resultado // 1º erro sintático
    publicar AST serializada
    // 3. Semântica + tabela de símbolos
    sem ← ANALISADOR_SEMANTICO(); válido ← sem.ANALISAR(programa)
    publicar sem.escopos, sem.símbolos
    anexar diagnósticos semânticos ordenados por linha
    se não válido: retornar resultado
    // 4. TAC
    tentar tac ← GERADOR_TAC().GERAR_TEXTO(programa)
    se NotImplementedError: anexar TAC_UNSUPPORTED_FEATURE; retornar resultado
    // 5. C++
    tentar cpp ← GERADOR_CPP().GERAR_TEXTO(programa, sem, fonte)
    se NotImplementedError ou CppProofError: anexar CPP_UNSUPPORTED_FEATURE; retornar
    se operação = analyze: resultado.ok ← verdadeiro; retornar resultado
    // 6. g++ (somente compile)
    gravar program.cpp e program.tac; compilar com g++; registrar binário
    resultado.ok ← (g++ terminou com sucesso)
    // Qualquer outra exceção: INTERNAL_ERROR na fase corrente
```

Fonte: Elaboração própria (2026).

Código 10 — Pseudocódigo do analisador léxico [`Lexer.tokenize`]

```
estrutura TOKEN: tipo, lexema, linha, coluna

função TOKENIZAR(fonte):
    pos ← 0; linha ← 1; coluna ← 1; tokens ← []; diagnósticos ← []

    enquanto pos < |fonte|:
        PULAR_ESPACOS_E_COMENTARIOS()
        se pos ≥ |fonte|: sair do laço
        c ← ESPIAR(); lin0 ← linha; col0 ← coluna

        se c é letra (Unicode) ou "_":
            lexema ← consumir enquanto ESPIAR() é alfanumérico ou "_"
            tipo ← KEYWORDS[lexema] se existir, senão IDENTIFIER
            emitir TOKEN(tipo, lexema, lin0, col0)

        senão se c é dígito:
            lexema ← consumir dígitos
            se ESPIAR() = ".": lexema ← lexema + AVANÇAR() + consumir dígitos
            emitir TOKEN(NUMBER, lexema, lin0, col0)
```

```
        senão se c = '"':
            AVANÇAR(); texto ← ""
            enquanto ESPIAR() ∉ {'"', FIM}:
                se ESPIAR() = "\":
                    AVANÇAR(); se FIM: sair
                    e ← AVANÇAR()
                    texto ← texto + {n→"\n", t→"\t", "→'"', \→"\"}.obter(e, e)
                senão texto ← texto + AVANÇAR()
            se ESPIAR() = '"': AVANÇAR()
            senão relatar(LEX_UNTERMINATED_STRING, lin0, col0)
            emitir TOKEN(STRING_LITERAL, texto, lin0, col0)   // emitido mesmo com erro

        senão:   // operadores e delimitadores
            op ← AVANÇAR()
            se op ∈ {"=", "!", "<", ">"} e ESPIAR() = "=": emitir EQ/NEQ/LTE/GTE; AVANÇAR()
            senão se op = "&" e ESPIAR() = "&": emitir AND; AVANÇAR()
            senão se op = "|" e ESPIAR() = "|": emitir OR; AVANÇAR()
            senão se op ∈ {"=", "!", "<", ">"}: emitir ASSIGN/NOT/LT/GT
            senão se op ∈ TABELA_SIMPLES (+ - * / % { } ( ) [ ] ; , .): emitir tipo da tabela
            senão emitir TOKEN(UNKNOWN, op, lin0, col0)        // inclui "&" e "|" isolados

    para cada token t com t.tipo = UNKNOWN: relatar(LEX_UNKNOWN_CHAR, t.linha, t.coluna)
    ordenar diagnósticos por (linha, coluna)
    emitir TOKEN(END_OF_FILE, "", linha, coluna)
    retornar tokens

procedimento PULAR_ESPACOS_E_COMENTARIOS():
    enquanto não FIM:
        se ESPIAR() é espaço: AVANÇAR()
        senão se ESPIAR() = "#": AVANÇAR até "\n" ou FIM   // "\n" fica para o próximo giro
        senão retornar

função AVANÇAR():
    c ← fonte[pos]; pos ← pos + 1
    se c = "\n": linha ← linha + 1; coluna ← 1  senão coluna ← coluna + 1
    retornar c
```

Fonte: Elaboração própria (2026).

B.2 Analisador sintático

Código 11 — Primitivas e erro do analisador sintático

```
ESPIAR()        → token atual (END_OF_FILE se passou do fim)
CASAR(tipo)     → se ESPIAR().tipo = tipo: AVANÇAR(); retornar verdadeiro; senão falso
CONSUMIR(tipo, msg):
    se ESPIAR().tipo = tipo: retornar AVANÇAR()
    FALHAR(SYN_EXPECTED_TOKEN, msg + " (Encontrado: " + ESPIAR().lexema + ")")

FALHAR(código, mensagem, linha, coluna):                          [_fail]
    d ← {phase: "parser", severity: "error", code, message, line, column}
    diagnósticos.anexar(d)
    se sair_no_erro: imprimir "[ERRO SINTATICO] Linha L: msg" em stderr; encerrar(1)
    lançar ParseError(d)
```

Fonte: Elaboração própria (2026).

Código 12 — Programa e comandos [`parse`, `_parse_statement`]

```
função PARSE():                                                    [parse]
    programa ← Program([])
    enquanto ESPIAR().tipo ≠ END_OF_FILE: programa.comandos.anexar(COMANDO())
    retornar programa

função COMANDO():                                                  [_parse_statement]
    se CASAR(CLASS):    retornar DECL_CLASSE()
    se CASAR(FUNC):     retornar DECL_FUNCAO()
    se CASAR(RETURN):   valor ← nulo se ESPIAR()=";" senão EXPRESSAO(); CASAR(";")
                        retornar ReturnStmt(valor)
    se CASAR(BREAK):    CASAR(";"); retornar BreakStmt
    se CASAR(CONTINUE): CASAR(";"); retornar ContinueStmt
    se CASAR(VAR):      retornar DECL_VARIAVEL("var")
    se ESPIAR() é TYPE: t ← AVANÇAR(); retornar DECL_VARIAVEL(t.lexema)
    se CASAR(SEQ) ou ESPIAR()="{": retornar BLOCO_SEQ()
    se CASAR(PAR):      retornar BLOCO_PAR()
    se CASAR(PRINT):    retornar PRINT()        // "(" [args] ")" ";"
    se CASAR(SEND):     retornar SEND()         // "(" canal "," msg ")" ";"
    se CASAR(IF):       retornar IF()
    se CASAR(WHILE):    retornar WHILE()
    se CASAR(DO):       retornar DO_WHILE()
    se CASAR(FOR):      retornar FOR()
    se ESPIAR().tipo ∈ {IDENTIFIER, THIS, NEW, RECEIVE, INPUT, C_CHANNEL,
                        MATRIX, "[", FORK, JOIN}:
        e ← EXPRESSAO()
        se CASAR("="): retornar CAUDA_ATRIBUICAO(e, exige_ponto_virgula=verdadeiro)
        se e é MethodCall, FuncCallExpr, ForkExpr ou JoinExpr:
            CONSUMIR(";"); retornar e
        FALHAR(SYN_INVALID_STATEMENT, "Instrucao ou expressao isolada invalida")
    FALHAR(SYN_UNRECOGNIZED_STATEMENT, "Instrucao nao reconhecida")

função CAUDA_ATRIBUICAO(alvo, exige_ponto_virgula):              [_parse_assignment_tail]
    valor ← EXPRESSAO(); se exige_ponto_virgula: CONSUMIR(";")
    caso alvo seja
        IdentifierExpr → retornar Assignment(alvo.nome, valor)
        PropertyAccess → retornar PropertyAssign(alvo.objeto, alvo.propriedade, valor)
        IndexExpr      → retornar IndexAssign(alvo, valor)
    FALHAR(SYN_INVALID_ASSIGNMENT, "Atribuicao invalida")
```

Fonte: Elaboração própria (2026).

Código 13 — Declarações, blocos e controle

```
função DECL_VARIAVEL(tipo, exige_ponto_virgula=verdadeiro):    [_parse_var_decl]
    nome ← CONSUMIR(IDENTIFIER).lexema
    inicial ← EXPRESSAO() se CASAR("=") senão nulo
    se exige_ponto_virgula: CONSUMIR(";")
    retornar VarDecl(nome, tipo, inicial)

função DECL_FUNCAO():                                              [_parse_func_decl]
    nome ← CONSUMIR(IDENTIFIER); CONSUMIR("(")
    params ← []
    se ESPIAR() ≠ ")":
        repetir: se ESPIAR() é TYPE: AVANÇAR()          // tipo do parâmetro é descartado
                 params.anexar(CONSUMIR(IDENTIFIER).lexema)
        enquanto CASAR(",")
    CONSUMIR(")")
    retornar FuncDecl(nome, params, BLOCO_SEQ())      // corpo exige "{"

função DECL_CLASSE():                                              [_parse_class_decl]
    nome ← CONSUMIR(IDENTIFIER)
    super ← CONSUMIR(IDENTIFIER) se CASAR(EXTENDS) senão nulo
    CONSUMIR("{"); atributos ← []; métodos ← []
    enquanto ESPIAR() ∉ {"}", END_OF_FILE}:
        se CASAR(VAR):  atributos.anexar(DECL_VARIAVEL("var"))
```

```
        senão se CASAR(FUNC): métodos.anexar(DECL_FUNCAO())
        senão FALHAR(SYN_EXPECTED_TOKEN, "Esperado 'var' ou 'func' na definicao da classe")
    CONSUMIR("}")
    retornar ClassDecl(nome, super, atributos, métodos)

função BLOCO_SEQ() / BLOCO_PAR():                    [_parse_seq_block/_parse_par_block]
    CONSUMIR("{"); comandos ← []
    enquanto ESPIAR() ∉ {"}", END_OF_FILE}: comandos.anexar(COMANDO())
    CONSUMIR("}")
    retornar SeqBlock(comandos) / ParBlock(comandos)

função IF():      CONSUMIR("("); c ← EXPRESSAO(); CONSUMIR(")")
                  então ← COMANDO(); senão_ ← COMANDO() se CASAR(ELSE) senão nulo
                  retornar IfStmt(c, então, senão_)
função WHILE():   CONSUMIR("("); c ← EXPRESSAO(); CONSUMIR(")"); retornar WhileStmt(c, COMANDO())
função DO_WHILE(): corpo ← COMANDO(); CONSUMIR(WHILE); CONSUMIR("(")
                   c ← EXPRESSAO(); CONSUMIR(")"); CASAR(";")       // ";" opcional
                   retornar DoWhileStmt(corpo, c)

função FOR():                                                [_parse_for_stmt]
    CONSUMIR("("); init ← cond ← passo ← nulo
    se CASAR(VAR):
        nome ← CONSUMIR(IDENTIFIER)
        se CASAR(IN):                                 // for-in
            iterável ← EXPRESSAO(); CONSUMIR(")")
            retornar ForStmt(iterador=nome, iterável, corpo=COMANDO())
        init ← VarDecl(nome, "var", EXPRESSAO() se CASAR("=") senão nulo); CONSUMIR(";")
    senão se ESPIAR() é TYPE:
        t ← AVANÇAR(); init ← DECL_VARIAVEL(t.lexema, exige_ponto_virgula=falso); CONSUMIR(";")
    senão se não CASAR(";"):
        e ← EXPRESSAO()
        init ← CAUDA_ATRIBUICAO(e, falso) se CASAR("=") senão e; CONSUMIR(";")
    se não CASAR(";"): cond ← EXPRESSAO(); CONSUMIR(";")
    se ESPIAR() ≠ ")":
        e ← EXPRESSAO(); passo ← CAUDA_ATRIBUICAO(e, falso) se CASAR("=") senão e
    CONSUMIR(")")
    retornar ForStmt(init, cond, passo, corpo=COMANDO())
```

Fonte: Elaboração própria (2026).

Código 14 — Expressões por precedência [`_parse_or` a `_parse_primary`]

```
EXPRESSAO()  → OU()
OU()         → e ← E();      enquanto CASAR("||"): e ← Binária("||", e, E())
E()          → e ← COMP();   enquanto CASAR("&&"): e ← Binária("&&", e, COMP())
COMP()       → e ← TERMO();  enquanto ESPIAR() ∈ {==,!=,<,>,<=,>=}: op ← AVANÇAR(); e ←
↪  Binária(op, e, TERMO())
TERMO()      → e ← FATOR();  enquanto ESPIAR() ∈ {+,-}:   op ← AVANÇAR(); e ← Binária(op, e,
↪  FATOR())
FATOR()      → e ← UNARIA(); enquanto ESPIAR() ∈ {*,/,%}: op ← AVANÇAR(); e ← Binária(op, e,
↪  UNARIA())
UNARIA()     → se ESPIAR() ∈ {-,+,!}: op ← AVANÇAR(); retornar Unária(op, UNARIA())
               retornar PRIMARIA()

função PRIMARIA():                                           [_parse_primary]
    t ← ESPIAR()
    se CASAR(FORK):
        nome ← CONSUMIR(IDENTIFIER, "Esperado nome de funcao global apos fork")
        args ← ARGUMENTOS_ENTRE("(", ")")
        destino ← EXPRESSAO() se CASAR(ON) senão nulo      // expressão completa
        nó ← ForkExpr(nome, args, destino)
    senão se CASAR(JOIN):
        CONSUMIR("("); h ← EXPRESSAO()
        tempo ← EXPRESSAO() se CASAR(",") senão nulo; CONSUMIR(")")
        nó ← JoinExpr(h, tempo)
    senão se CASAR(MATRIX):   nó ← MatrixCreateExpr(três expressões entre "(" "," "," ")")
```

```
    senão se CASAR("["):      nó ← ListLiteral(lista separada por "," até "]")
    senão se CASAR(C_CHANNEL):nó ← CChannelExpr(ip, porta)
    senão se CASAR(RECEIVE):  nó ← ReceiveExpr(canal)
    senão se CASAR(INPUT):    nó ← InputExpr(prompt opcional)
    senão se CASAR(NEW):      nó ← NewExpr(CONSUMIR(IDENTIFIER), ARGUMENTOS_ENTRE("(", ")"))
    senão se CASAR(THIS):     nó ← ThisExpr
    senão se CASAR(NUMBER | STRING_LITERAL | BOOL_LITERAL): nó ← literal correspondente
    senão se CASAR(IDENTIFIER):
        nó ← FuncCallExpr(t.lexema, ARGUMENTOS até ")") se CASAR("(") senão
        ↪  IdentifierExpr(t.lexema)
    senão se CASAR("("):      nó ← EXPRESSAO(); CONSUMIR(")")
    senão FALHAR(SYN_EXPECTED_EXPRESSION, "Expressao esperada")

    repetir:                                          // sufixos, associativos à esquerda
        se CASAR("."):
            p ← CONSUMIR(IDENTIFIER)
            nó ← MethodCall(nó, p, ARGUMENTOS até ")") se CASAR("(") senão PropertyAccess(nó, p)
        senão se CASAR("["):
            nó ← IndexExpr(nó, EXPRESSAO()); CONSUMIR("]")
        senão sair
    retornar nó
```

Fonte: Elaboração própria (2026).

B.3 Tabela de símbolos

Código 15 — Estruturas da tabela de símbolos

```
estrutura SIMBOLO:
    nome, tipo                      // tipo: "var", "number", "int", "string", "bool",
                                    //       "list", "matrix", "c_channel", "task", nome de
                                    ↪  classe,
                                    //       "FUNC" ou "CLASS"
    params[], atributos{nome→tipo}, métodos{nome→params}
    tipo_valor : ValueType | nulo    // tipo estrutural (elementos de listas, resultado de task)

estrutura VALUE_TYPE: tipo_base, elementos (alternativas, sem posição), resultado (para task)

pilha_escopos  : lista de dicionários nome → SIMBOLO   // escopo [0] = global
escopos_pub    : lista de {id "sN", kind, name, parent, line}   // publicada na Web didática
simbolos_pub   : lista de {name, kind, type, scope, line, params}
pilha_ids      : ids dos escopos abertos (para "parent")
pilha_nós      : nós da AST em validação (define o kind do escopo e do símbolo)
especializado  : verdadeiro durante a reanálise de tarefas Fork/Join (não publica nada)
```

Fonte: Elaboração própria (2026).

Código 16 — Operações da tabela de símbolos

```
procedimento ENTRAR_ESCOPO():
    pilha_escopos.empilhar({})
    se especializado: retornar
    nó ← pilha_nós.topo (ou nulo no início)
    kind ← caso nó seja ClassDecl → "class"
                        FuncDecl  → "method" se dentro de classe senão "function"
                        SeqBlock  → "seq"; ParBlock → "par"; ForStmt → "for"
                        nulo      → "global"
    id ← "s" + |escopos_pub|
```

```
    escopos_pub.anexar({id, kind, name (classe, função ou Classe.método), parent ←
    ↪  pilha_ids.topo, line})
    pilha_ids.empilhar(id)

procedimento SAIR_ESCOPO():
    se |pilha_escopos| > 1:                 // o global nunca é desempilhado
        pilha_escopos.desempilhar()
        se não especializado: pilha_ids.desempilhar()

procedimento DEFINIR(nome, tipo, linha, params=[]):
    se nome ∈ pilha_escopos.topo:
        relatar(linha, "Variavel 'nome' ja declarada.", SEM_REDECLARED); retornar
    pilha_escopos.topo[nome] ← SIMBOLO(nome, tipo, params)
    se especializado: retornar
    nó ← pilha_nós.topo; kind_escopo ← kind do escopo atual
    kind ← caso nó seja VarDecl   → "attribute" se kind_escopo = "class" senão "variable"
                        FuncDecl  → se tipo = "FUNC": "method"/"function" senão "parameter"
                        ClassDecl → "class"
                        outro     → "iterator"          // variável do for-in
    tipo_pub ← nulo se kind ∈ {function, method, class}
               senão "var" se tipo ∈ {unknown, FUNC, CLASS} senão tipo
    simbolos_pub.anexar({nome, kind, tipo_pub, escopo atual, linha, params se função/método})

função BUSCAR(nome):
    para escopo de pilha_escopos.topo até pilha_escopos[0]:
        se nome ∈ escopo: retornar escopo[nome]
    retornar nulo                           // escopo léxico, do mais interno ao global
```

Fonte: Elaboração própria (2026).

B.4 Analisador semântico e tratamento de erros

Código 17 — Entrada do analisador semântico [`analyze`]

```
função ANALISAR(programa):
    // Passo 1: coleta antecipada (só nível global)            [_collect_declarations]
    para cada comando c do programa:
        se c é FuncDecl:
            se c.nome ∈ NATIVAS: relatar(SEM_RESERVED_NAME)
            funções[c.nome] ← SIMBOLO(c.nome, "FUNC", c.params)
        se c é ClassDecl:
            registrar nó, superclasse, inicializador sintético dos atributos
            classes[c.nome] ← SIMBOLO("CLASS", atributos{nome→tipo inferido},
            ↪  métodos{nome→params})
    para cada classe: CADEIA_HERANCA(classe)   // relata SEM_INHERITANCE_CYCLE se houver ciclo
    COLETAR_FUNCOES_FJ()     // fecho: funções/métodos que contêm ou chamam fork/join
    fj_ativo ← programa contém ForkExpr ou JoinExpr
    globais ← funções ∪ {variáveis globais declaradas com seu tipo inferido}

    // Passo 2: validação em ordem do texto
    para cada comando c do programa: VALIDAR(c, valor_exigido=falso)
    juntar assinaturas provisórias de fork que não tiveram chamada especializada
    retornar não houve_erros
```

Fonte: Elaboração própria (2026).

Código 18 — Relato de erros [`_report`]

```
procedimento RELATAR(linha, mensagem, código):
    se especializado e mensagem não começa com "FJ-": retornar   // evita duplicar erros comuns
    se mensagem começa com "FJ-":
        chave ← (linha, classe atual, função atual, mensagem)
        se chave já relatada: retornar; senão marcar chave
    diagnósticos.anexar({phase: "semantic", severity: "error", code, message, line, column: nulo})
    se emitir_diagnosticos: imprimir "[ERRO SEMANTICO] Linha L (Funcao: f): msg" em stderr
    houve_erros ← verdadeiro
```

Fonte: Elaboração própria (2026).

Código 19 — Inferência e compatibilidade de tipos

```
função INFERIR(nó):
    caso nó seja
        ForkExpr         → "task"
        NumberExpr       → "number";  StringExpr, InputExpr, ReceiveExpr → "string"
        BoolExpr         → "bool";    CChannelExpr → "c_channel"
        ListLiteral      → "list";    MatrixCreateExpr → "matrix"
        NewExpr          → nome da classe;  ThisExpr → classe atual
        IdentifierExpr   → BUSCAR(nome).tipo  (tipo estrutural se for task)  ou "unknown"
        UnaryExpr        → "bool" se op = "!" senão INFERIR(operando)
        BinaryExpr       → "bool" para comparações e lógicos
                           "string" se op = "+" e algum lado é string
                           "list"   se op = "+" e algum lado é list
                           "number" nos demais
        PropertyAccess   → tipo do atributo na cadeia de herança, ou "unknown"
        FuncCallExpr     → random/exp/len/to_number → "number"; substr/to_string → "string"
                           range/parse_ints → "list"; demais → "unknown"
        IndexExpr        → "number" se o objeto é list, senão "unknown"
        outro            → "unknown"

função COMPATIVEL(declarado, real):
    se declarado ou real ∈ {"var", "unknown"}: retornar verdadeiro
    se {declarado, real} = {"int", "number"}: retornar verdadeiro
    retornar declarado = real
```

Fonte: Elaboração própria (2026).

Código 20 — Validação por nó [`validate_node`]

```
procedimento VALIDAR(nó, valor_exigido=verdadeiro):
    pilha_nós.empilhar(nó)
    caso nó seja
    VarDecl:
        VALIDAR(nó.inicial); real ← INFERIR(nó.inicial)
        declarado ← real se nó.tipo = "var" e real ≠ "unknown" senão nó.tipo
        se não COMPATIVEL(declarado, real): relatar(SEM_TYPE_MISMATCH)
        DEFINIR(nó.nome, declarado); guardar tipo estrutural no símbolo
    Assignment:
        VERIFICAR_GLOBAL_NA_TAREFA(nó.nome)                    // FJ-SEM-05
        s ← BUSCAR(nó.nome); se s = nulo: relatar(SEM_UNDECLARED_VARIABLE)
        VALIDAR(nó.valor)
        se s e não COMPATIVEL(s.tipo, INFERIR(nó.valor)): relatar(SEM_TYPE_MISMATCH)
    IdentifierExpr:
        VERIFICAR_GLOBAL_NA_TAREFA(nó.nome)
        se BUSCAR(nó.nome) = nulo: relatar(SEM_UNDECLARED_VARIABLE)
    BinaryExpr:
        VALIDAR(esq); VALIDAR(dir); te, td ← INFERIR(esq), INFERIR(dir)
        se fj_ativo e op aritmético e (te ou td ∈ {task, c_channel, classe}): relatar(FJ-SEM-06)
```

```
        se op ∈ {+, ==, !=, <, <=, >, >=} e um lado é string e o outro é tipo conhecido ≠ string:
            relatar(SEM_TYPE_MISMATCH)
        se op ∈ {-, *, /, %} e algum lado conhecido não é number/int:
        ↪  relatar(SEM_NUMERIC_OPERANDS)
    UnaryExpr:
        VALIDAR(operando); se fj_ativo e op ∈ {+,-} e operando é task/canal/objeto:
        ↪  relatar(FJ-SEM-06)
    SeqBlock, ParBlock:
        ENTRAR_ESCOPO(); para cada c: VALIDAR(c, falso); SAIR_ESCOPO()
    IfStmt:     VALIDAR(cond); VALIDAR(então, falso); VALIDAR(senão, falso)
    WhileStmt, DoWhileStmt:
        VALIDAR(cond); profundidade_laço += 1; VALIDAR(corpo, falso); profundidade_laço −= 1
    ForStmt:
        ENTRAR_ESCOPO()
        se for-in: VALIDAR(iterável); DEFINIR(iterador, "var")
                   tipo estrutural do iterador ← união dos elementos do iterável
        senão: VALIDAR(init, falso); VALIDAR(cond); VALIDAR(passo, falso)
        profundidade_laço += 1; VALIDAR(corpo, falso); profundidade_laço −= 1
        SAIR_ESCOPO()
    BreakStmt, ContinueStmt:
        se profundidade_laço = 0: relatar(SEM_OUTSIDE_LOOP)
    ClassDecl:
        se superclasse ∉ classes: relatar(SEM_UNDECLARED_SUPERCLASS)
        DEFINIR(nome, "CLASS"); ENTRAR_ESCOPO(); classe_atual ← nome
        para cada atributo: VALIDAR(atributo); atualizar tipo do atributo na classe
        para cada método: VALIDAR(método)
        restaurar classe_atual; SAIR_ESCOPO()
    FuncDecl:
        DEFINIR(nome, "FUNC", params); ENTRAR_ESCOPO(); função_atual ← nome
        para cada p em params: DEFINIR(p, "var")
        VALIDAR(corpo)                                  // abre o escopo "seq" do corpo
        restaurar função_atual; SAIR_ESCOPO()
    ReturnStmt:
        VALIDAR(valor); se em análise especializada: acumular tipo do retorno
    FuncCallExpr:
        se em tarefa e nome resolve algo que não é a func global: relatar(FJ-SEM-05)
        se nome ∉ funções e nome ∉ NATIVAS: relatar(SEM_UNDECLARED_FUNCTION)
        senão se nome ∈ funções e |args| ≠ |params|: relatar(SEM_ARITY)
        VALIDAR cada argumento
        se fj_ativo e a chamada envolve FJ: tipo ← VERIFICAR_FUNCAO(especializada nos argumentos)
        se nome ∈ {substr(string,number,number), to_string(number), len(coleção)}:
            aridade errada → SEM_ARITY; tipo conhecido errado → SEM_TYPE_MISMATCH
    MethodCall:
        VALIDAR(objeto) e argumentos; tipo ← INFERIR(objeto)
        se tipo é classe e método ∉ cadeia de herança: relatar(SEM_UNDECLARED_METHOD)
        senão se |args| ≠ |params do método|: relatar(SEM_ARITY)
        se lista.append/push_back com string: relatar(SEM_UNSUPPORTED_FEATURE)
        se fj_ativo: atualizar tipos estruturais (append, pop, size);
                     em tarefa, método de tipo desconhecido → FJ-SEM-05
    PropertyAccess, PropertyAssign:
        se objeto é classe e atributo ∉ cadeia de herança: relatar(SEM_UNDECLARED_ATTRIBUTE)
        PropertyAssign com tipo incompatível: relatar(SEM_TYPE_MISMATCH)
    NewExpr:
        se classe ∉ classes: relatar(SEM_UNDECLARED_CLASS)
        se em tarefa: verificar inicializadores de toda a cadeia de herança isolados
    ListLiteral:   elemento string → relatar(SEM_UNSUPPORTED_FEATURE)
    IndexExpr:     objeto string   → relatar(SEM_UNSUPPORTED_FEATURE)   // usar substr
    IndexAssign:   VALIDAR(alvo, valor); se fj_ativo: estender tipo da coleção
    ThisExpr:      em tarefa global → relatar(FJ-SEM-05)
    InputExpr, SendStmt, CChannelExpr, ReceiveExpr:
        em tarefa → relatar(FJ-SEM-05); VALIDAR os filhos
    ForkExpr:      VALIDAR_FORK(nó); se não valor_exigido: relatar(FJ-SEM-07)
    JoinExpr:      VALIDAR_JOIN(nó, valor_exigido)
    pilha_nós.desempilhar()
```

Fonte: Elaboração própria (2026).

Código 21 — Regras de Fork/Join [`_validate_fork`, `_validate_join`]

```
procedimento VALIDAR_FORK(nó):
    VALIDAR cada argumento; VALIDAR(nó.destino)
    se destino e tipo(destino) ∉ {string, var, unknown}: relatar(FJ-SEM-04)
    f ← funções_globais[nó.nome]; s ← BUSCAR(nó.nome)
    se f = nulo ou s sombreia a global: relatar(FJ-SEM-01); retornar
    se s ≠ nulo e s.tipo ≠ "FUNC":      relatar(FJ-SEM-01); retornar
    se |args| ≠ |f.params|:             relatar(FJ-SEM-02 / SEM_ARITY); retornar
    tipos_args ← tipo estrutural de cada argumento (resolvendo chamadas)
    para cada t em tipos_args: VERIFICAR_TRANSPORTE(t, "argumento")
    resultado ← VERIFICAR_FUNCAO(f, tipos_args, isolada=verdadeiro)
    VERIFICAR_TRANSPORTE(resultado, "retorno")
    tipo(nó) ← task(resultado)
    registrar assinatura (tipos_args, resultado) em fork_signatures
        (ou em assinaturas provisórias, se analisada fora de contexto concreto)

procedimento VALIDAR_JOIN(nó, valor_exigido):
    VALIDAR(handle); VALIDAR(timeout)
    h ← tipo estrutural do handle
    se h não é task:
        se h ∈ {var, unknown}:
            para cada alternativa conhecida não-task em h.elementos: relatar(FJ-SEM-03)
            anotar em unresolved_fj ("join generico depende do tipo do argumento")
        senão relatar(FJ-SEM-03)
    se timeout e tipo(timeout) ∉ {number, int, var, unknown}: relatar(FJ-SEM-04)
    tipo(nó) ← h.resultado (ou unknown)
    se valor_exigido e h.resultado = void: relatar(FJ-SEM-08)

procedimento VERIFICAR_TRANSPORTE(t, papel):
    se t ∈ {unknown, var}: verificar alternativas; anotar "tipo nao comprovado" em unresolved_fj
    senão se t ∈ {list, matrix}: verificar cada tipo de elemento
    senão se t ∉ {number, int, bool, string, void}: relatar(FJ-SEM-06)   // objeto, canal, task

procedimento VERIFICAR_GLOBAL_NA_TAREFA(nome):
    se não isolada: retornar
    encontrar o escopo mais interno que define nome
    se é o global e o símbolo não é FUNC nem CLASS: relatar(FJ-SEM-05)

função VERIFICAR_FUNCAO(f, tipos_args, isolada, dono=nulo):          // especialização
    chave ← (f, tipos_args, isolada, dono, tipos globais, provisória)
    se chave ∈ cache: retornar cache[chave]
    se (f, isolada, dono) já em análise:          // recursão
        anotar "retorno recursivo nao comprovado" em unresolved_fj; retornar unknown
    salvar estado; especializado ← verdadeiro
    pilha_escopos ← [cópia dos globais]           // nunca herda locais do chamador
    se dono: abrir escopo com atributos da cadeia de herança do dono
    abrir escopo; DEFINIR cada parâmetro com o tipo do argumento
    VALIDAR(f.corpo, falso)                       // só erros "FJ-" são relatados aqui
    resultado ← união dos tipos de return, ou void se não houver return
    cache[chave] ← resultado; restaurar estado
    retornar resultado
```

Fonte: Elaboração própria (2026).

B.5 Gerador de código intermediário

Código 22 — Tradução de Fork/Join e `par` para TAC

```
EXPR(ForkExpr f(a1..an) on d):
    para cada ai: ti ← novo_temp; emitir "ti = capture <EXPR(ai)>"     // cópia por valor
    dest ← "\"local\"" se d = nulo senão salvar(EXPR(d))
```

```
    retornar salvar("fork f(t1, …, tn) on dest")

JOIN(JoinExpr h, timeout t, valor_exigido):
    hv ← EXPR(h); se t: hv ← salvar(hv)                // t pode alterar a variável de h
    instr ← "join hv" + (", timeout " + EXPR(t) se t)
    se valor_exigido: retornar salvar(instr)  senão emitir instr

COMANDO(ForkExpr) → recusa TAC_UNSUPPORTED_FEATURE ("fork descartado; requer handle (FJ-SEM-07)")

COMANDO(ParBlock com n comandos):
    R ← rótulo PAR; W ← rótulo PAR_WAIT; Bi ← rótulo PAR_BRANCH por comando
    emitir "par_begin R, n"; para cada i: emitir "par_spawn R, i, Bi"
    emitir "goto W"
    para cada i: emitir "Bi:"; "par_branch R, i"
                 traduzir comando i com pilhas de break/continue vazias
                 (return vira "par_return R, i[, value v]")
                 emitir "par_end_branch R, i"
    emitir "W:"; "par_barrier R"; "par_end R"
```

Fonte: Elaboração própria (2026).

B.6 MiniCompilador do Teste 11

Código 23 — Programa principal do MiniCompilador

```
entrada ← input("Digite uma expressão infixada: ")
mc ← novo MiniCompilador
mc.compilar(entrada)

procedimento compilar(entrada)
    e ← entrada
    analisarLexico()
    se erro = "" então
        imprimirTokens()               ▷ os tokens saem antes da análise sintática, como na
        ↪  referência
        analisarSintatico()
    se erro = "" então analisarSemantico()
    se erro = "" então gerarSaidas(avaliar(raiz))   ▷ avaliar também pode falhar (estouro de 32
    ↪  bits)
    se erro ≠ "" então imprimir "❌ " + erro       ▷ nada de AST, TAC, Assembly ou resultado
```

Fonte: Elaboração própria (2026).

Código 24 — Análise léxica do MiniCompilador

```
procedimento analisarLexico()
    i ← 0; n ← len(e)
    enquanto i < n e erro = "":
        c ← substr(e, i, 1)
        se c é espaço ou tabulação: i ← i + 1
        senão se c = "#": i ← n                        ▷ comentário até o fim da linha
        senão se c é dígito:
            ini ← i
            enquanto i < n e substr(e, i, 1) é dígito: i ← i + 1
            num ← substr(e, ini, i − ini)
            se o próximo caractere é ".":       falhar("léxico", "número decimal não suportado na
            ↪  " + coluna(ini))
            senão se len(num) > 1 e num começa com "0":
```

```
                                            falhar("léxico", "zero à esquerda não suportado
                                            ↪  em '" + num + "' …")
            senão se len(num) > 10 ou valor(num) > 2147483647:
                                            falhar("léxico", "constante '" + num + "' fora
                                            ↪  do intervalo 0..2147483647 …")
            senão adicionarToken(NUMERO, ini, i − ini)
        senão:
            se c ∈ {+, −, *}:   adicionarToken(OPERADOR, i, 1)
            senão se c ∈ {(, )}: adicionarToken(PARENTESE, i, 1)
            senão rejeitarCaractere(c, i)
            i ← i + 1
    se erro = "" e não há tokens: falhar("léxico", "entrada vazia")

procedimento rejeitarCaractere(c, i)            ▷ mensagem específica para cada caso proibido
    "/"          → "divisão '/' não suportada"
    "="          → "atribuição '=' não suportada"
    letra ou "_" → "identificador não suportado"
    "."          → "número decimal não suportado"
    imprimível   → "caractere desconhecido '<c>'"
    outro        → "caractere não suportado"
    (todas terminam com " na " + coluna(i))
```

Fonte: Elaboração própria (2026).

Código 25 — Gramática de expressões do MiniCompilador

```
expr   → term   { ("+" | "−") term }
term   → factor { "*" factor }
factor → NUMERO | "(" expr ")"
```

Fonte: Elaboração própria (2026).

Código 26 — Análise sintática do MiniCompilador

```
procedimento novoNo(t, esq, dir)                ▷ acrescenta o nó e o deixa em ultimo
    ntok.append(t); nesq.append(esq); ndir.append(dir); ultimo ← len(ntok) − 1

procedimento parseExpr()
    parseTerm(); no ← ultimo
    enquanto erro = "" e atual() ∈ {"+", "−"}:
        t ← pos; pos ← pos + 1
        parseTerm()
        novoNo(t, no, ultimo); no ← ultimo        ▷ o nó antigo vira filho esquerdo:
        ↪  associatividade à esquerda
    ultimo ← no

procedimento parseTerm()                          ▷ igual a parseExpr, com "*" e parseFactor

procedimento parseFactor()
    se erro ≠ "": retornar
    se pos ≥ número de tokens: falhar("sintático", "expressão incompleta: esperado número ou '('
    ↪  no fim da entrada")
    senão se lexema(pos) = "(":
        t ← pos; pos ← pos + 1
        parseExpr()
        se erro = "" e atual() ≠ ")":
            se acabaram os tokens: falhar("sintático", "'(' da coluna X sem ')' correspondente")
            senão                  falhar("sintático", "esperado ')' e encontrado '<lexema>' na
            ↪  coluna Y")
        pos ← pos + 1                               ▷ o ")" não vira nó: a AST já guarda o
        ↪  agrupamento
```

```
    senão se tipos[pos] = NUMERO:
        pos ← pos + 1; novoNo(pos − 1, −1, −1)    ▷ folha
    senão se lexema(pos) = "−": falhar("sintático", "menos unário não suportado na coluna …")
    senão                       falhar("sintático", "esperado número ou '(' e encontrado
    ↪  '<lexema>' …")

procedimento analisarSintatico()
    pos ← 0; parseExpr(); raiz ← ultimo
    se erro = "" e pos < número de tokens:
        falhar("sintático", "token '<lexema>' inesperado depois do fim da expressão, na coluna …")
```

Fonte: Elaboração própria (2026).

Código 27 — Análise semântica e tratamento de erros do MiniCompilador

```
procedimento analisarSemantico()
    ▷ um registrador por nó; só r0–r12 estão livres (r13–r15 são SP, LR e PC)
    se len(ntok) > 13: falhar("semântico", "a AST tem N nós e o Assembly comporta no máximo 13
    ↪  (registradores r0 a r12)")

função avaliar(no)                                ▷ pós-ordem; também é a última verificação
↪  semântica
    se no é folha: retornar valor(lexema(ntok[no]))
    a ← avaliar(nesq[no]); b ← avaliar(ndir[no])
    v ← a op b, com op = lexema(ntok[no]) ∈ {+, −, *}
    se v > 2147483647 ou v < −2147483648:
        falhar("semântico", "o resultado de '<expressão parentizada>' não cabe em 32 bits com
        ↪  sinal")
    retornar v
```

Fonte: Elaboração própria (2026).

Código 28 — Tabela de símbolos do MiniCompilador

```
procedimento registrarSimbolos()
    para cada nó folha no, na ordem de criação:
        v ← valor(lexema(ntok[no]))
        se v não está em svals:
            svals.append(v); stoks.append(ntok[no])
            j ← último índice
            enquanto j > 0 e svals[j − 1] > svals[j]:   ▷ inserção ordenada
                trocar svals[j] com svals[j − 1]; trocar stoks[j] com stoks[j − 1]; j ← j − 1

procedimento imprimirSimbolos()
    para cada j: imprimir " ◇  " + lexema(stoks[j]) + " → CONST"
```

Fonte: Elaboração própria (2026).

Código 29 — Geração da AST, do TAC e do Assembly ARMv7

```
procedimento imprimirAST(no, nivel)            ▷ ordem simétrica invertida: direita, nó, esquerda
    se no ≥ 0:
        imprimirAST(ndir[no], nivel + 1)
        imprimir (2 × nivel espaços) + lexema(ntok[no])
        imprimirAST(nesq[no], nivel + 1)

função gerarTAC(no)                            ▷ pós-ordem; devolve o endereço do valor do nó
```

```
    se no é folha: retornar lexema(ntok[no])
    a1 ← gerarTAC(nesq[no]); a2 ← gerarTAC(ndir[no])
    t ← "t" + temp; temp ← temp + 1
    imprimir t + " = " + a1 + " " + op + " " + a2
    retornar t

função formatExpr(no)                          ▷ expressão totalmente parentizada, para os
↪  comentários
    se no é folha: retornar lexema
    retornar "(" + formatExpr(esq) + " " + op + " " + formatExpr(dir) + ")"

função gerarARM(no)                            ▷ pós-ordem; registrador novo por nó, sem
↪  reaproveitamento
    se no é folha:
        r ← "r" + reg; reg ← reg + 1
        se valor > 255: imprimir "    LDR    " + r + ", =" + k     ▷ não cabe no imediato de MOV
        senão          imprimir "    MOV    " + r + ", #" + k
        retornar r
    a1 ← gerarARM(esq); a2 ← gerarARM(dir)
    r ← "r" + reg; reg ← reg + 1                ▷ alocado depois dos dois filhos
    imprimir "    " + ADD|SUB|MUL (campo de 6 colunas) + " " + r + ", " + a1 + ", " + a2
             + "    @ resultado de '" + formatExpr(no) + "'"
    retornar r

procedimento gerarSaidas(resultado)
    registrarSimbolos()
    imprimir título "🌳 Árvore Sintática Abstrata (AST):"; imprimirAST(raiz, 0)
    imprimir título "📦 Código de Três Endereços (TAC):"; gerarTAC(raiz)
    imprimir título "⚙ Assembly ARMv7 (CPUlator):"
    imprimir ".global _start", ".text", "_start:"
    gerarARM(raiz)
    imprimir "    b .    @ loop infinito para parar o programa"
    imprimir título "📑 Tabela de Símbolos:"; imprimirSimbolos()
    imprimir título "🧾 Resultado final da expressão:"; imprimir "➡  " + resultado + ".0"
```

Fonte: Elaboração própria (2026).

APÊNDICE C — SAÍDAS DOS PROGRAMAS DE TESTE

Este apêndice reproduz as saídas citadas no capítulo 7. Nos registros da execução entre dois computadores, o terceiro octeto dos endereços IP aparece como `x` e os nomes das máquinas, como `pc-A` e `pc-B`.

C.1 Programas de teste

Código 30 — Saída do Teste 1 (entrada: +, 2, 3)

```
==== Calculadora cliente-servidor ====
Opcoes: +, -, *, /
Operacao (+,-,*,/): Operando 1: Operando 2: Servidor recebeu: + 2 3
Resultado: 5.000000
```

Fonte: Elaboração própria (2026).

Código 31 — Saída do Teste 2

```
Fibonacci:
0
1
1
2
3
5
8
13
Fatorial de 5: 120
```

Fonte: Elaboração própria (2026).

Código 32 — Trecho da saída do Teste 3

```
==== Teste 3: neuronio simples com classes e objetos ====
Entrada: 1 Saida desejada: 0
Iteracao: 1
Soma ponderada: 1
Saida: 1
Erro: -1
Peso entrada: 0.5
Peso bias: 0.5
Iteracao: 2
Soma ponderada: 0.98
Saida: 1
Erro: -1
Peso entrada: 0.49
Peso bias: 0.49
[... iterações 3 a 50 omitidas ...]
Erro: 0
Peso entrada: -3.08781e-16
Peso bias: -3.08781e-16
Neuronio aprendeu dinamicamente.
Iteracoes totais: 51
```

```
Peso final: -3.08781e-16
Peso bias final: -3.08781e-16
Valor desejado: 0
```

Fonte: Elaboração própria (2026).

Código 33 — Saída do Teste 4

```
==== Teste 4: rede neural XOR com backpropagation ====
Input: [0, 0], Predicted Output: 0.00684395
Input: [0, 1], Predicted Output: 0.986493
Input: [1, 0], Predicted Output: 0.98925
Input: [1, 1], Predicted Output: 0.0140296
```

Fonte: Elaboração própria (2026).

Código 34 — Recomendações do Teste 5

```
Produtos recomendados para voce:
Laptop probabilidade: 0.723522
Tablet probabilidade: 0.711771
Fones de ouvido probabilidade: 0.716263
Camisa probabilidade: 0.564145
Jaqueta probabilidade: 0.723522
Sapatos probabilidade: 0.711771
Geladeira probabilidade: 0.716263
Maquina de lavar probabilidade: 0.7191
Ar condicionado probabilidade: 0.723522
Nao-ficcao probabilidade: 0.716263
Ficcao cientifica probabilidade: 0.564145
Fantasia probabilidade: 0.7191
```

Fonte: Elaboração própria (2026).

Código 35 — Saída do Teste 6 (entrada: `10 -3 -40 80 70 -100`)

```
==== Ordenacao com Quicksort ====
Insira os elementos do vetor separados por espaco:
Vetor original: [10, -3, -40, 80, 70, -100]
Vetor ordenado: [-100, -40, -3, 10, 70, 80]
```

Fonte: Elaboração própria (2026).

Código 36 — Início do TAC gerado para o Teste 7

```
func quicksort_iterativo:
t0 = []
stack = t0
param arr
t1 = call len, 1
t2 = t1 > 1
ifFalse t2 goto ENDIF_0
```

```
param 0
t3 = call stack.append, 1
param arr
t4 = call len, 1
t5 = t4 - 1
param t5
t6 = call stack.append, 1
ENDIF_0:
WHILE_1:
param stack
t7 = call len, 1
t8 = t7 > 0
ifFalse t8 goto ENDWHILE_2
t9 = call stack.pop, 0
fim = t9
t10 = call stack.pop, 0
inicio = t10
t11 = inicio >= fim
ifFalse t11 goto ENDIF_3
goto WHILE_1
ENDIF_3:
t12 = arr[fim]
pivot = t12
t13 = inicio - 1
i = t13
j = inicio
WHILE_4:
t14 = j < fim
ifFalse t14 goto ENDWHILE_5
t15 = arr[j]
t16 = t15 <= pivot
ifFalse t16 goto ENDIF_6
t17 = i + 1
i = t17
t18 = arr[i]
tmp = t18
t19 = arr[j]
arr[i] = t19
arr[j] = tmp
ENDIF_6:
```

Fonte: Elaboração própria (2026).

Código 37 — Trecho da saída do Teste 8 em modo paralelo remoto

```
=== TESTE 8: 160 avaliacoes, 4 avaliadores ===
MODO par
[Professor] Concluiu: Aluno 1 - Atividade 1 (id 0, nota 73)
[Professor] Concluiu: Aluno 1 - Atividade 2 (id 1, nota 76)
[Professor] Concluiu: Aluno 1 - Atividade 3 (id 2, nota 51)
[Professor] Concluiu: Aluno 1 - Atividade 4 (id 3, nota 6)
[... 154 linhas de avaliações concluídas ...]
[Monitor 3] Concluiu: Aluno 39 - Atividade 4 (id 155, nota 6)
[Monitor 3] Concluiu: Aluno 40 - Atividade 1 (id 156, nota 45)
[Monitor 3] Concluiu: Aluno 40 - Atividade 2 (id 157, nota 15)
[Monitor 3] Concluiu: Aluno 40 - Atividade 3 (id 158, nota 47)
[Monitor 3] Concluiu: Aluno 40 - Atividade 4 (id 159, nota 7)
AVALIADOR 3 Monitor 3 40
TOTAL 160
UNICOS 160
DUPLICADOS 0
PERDIDOS 0
SOMA_NOTAS 7630
```

Fonte: Elaboração própria (2026).

Código 38 — Saída do Teste 9 em modo sequencial

```
=== TESTE 9: 160 avaliacoes + Mandelbrot ===
MODO seq
AVALIADOR 0 Professor 40
AVALIADOR 1 Monitor 1 40
AVALIADOR 2 Monitor 2 40
AVALIADOR 3 Monitor 3 40
TOTAL 160
UNICOS 160
DUPLICADOS 0
PERDIDOS 0
SOMA_NOTAS 7728
=== MANDELBROT 50x20 max_iter=100 ===


                                .
                              ..=:.
                               @@@:
                           +@.@@@@@@@@@. :..
                          .@@@@@@@@@@@@@@:
                  .     #@@@@@@@@@@@@@@@@@@:
               .@:@@@- :@@@@@@@@@@@@@@@@@@@:
            - .@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@.
            - .@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
               .@:@@@- :@@@@@@@@@@@@@@@@@@@:
                  .     #@@@@@@@@@@@@@@@@@@:
                          .@@@@@@@@@@@@@@@:
                           +@.@@@@@@@@@. :..
                                 @@@:
                               ..=:.
                                  .


=== FIM MANDELBROT ===
```

Fonte: Elaboração própria (2026).

Código 39 — Trecho da saída do Teste 10 em modo paralelo (grau 3)

```
=== TESTE 10: 160 avaliacoes + Mandelbrot + Sierpinski ===
MODO par
AVALIADOR 0 Professor 40
AVALIADOR 1 Monitor 1 40
AVALIADOR 2 Monitor 2 40
AVALIADOR 3 Monitor 3 40
TOTAL 160
UNICOS 160
DUPLICADOS 0
PERDIDOS 0
SOMA_NOTAS 7728
[... fractal de Mandelbrot igual ao do Teste 9 ...]
=== SIERPINSKI grau=3 ===
############################
# ## ## ## ## ## ## ## ## #
############################
###   ######   ######   ###
# #   # ## #   # ## #   # #
###   ######   ######   ###
############################
# ## ## ## ## ## ## ## ## #
############################
```

```
#########         #########
# ## ## #         # ## ## #
#########         #########
###   ###         ###   ###
# #   # #         # #   # #
###   ###         ###   ###
#########         #########
# ## ## #         # ## ## #
#########         #########
###########################
# ## ## ## ## ## ## ## ## #
###########################
###   ######   ######   ###
# #   # ## #   # ## #   # #
###   ######   ######   ###
###########################
# ## ## ## ## ## ## ## ## #
###########################
=== FIM SIERPINSKI ===
```

Fonte: Elaboração própria (2026).

Código 40 — Saída do MiniCompilador no caso 1 do Teste 11

```
Digite uma expressão infixada:
🔍 Tokens gerados pelo analisador léxico:
[PARENTESE: '(']
[PARENTESE: '(']
[NUMERO: '2']
[OPERADOR: '+']
[NUMERO: '4']
[PARENTESE: ')']
[OPERADOR: '*']
[NUMERO: '7']
[PARENTESE: ')']
[OPERADOR: '-']
[PARENTESE: '(']
[NUMERO: '5']
[OPERADOR: '*']
[NUMERO: '4']
[PARENTESE: ')']

🌳 Árvore Sintática Abstrata (AST):
    4
  *
    5
-
    7
  *
      4
    +
      2

📦 Código de Três Endereços (TAC):
t1 = 2 + 4
t2 = t1 * 7
t3 = 5 * 4
t4 = t2 - t3

⚙ Assembly ARMv7 (CPUlator):
.global _start
.text
_start:
    MOV    r0, #2
    MOV    r1, #4
    ADD    r2, r0, r1    @ resultado de '(2 + 4)'
```

```
    MOV    r3, #7
    MUL    r4, r2, r3    @ resultado de '((2 + 4) * 7)'
    MOV    r5, #5
    MOV    r6, #4
    MUL    r7, r5, r6    @ resultado de '(5 * 4)'
    SUB    r8, r4, r7    @ resultado de '(((2 + 4) * 7) - (5 * 4))'
    b .    @ loop infinito para parar o programa

📑 Tabela de Símbolos:
 ◇ 2 → CONST
 ◇ 4 → CONST
 ◇ 5 → CONST
 ◇ 7 → CONST

🧾 Resultado final da expressão:
➡ 22.0
```

Fonte: Elaboração própria (2026).

C.2 Execução entre dois computadores

Código 41 — Início do registro de um trabalhador na máquina B (Teste 8)

```
[minipar_worker pc-B:29268] escutando em 192.168.x.22:7201 (host=pc-B pid=29268
↪  build_id=69d0f55f9f4d9e149f97818b34ca455aecb1515e1567660c8044069f843759b1)
[minipar_worker pc-B:29268] conexao de 192.168.x.105:50569 aceita (build_id ok)
[minipar_worker pc-B:29268] TASK task-1 fn=avaliar_lote de 192.168.x.105:50569: inicio
[minipar_worker pc-B:29268] RESULT task-1 fn=avaliar_lote ok em 314 ms
[minipar_worker pc-B:29268] conexao de 192.168.x.105:50569 encerrada
```

Fonte: Elaboração própria (2026).

Código 42 — Saída do coordenador na máquina A (Teste 8)

```
=== TESTE 8: 160 avaliacoes, 4 avaliadores ===
MODO par
[...]
AVALIADOR 3 Monitor 3 40
TOTAL 160
UNICOS 160
DUPLICADOS 0
PERDIDOS 0
SOMA_NOTAS 7630
```

Fonte: Elaboração própria (2026).

Código 43 — Saída do coordenador com um trabalhador indisponível

```
=== TESTE 8: 160 avaliacoes, 4 avaliadores ===
MODO par
[minipar_rt] join task-1 falhou: WORKER_UNAVAILABLE: worker 192.168.x.22:7201 indisponivel: tempo
↪  de conexao esgotado (3000 ms)
```

Fonte: Elaboração própria (2026).

Código 44 — Registro do trabalhador na máquina B (demonstração do runtime)

```
[minipar_worker pc-B:28212] escutando em 192.168.x.131:7000 (host=pc-B pid=28212
↪  build_id=minipar-demo-run03-v1)
[minipar_worker pc-B:28212] conexao de 192.168.x.31:62502 aceita (build_id ok)
[minipar_worker pc-B:28212] TASK task-1 fn=identidade de 192.168.x.31:62502: inicio
[minipar_worker pc-B:28212] RESULT task-1 fn=identidade ok em 0 ms
[minipar_worker pc-B:28212] TASK task-2 fn=soma_quadrados de 192.168.x.31:62502: inicio
[minipar_worker pc-B:28212] TASK task-3 fn=soma_quadrados de 192.168.x.31:62502: inicio
[minipar_worker pc-B:28212] RESULT task-2 fn=soma_quadrados ok em 149 ms
[minipar_worker pc-B:28212] RESULT task-3 fn=soma_quadrados ok em 150 ms
[minipar_worker pc-B:28212] TASK task-6 fn=trabalho_longo de 192.168.x.31:62502: inicio
```

Fonte: Elaboração própria (2026).

APÊNDICE D — CÓDIGO-FONTE DOS PROGRAMAS DE TESTE

Este apêndice reproduz o código-fonte MiniPar dos onze programas de teste. Os Testes 1 a 7 ficam em `tests/`, e os Testes 8 a 11, em `tests/2026_2/`.

D.1 Teste 1: calculadora cliente-servidor

Código 45 — Programa do Teste 1

```
# Calcula operação aritmética recebida como string (ex.: "+", "-", "*", "/")
func calcular(op, v1, v2) {
    if (op == "+") {
        return v1 + v2;
    }
    if (op == "-") {
        return v1 - v2;
    }
    if (op == "*") {
        return v1 * v2;
    }
    if (op == "/") {
        return v1 / v2;
    }
    return 0.0;
}

seq {
    print("==== Calculadora cliente-servidor ====");
    print("Opcoes: +, -, *, /");
    var requisicao = c_channel("127.0.0.1", 5101);
    var resposta = c_channel("127.0.0.1", 5102);

    par {
        # Thread servidor: recebe operação e operandos, devolve resultado
        seq {
            var op = receive(requisicao);
            var s_v1 = receive(requisicao);
            var s_v2 = receive(requisicao);
            var v1 = to_number(s_v1);
            var v2 = to_number(s_v2);
            print("Servidor recebeu:", op, s_v1, s_v2);
            var res = calcular(op, v1, v2);
            send(resposta, res);
        }
        # Thread cliente: lê input do usuário e envia ao servidor
        seq {
            var operacao = input("Operacao (+,-,*,/): ");
            var operando1 = input("Operando 1: ");
            var operando2 = input("Operando 2: ");
            send(requisicao, operacao);
            send(requisicao, operando1);
            send(requisicao, operando2);
            var resultado = receive(resposta);
            print("Resultado:", resultado);
        }
    }
}
```

Fonte: Elaboração própria (2026).

D.2 Teste 2: fatorial e Fibonacci em paralelo

Código 46 — Programa do Teste 2

```
par {
    seq {
        var n = 5;
        var fat = 1;
        var i = 1;
        while (i <= n) {
            fat = fat * i;
            i = i + 1;
        }
        print("Fatorial de 5:", fat);
    }
    seq {
        var limite = 8;
        var a = 0;
        var b = 1;
        var i = 0;
        print("Fibonacci:");
        while (i < limite) {
            print(a);
            var prox = a + b;
            a = b;
            b = prox;
            i = i + 1;
        }
    }
}
```

Fonte: Elaboração própria (2026).

D.3 Teste 3: neurônio

Código 47 — Programa do Teste 3

```
class Neuronio {
    var input_val = 1;
    var output_desejado = 0;
    var peso = 0.5;
    var bias = 1;
    var peso_bias = 0.5;
    var taxa_aprendizado = 0.01;
    var erro = 999;
    var iteracao = 0;

    func ativacao(soma) {
        if (soma > 0) {
            return 1;
        }
        return 0;
    }

    func treinar() {
        print("==== Teste 3: neuronio simples com classes e objetos ====");
        print("Entrada:", this.input_val, "Saida desejada:", this.output_desejado);

        while (this.erro != 0) {
            this.iteracao = this.iteracao + 1;

```

```
            var soma = (this.input_val * this.peso) + (this.bias * this.peso_bias);
            var saida = this.ativacao(soma);
            this.erro = this.output_desejado - saida;

            print("Iteracao:", this.iteracao);
            print("Soma ponderada:", soma);
            print("Saida:", saida);
            print("Erro:", this.erro);
            print("Peso entrada:", this.peso);
            print("Peso bias:", this.peso_bias);

            if (this.erro != 0) {
                this.peso = this.peso + ((this.taxa_aprendizado * this.input_val) * this.erro);
                this.peso_bias = this.peso_bias + ((this.taxa_aprendizado * this.bias) *
                ↪  this.erro);
            }
        }

        print("Neuronio aprendeu dinamicamente.");
        print("Iteracoes totais:", this.iteracao);
        print("Peso final:", this.peso);
        print("Peso bias final:", this.peso_bias);
        print("Valor desejado:", this.output_desejado);
    }
}

seq {
    var neuronio = new Neuronio();
    neuronio.treinar();
}
```

Fonte: Elaboração própria (2026).

D.4 Teste 4: rede neural para a função XOR

Código 48 — Programa do Teste 4

```
func sigmoid(x) {
    return 1 / (1 + exp(0 - x));
}

func derivada_sigmoid(saida) {
    return saida * (1 - saida);
}

class RedeNeuralXOR {
    var taxa_aprendizado = 0.5;

    var h1_w1 = 0.2;
    var h1_w2 = -0.4;
    var h1_bias = 0.0;

    var h2_w1 = -0.3;
    var h2_w2 = 0.6;
    var h2_bias = 0.1;

    var h3_w1 = 0.7;
    var h3_w2 = 0.5;
    var h3_bias = -0.2;

    var out_w1 = 0.4;
    var out_w2 = -0.5;
    var out_w3 = 0.3;
```

```
    var out_bias = 0.0;

    var h1 = 0.0;
    var h2 = 0.0;
    var h3 = 0.0;
    var saida = 0.0;

    func feedforward(x1, x2) {
        this.h1 = sigmoid((x1 * this.h1_w1) + (x2 * this.h1_w2) + this.h1_bias);
        this.h2 = sigmoid((x1 * this.h2_w1) + (x2 * this.h2_w2) + this.h2_bias);
        this.h3 = sigmoid((x1 * this.h3_w1) + (x2 * this.h3_w2) + this.h3_bias);

        this.saida = sigmoid(
            (this.h1 * this.out_w1) +
            (this.h2 * this.out_w2) +
            (this.h3 * this.out_w3) +
            this.out_bias
        );

        return this.saida;
    }

    func treinar_amostra(x1, x2, desejado) {
        var previsto = this.feedforward(x1, x2);
        var erro = desejado - previsto;
        var delta_saida = erro * derivada_sigmoid(previsto);

        var antigo_out_w1 = this.out_w1;
        var antigo_out_w2 = this.out_w2;
        var antigo_out_w3 = this.out_w3;

        this.out_w1 = this.out_w1 + ((this.h1 * delta_saida) * this.taxa_aprendizado);
        this.out_w2 = this.out_w2 + ((this.h2 * delta_saida) * this.taxa_aprendizado);
        this.out_w3 = this.out_w3 + ((this.h3 * delta_saida) * this.taxa_aprendizado);
        this.out_bias = this.out_bias + (delta_saida * this.taxa_aprendizado);

        var delta_h1 = (delta_saida * antigo_out_w1) * derivada_sigmoid(this.h1);
        var delta_h2 = (delta_saida * antigo_out_w2) * derivada_sigmoid(this.h2);
        var delta_h3 = (delta_saida * antigo_out_w3) * derivada_sigmoid(this.h3);

        this.h1_w1 = this.h1_w1 + ((x1 * delta_h1) * this.taxa_aprendizado);
        this.h1_w2 = this.h1_w2 + ((x2 * delta_h1) * this.taxa_aprendizado);
        this.h1_bias = this.h1_bias + (delta_h1 * this.taxa_aprendizado);

        this.h2_w1 = this.h2_w1 + ((x1 * delta_h2) * this.taxa_aprendizado);
        this.h2_w2 = this.h2_w2 + ((x2 * delta_h2) * this.taxa_aprendizado);
        this.h2_bias = this.h2_bias + (delta_h2 * this.taxa_aprendizado);

        this.h3_w1 = this.h3_w1 + ((x1 * delta_h3) * this.taxa_aprendizado);
        this.h3_w2 = this.h3_w2 + ((x2 * delta_h3) * this.taxa_aprendizado);
        this.h3_bias = this.h3_bias + (delta_h3 * this.taxa_aprendizado);
    }

    func treinar() {
        var epoca = 0;

        while (epoca < 20000) {
            this.treinar_amostra(0, 0, 0);
            this.treinar_amostra(0, 1, 1);
            this.treinar_amostra(1, 0, 1);
            this.treinar_amostra(1, 1, 0);
            epoca = epoca + 1;
        }
    }

    func testar() {
        print("==== Teste 4: rede neural XOR com backpropagation ====");
        print("Input: [0, 0], Predicted Output:", this.feedforward(0, 0));
        print("Input: [0, 1], Predicted Output:", this.feedforward(0, 1));
        print("Input: [1, 0], Predicted Output:", this.feedforward(1, 0));
```

```
        print("Input: [1, 1], Predicted Output:", this.feedforward(1, 1));
    }
}

seq {
    var rede = new RedeNeuralXOR();
    rede.treinar();
    rede.testar();
}
```

Fonte: Elaboração própria (2026).

D.5 Teste 5: sistema de recomendação

Código 49 — Programa do Teste 5

```
func relu(x) {
    if (x > 0) {
        return x;
    }
    return 0.0;
}

func sigmoid(x) {
    return 1 / (1 + exp(0 - x));
}

func peso_entrada_oculta(produto, neuronio) {
    var base = ((produto + 1) * (neuronio + 2)) % 7;
    return (base + 1) / 20.0;
}

func bias_oculto(neuronio) {
    return (neuronio % 3) / 20.0;
}

func peso_oculta_saida(neuronio, produto) {
    var base = ((neuronio + 3) * (produto + 1)) % 5;
    return (base + 1) / 25.0;
}

func bias_saida(produto, historico) {
    if (historico[produto] == 1) {
        return -0.25;
    }
    return -0.10;
}

func nome_produto(indice) {
    if (indice == 0) {
        return "Smartphone";
    }
    if (indice == 1) {
        return "Laptop";
    }
    if (indice == 2) {
        return "Tablet";
    }
    if (indice == 3) {
        return "Fones de ouvido";
    }
    if (indice == 4) {
        return "Camisa";
```

```
    }
    if (indice == 5) {
        return "Jeans";
    }
    if (indice == 6) {
        return "Jaqueta";
    }
    if (indice == 7) {
        return "Sapatos";
    }
    if (indice == 8) {
        return "Geladeira";
    }
    if (indice == 9) {
        return "Micro-ondas";
    }
    if (indice == 10) {
        return "Maquina de lavar";
    }
    if (indice == 11) {
        return "Ar condicionado";
    }
    if (indice == 12) {
        return "Ficcao";
    }
    if (indice == 13) {
        return "Nao-ficcao";
    }
    if (indice == 14) {
        return "Ficcao cientifica";
    }
    return "Fantasia";
}

func ler_historico_compras() {
    # Vetor binário: 1 = comprou, 0 = não comprou (16 produtos do catálogo)
    var historico = [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0];
    print("==== Teste 5: recomendacao e-commerce com rede neural ====");
    print("Informe o historico de compras (1 = comprou, 0 = nao comprou):");
    var i = 0;
    while (i < 16) {
        print(nome_produto(i), "- comprou? (0/1):");
        historico[i] = to_number(input(""));
        i = i + 1;
    }
    return historico;
}

class RedeNeuralRecomendacao {
    var input_size = 16;
    var hidden_size = 10;   # camada oculta com ReLU
    var output_size = 16;   # uma saída sigmoid por produto

    func forward(historico) {
        var ocultas = [0, 0, 0, 0, 0, 0, 0, 0, 0, 0];
        var saidas = [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0];

        # Camada oculta: entrada × pesos + bias → ReLU
        var h = 0;
        while (h < this.hidden_size) {
            var soma_oculta = 0.0;
            var entrada = 0;

            while (entrada < this.input_size) {
                soma_oculta = soma_oculta + (historico[entrada] * peso_entrada_oculta(entrada,
                ↪  h));
                entrada = entrada + 1;
            }

            ocultas[h] = relu(soma_oculta + bias_oculto(h));
```

```
            h = h + 1;
        }

        var produto = 0;
        # Camada de saída: ocultas × pesos + bias → sigmoid (probabilidade 0–1)
        while (produto < this.output_size) {
            var soma_saida = 0.0;
            var neuronio = 0;

            while (neuronio < this.hidden_size) {
                soma_saida = soma_saida + (ocultas[neuronio] * peso_oculta_saida(neuronio,
                ↪  produto));
                neuronio = neuronio + 1;
            }

            saidas[produto] = sigmoid(soma_saida + bias_saida(produto, historico));
            produto = produto + 1;
        }

        return saidas;
    }
}

class Recomendador {
    func recomendar(historico) {
        var rede = new RedeNeuralRecomendacao();
        var probabilidades = rede.forward(historico);

        print("Produtos recomendados para voce:");

        var produto = 0;
        while (produto < 16) {
            # Recomenda apenas produtos não comprados com probabilidade > 0.55
            if ((historico[produto] == 0) && (probabilidades[produto] > 0.55)) {
                print(nome_produto(produto), "probabilidade:", probabilidades[produto]);
            }
            produto = produto + 1;
        }
    }
}

seq {
    var historico = ler_historico_compras();
    var recomendador = new Recomendador();
    recomendador.recomendar(historico);
}
```

Fonte: Elaboração própria (2026).

D.6 Teste 6: Quicksort com classes

Código 50 — Programa do Teste 6

```
class Quicksort {
    var array = [0];

    func carregar(arr) {
        this.array = arr;
    }

    func trocar(i, j) {
        var temp = this.array[i];
        this.array[i] = this.array[j];
```

```
        this.array[j] = temp;
    }

    func particionar(inicio, fim) {
        var pivot = this.array[fim];
        var i = inicio - 1;
        var j = inicio;
        while (j < fim) {
            if (this.array[j] <= pivot) {
                i = i + 1;
                this.trocar(i, j);
            }
            j = j + 1;
        }
        i = i + 1;
        this.trocar(i, fim);
        return i;
    }

    func quicksort(inicio, fim) {
        if (inicio < fim) {
            var p = this.particionar(inicio, fim);
            this.quicksort(inicio, p - 1);
            this.quicksort(p + 1, fim);
        }
    }

    func ordenar() {
        var n = len(this.array) - 1;
        if (n >= 0) {
            this.quicksort(0, n);
        }
        return this.array;
    }
}

seq {
    print("==== Ordenacao com Quicksort ====");
    print("Insira os elementos do vetor separados por espaco:");
    var entrada = input("");
    var vetor = parse_ints(entrada);
    print("Vetor original:", vetor);
    var algoritmo = new Quicksort();
    algoritmo.carregar(vetor);
    var vetor_ordenado = algoritmo.ordenar();
    print("Vetor ordenado:", vetor_ordenado);
}
```

Fonte: Elaboração própria (2026).

D.7 Teste 7: Quicksort iterativo

Código 51 — Programa do Teste 7

```
func quicksort_iterativo(arr) {
    var stack = [];
    if (len(arr) > 1) {
        stack.append(0);
        stack.append(len(arr) - 1);
    }
    while (len(stack) > 0) {
        var fim = stack.pop();
        var inicio = stack.pop();
```

```
        if (inicio >= fim) {
            continue;
        }
        var pivot = arr[fim];
        var i = inicio - 1;
        var j = inicio;
        while (j < fim) {
            if (arr[j] <= pivot) {
                i = i + 1;
                var tmp = arr[i];
                arr[i] = arr[j];
                arr[j] = tmp;
            }
            j = j + 1;
        }
        i = i + 1;
        var tmp2 = arr[i];
        arr[i] = arr[fim];
        arr[fim] = tmp2;
        stack.append(inicio);
        stack.append(i - 1);
        stack.append(i + 1);
        stack.append(fim);
    }
    return arr;
}

seq {
    var array = [33, 12, 98, 5, 61];
    var ordenado = quicksort_iterativo(array);
    print("Array ordenado:", ordenado);
}
```

Fonte: Elaboração própria (2026).

D.8 Teste 8: avaliação com quatro avaliadores

Código 52 — Programa do Teste 8

```
# Teste 8 (MiniPar 2026.2) - Correcao de 160 atividades por 4 avaliadores.
# 40 alunos x 4 atividades = 160 atividades (id 0..159; aluno = id/4, atividade = id%4).
# Avaliador k (0..3) recebe a fatia [40k, 40k+40). Modo "par": 4 fork antes de qualquer join.
# Entrada (stdin, uma linha cada): modo (seq|par), carga, destino_0..destino_3.

# ---- Nucleo canonico compartilhado com T09/T10 (nao alterar nomes/formulas) ----
func lcg(s) { return (s * 75 + 74) % 65537; }
func nota_de(id, carga) {
    var s = lcg(id + 1);
    var passos = carga + s % carga;
    var k = 0;
    while (k < passos) { s = lcg(s); k = k + 1; }
    return s % 101;
}
func avaliar_lote(avaliador, inicio, fim, carga) {
    var r = [];
    r.append(avaliador);
    var i = inicio;
    while (i < fim) { r.append(i); r.append(nota_de(i, carga)); i = i + 1; }
    return r;
}
# ---- fim do nucleo canonico ----

# Converte texto decimal (so digitos) em inteiro. to_number devolve decimal e o
```

```
# backend CORE-07 nao mistura inteiro com decimal em '%'; por isso o parse manual.
func digito(c) {
    if (c == "0") { return 0; }
    if (c == "1") { return 1; }
    if (c == "2") { return 2; }
    if (c == "3") { return 3; }
    if (c == "4") { return 4; }
    if (c == "5") { return 5; }
    if (c == "6") { return 6; }
    if (c == "7") { return 7; }
    if (c == "8") { return 8; }
    if (c == "9") { return 9; }
    return -1;
}
func para_inteiro(t) {
    var n = 0;
    var i = 0;
    while (i < len(t)) {
        var d = digito(substr(t, i, 1));
        if (d >= 0) { n = n * 10 + d; }
        i = i + 1;
    }
    return n;
}

func nome_avaliador(k) {
    if (k == 0) { return "Professor"; }
    if (k == 1) { return "Monitor 1"; }
    if (k == 2) { return "Monitor 2"; }
    return "Monitor 3";
}

var modo = input();
var carga = para_inteiro(input());
var d0 = input();
var d1 = input();
var d2 = input();
var d3 = input();

var total_atividades = 160;
var num_avaliadores = 4;
var fatia = total_atividades / num_avaliadores;

print("=== TESTE 8: 160 avaliacoes, 4 avaliadores ===");
print("MODO " + modo);

var resultados = [];
if (modo == "par") {
    # Dispara as 4 tarefas antes de qualquer join.
    var hs = [];
    hs.append(fork avaliar_lote(0, 0 * fatia, 0 * fatia + fatia, carga) on d0);
    hs.append(fork avaliar_lote(1, 1 * fatia, 1 * fatia + fatia, carga) on d1);
    hs.append(fork avaliar_lote(2, 2 * fatia, 2 * fatia + fatia, carga) on d2);
    hs.append(fork avaliar_lote(3, 3 * fatia, 3 * fatia + fatia, carga) on d3);
    for (var h in hs) { resultados.append(join(h)); }
} else {
    var k = 0;
    while (k < num_avaliadores) {
        resultados.append(avaliar_lote(k, k * fatia, k * fatia + fatia, carga));
        k = k + 1;
    }
}

# Agregacao: contagem por id (vistos[id]), detecta perda e duplicata.
# AVALIADOR imprime o primeiro elemento devolvido pela tarefa (deve ser k).
var vistos = [];
var z = 0;
while (z < total_atividades) { vistos.append(0); z = z + 1; }

var total = 0;
```

```
var soma_notas = 0;
var k2 = 0;
while (k2 < num_avaliadores) {
    var r = resultados[k2];
    var qtd = (len(r) - 1) / 2;
    var j = 1;
    while (j < len(r)) {
        # Listas numericas sao vector<double> no backend; a atribuicao a uma
        # variavel declarada como inteiro converte o valor (ids e notas sao exatos).
        var id = 0;
        id = r[j];
        var nota = 0;
        nota = r[j + 1];
        # id fora de [0,160) nao entra em vistos: TOTAL passa a diferir de UNICOS.
        if (id >= 0 && id < total_atividades) { vistos[id] = vistos[id] + 1; }
        soma_notas = soma_notas + nota;
        total = total + 1;
        # Uma linha por atividade (como os 160 textos devolvidos na referencia Python);
        # o harness confere id, aluno, atividade e nota de cada uma.
        print("[" + nome_avaliador(k2) + "] Concluiu: Aluno " + to_string(id / 4 + 1) + " -
        ↪  Atividade " + to_string(id % 4 + 1) + " (id " + to_string(id) + ", nota " +
        ↪  to_string(nota) + ")");
        j = j + 2;
    }
    print("AVALIADOR " + to_string(r[0]) + " " + nome_avaliador(k2) + " " + to_string(qtd));
    k2 = k2 + 1;
}

var unicos = 0;
var duplicados = 0;
var perdidos = 0;
var w = 0;
while (w < total_atividades) {
    if (vistos[w] == 0) { perdidos = perdidos + 1; }
    if (vistos[w] >= 1) { unicos = unicos + 1; }
    if (vistos[w] > 1) { duplicados = duplicados + vistos[w] - 1; }
    w = w + 1;
}

print("TOTAL " + to_string(total));
print("UNICOS " + to_string(unicos));
print("DUPLICADOS " + to_string(duplicados));
print("PERDIDOS " + to_string(perdidos));
print("SOMA_NOTAS " + to_string(soma_notas));
```

Fonte: Elaboração própria (2026).

D.9 Teste 9: avaliações e fractal de Mandelbrot

Código 53 — Programa do Teste 9

```
# Teste 9 (MiniPar 2026.2, card T09-01): 160 avaliacoes + Mandelbrot ASCII.
# Amplia o Teste 8 com um quinto trabalhador logico que calcula o fractal.
#
# Entrada (uma linha por valor): modo (seq|par), carga, largura, altura, max_iter
# e 5 destinos (4 avaliadores + mandelbrot; lidos e ignorados no modo seq).
# Destino "local" executa no proprio processo; "host:porta" envia a um worker.
#
# Restricoes do backend CORE-07 respeitadas: literais inteiros cabem em int,
# sem listas de strings, funcoes de tarefa sem globais nem recursao.

# ---------------- Nucleo canonico do Teste 8 (copia exata) ----------------
func lcg(s) { return (s * 75 + 74) % 65537; }
```

```
func nota_de(id, carga) {
    var s = lcg(id + 1);
    var passos = carga + s % carga;
    var k = 0;
    while (k < passos) { s = lcg(s); k = k + 1; }
    return s % 101;
}
func avaliar_lote(avaliador, inicio, fim, carga) {
    var r = [];
    r.append(avaliador);
    var i = inicio;
    while (i < fim) { r.append(i); r.append(nota_de(i, carga)); i = i + 1; }
    return r;
}

# ---------------- Quinto trabalhador: Mandelbrot ASCII ----------------
# Mesma formula da referencia Python (trabalhador_mandelbrot). O "* 1.0" forca
# divisao em ponto flutuante: entre dois int, "/" gera divisao inteira no C++.
# Indice do caractere sem floor: (it*8 - (it*8) % max_iter) / max_iter, em
# aritmetica inteira, igual a floor(it*8/max_iter).
func mandelbrot(largura, altura, max_iter) {
    var caracteres = " .:-=+#%@";
    var texto = "";
    var y = 0;
    while (y < altura) {
        var cy = (y * 1.0 / altura) * 2.4 - 1.2;
        var linha = "";
        var x = 0;
        while (x < largura) {
            var cx = (x * 1.0 / largura) * 3.0 - 2.0;
            var zx = 0.0;
            var zy = 0.0;
            var it = 0;
            while (zx * zx + zy * zy <= 4.0 && it < max_iter) {
                var tmp = zx * zx - zy * zy + cx;
                zy = 2.0 * zx * zy + cy;
                zx = tmp;
                it = it + 1;
            }
            var k = (it * 8 - (it * 8) % max_iter) / max_iter;
            linha = linha + substr(caracteres, k, 1);
            x = x + 1;
        }
        if (y > 0) { texto = texto + "\n"; }
        texto = texto + linha;
        y = y + 1;
    }
    return texto;
}

# ---------------- Auxiliares do coordenador ----------------
func nome_avaliador(k) {
    if (k == 0) { return "Professor"; }
    if (k == 1) { return "Monitor 1"; }
    if (k == 2) { return "Monitor 2"; }
    return "Monitor 3";
}

# Converte uma linha com digitos decimais em int (to_number devolve double,
# e "%" exige int no backend). Caractere invalido aborta com mensagem.
func para_inteiro(s) {
    var digitos = "0123456789";
    var n = 0;
    var i = 0;
    var tam = len(s);
    if (tam == 0) {
        print("ERRO entrada numerica vazia");
        n = 0 - 1;
    }
    while (i < tam && n >= 0) {
```

```
        var c = substr(s, i, 1);
        var d = 0;
        var achou = 0;
        while (d < 10 && achou == 0) {
            if (substr(digitos, d, 1) == c) { achou = 1; } else { d = d + 1; }
        }
        if (achou == 0) {
            print("ERRO entrada numerica invalida: " + s);
            n = 0 - 1;
        } else {
            n = n * 10 + d;
            i = i + 1;
        }
    }
    return n;
}

# ---------------- Programa principal ----------------
var modo = input();
var carga = para_inteiro(input());
var largura = para_inteiro(input());
var altura = para_inteiro(input());
var max_iter = para_inteiro(input());
var d0 = input();
var d1 = input();
var d2 = input();
var d3 = input();
var d4 = input();

var lote0 = [];
var lote1 = [];
var lote2 = [];
var lote3 = [];
var fractal = "";

if (modo == "par") {
    # Os 5 forks saem antes do primeiro join: as 5 tarefas ficam simultaneas.
    var h0 = fork avaliar_lote(0, 0, 40, carga) on d0;
    var h1 = fork avaliar_lote(1, 40, 80, carga) on d1;
    var h2 = fork avaliar_lote(2, 80, 120, carga) on d2;
    var h3 = fork avaliar_lote(3, 120, 160, carga) on d3;
    var hm = fork mandelbrot(largura, altura, max_iter) on d4;
    lote0 = join(h0);
    lote1 = join(h1);
    lote2 = join(h2);
    lote3 = join(h3);
    fractal = join(hm);
} else {
    lote0 = avaliar_lote(0, 0, 40, carga);
    lote1 = avaliar_lote(1, 40, 80, carga);
    lote2 = avaliar_lote(2, 80, 120, carga);
    lote3 = avaliar_lote(3, 120, 160, carga);
    fractal = mandelbrot(largura, altura, max_iter);
}

print("=== TESTE 9: 160 avaliacoes + Mandelbrot ===");
print("MODO " + modo);

# Agregacao: vistos[id] conta quantas vezes cada atividade voltou.
var vistos = [];
var v = 0;
while (v < 160) { vistos.append(0); v = v + 1; }
var total = 0;
var soma = 0;
var lotes = [lote0, lote1, lote2, lote3];
var k = 0;
while (k < 4) {
    var lote = lotes[k];
    var qtd = 0;
    if (lote[0] != k) { print("ERRO lote " + to_string(k) + " com avaliador trocado"); }
```

```
    var p = 1;
    while (p + 1 < len(lote)) {
        var id = lote[p];
        if (id >= 0 && id < 160) { vistos[id] = vistos[id] + 1; }
        soma = soma + lote[p + 1];
        qtd = qtd + 1;
        p = p + 2;
    }
    total = total + qtd;
    print("AVALIADOR " + to_string(k) + " " + nome_avaliador(k) + " " + to_string(qtd));
    k = k + 1;
}

var unicos = 0;
var duplicados = 0;
var perdidos = 0;
var a = 0;
while (a < 160) {
    if (vistos[a] == 0) { perdidos = perdidos + 1; }
    if (vistos[a] >= 1) { unicos = unicos + 1; }
    if (vistos[a] > 1) { duplicados = duplicados + vistos[a] - 1; }
    a = a + 1;
}
print("TOTAL " + to_string(total));
print("UNICOS " + to_string(unicos));
print("DUPLICADOS " + to_string(duplicados));
print("PERDIDOS " + to_string(perdidos));
print("SOMA_NOTAS " + to_string(soma));
print("=== MANDELBROT " + to_string(largura) + "x" + to_string(altura) + " max_iter=" +
↪  to_string(max_iter) + " ===");
print(fractal);
print("=== FIM MANDELBROT ===");
```

Fonte: Elaboração própria (2026).

D.10 Teste 10: avaliações e dois fractais

Código 54 — Programa do Teste 10

```
# Teste 10 (MiniPar 2026.2): 160 avaliacoes + Mandelbrot + Sierpinski.
# Seis trabalhos: 4 avaliadores (40 atividades cada), Mandelbrot ASCII e
# Tapete de Sierpinski ASCII. No modo "par" os 6 fork sao emitidos antes do
# primeiro join; no modo "seq" a MESMA carga completa roda em chamadas diretas.
#
# Entrada (uma por linha): modo (seq|par), carga, largura, altura, max_iter,
# grau, e 6 destinos (4 avaliadores, mandelbrot, sierpinski; ignorados em seq).
# Destino "local" = thread do proprio processo; "host:porta" = worker remoto.
#
# Restricoes do backend CORE-07 respeitadas: tarefas de fork nao usam globais
# nem recursao (o Sierpinski e iterativo); IDs numericos (sem listas de strings).

# ---------- Nucleo canonico das avaliacoes (identico ao T08/T09) ----------
func lcg(s) { return (s * 75 + 74) % 65537; }

func nota_de(id, carga) {
    var s = lcg(id + 1);
    var passos = carga + s % carga;
    var k = 0;
    while (k < passos) { s = lcg(s); k = k + 1; }
    return s % 101;
}

func avaliar_lote(avaliador, inicio, fim, carga) {
```

```
    var r = [];
    r.append(avaliador);
    var i = inicio;
    while (i < fim) { r.append(i); r.append(nota_de(i, carga)); i = i + 1; }
    return r;
}

func nome_avaliador(k) {
    if (k == 0) { return "Professor"; }
    if (k == 1) { return "Monitor 1"; }
    if (k == 2) { return "Monitor 2"; }
    return "Monitor 3";
}

# ---------- Quinto trabalho: Mandelbrot ASCII ----------
func mandelbrot(largura, altura, max_iter) {
    var paleta = " .:-=+#%@";
    var texto = "";
    var y = 0;
    while (y < altura) {
        var linha = "";
        var cy = (y * 1.0 / altura) * 2.4 - 1.2;
        var x = 0;
        while (x < largura) {
            var cx = (x * 1.0 / largura) * 3.0 - 2.0;
            var zx = 0.0;
            var zy = 0.0;
            var it = 0;
            while (zx * zx + zy * zy <= 4.0 && it < max_iter) {
                var tmp = zx * zx - zy * zy + cx;
                zy = 2.0 * zx * zy + cy;
                zx = tmp;
                it = it + 1;
            }
            var k = (it * 8 - (it * 8) % max_iter) / max_iter;
            linha = linha + substr(paleta, k, 1);
            x = x + 1;
        }
        if (y > 0) { texto = texto + "\n"; }
        texto = texto + linha;
        y = y + 1;
    }
    return texto;
}

# ---------- Sexto trabalho: Tapete de Sierpinski (iterativo) ----------
# Celula (i, j) e vazia se, em algum nivel da base 3, os dois digitos valem 1.
func sierpinski(grau) {
    var n = 1;
    var g = 0;
    while (g < grau) { n = n * 3; g = g + 1; }
    var texto = "";
    var i = 0;
    while (i < n) {
        var linha = "";
        var j = 0;
        while (j < n) {
            var a = i;
            var b = j;
            var vazio = 0;
            while (a > 0 || b > 0) {
                if (a % 3 == 1 && b % 3 == 1) { vazio = 1; }
                a = (a - a % 3) / 3;
                b = (b - b % 3) / 3;
            }
            if (vazio == 1) { linha = linha + " "; } else { linha = linha + "#"; }
            j = j + 1;
        }
        if (i > 0) { texto = texto + "\n"; }
        texto = texto + linha;
```

```
        i = i + 1;
    }
    return texto;
}

# ---------- Leitura de inteiros ----------
# to_number devolve decimal e o backend CORE-07 nao mistura int com decimal;
# por isso os parametros inteiros sao lidos digito a digito (ignora o resto).
func digito(c) {
    if (c == "0") { return 0; }
    if (c == "1") { return 1; }
    if (c == "2") { return 2; }
    if (c == "3") { return 3; }
    if (c == "4") { return 4; }
    if (c == "5") { return 5; }
    if (c == "6") { return 6; }
    if (c == "7") { return 7; }
    if (c == "8") { return 8; }
    if (c == "9") { return 9; }
    return -1;
}

func para_inteiro(s) {
    var n = 0;
    var i = 0;
    while (i < len(s)) {
        var d = digito(substr(s, i, 1));
        if (d >= 0) { n = n * 10 + d; }
        i = i + 1;
    }
    return n;
}

# ---------- Fluxo principal ----------
var modo = input();
var carga = para_inteiro(input());
var largura = para_inteiro(input());
var altura = para_inteiro(input());
var max_iter = para_inteiro(input());
var grau = para_inteiro(input());
var d0 = input();
var d1 = input();
var d2 = input();
var d3 = input();
var dm = input();
var ds = input();

var lotes = [];
var texto_m = "";
var texto_s = "";

if (modo == "par") {
    # Os 6 trabalhos sao disparados antes do primeiro join.
    var hs = [];
    hs.append(fork avaliar_lote(0, 0, 40, carga) on d0);
    hs.append(fork avaliar_lote(1, 40, 80, carga) on d1);
    hs.append(fork avaliar_lote(2, 80, 120, carga) on d2);
    hs.append(fork avaliar_lote(3, 120, 160, carga) on d3);
    var hm = fork mandelbrot(largura, altura, max_iter) on dm;
    var hsp = fork sierpinski(grau) on ds;
    for (var h in hs) { lotes.append(join(h)); }
    texto_m = join(hm);
    texto_s = join(hsp);
} else {
    # Mesma carga completa, sem fork: 4 lotes + Mandelbrot + Sierpinski.
    lotes.append(avaliar_lote(0, 0, 40, carga));
    lotes.append(avaliar_lote(1, 40, 80, carga));
    lotes.append(avaliar_lote(2, 80, 120, carga));
    lotes.append(avaliar_lote(3, 120, 160, carga));
    texto_m = mandelbrot(largura, altura, max_iter);
```

```
    texto_s = sierpinski(grau);
}

print("=== TESTE 10: 160 avaliacoes + Mandelbrot + Sierpinski ===");
print("MODO", modo);

var vistos = [];
var v = 0;
while (v < 160) { vistos.append(0); v = v + 1; }

var total = 0;
var soma = 0;
var fora = 0;
for (var r in lotes) {
    var qtd = (len(r) - 1) / 2;
    print("AVALIADOR", r[0], nome_avaliador(r[0]), qtd);
    var p = 1;
    while (p < len(r)) {
        var id = r[p];
        if (id >= 0 && id < 160) { vistos[id] = vistos[id] + 1; } else { fora = fora + 1; }
        soma = soma + r[p + 1];
        total = total + 1;
        p = p + 2;
    }
}

var unicos = 0;
var duplicados = 0;
var perdidos = 0;
v = 0;
while (v < 160) {
    if (vistos[v] == 0) { perdidos = perdidos + 1; }
    if (vistos[v] >= 1) { unicos = unicos + 1; }
    if (vistos[v] > 1) { duplicados = duplicados + vistos[v] - 1; }
    v = v + 1;
}
duplicados = duplicados + fora;

print("TOTAL", total);
print("UNICOS", unicos);
print("DUPLICADOS", duplicados);
print("PERDIDOS", perdidos);
print("SOMA_NOTAS", soma);
print("=== MANDELBROT " + to_string(largura) + "x" + to_string(altura) + " max_iter=" +
↪  to_string(max_iter) + " ===");
print(texto_m);
print("=== FIM MANDELBROT ===");
print("=== SIERPINSKI grau=" + to_string(grau) + " ===");
print(texto_s);
print("=== FIM SIERPINSKI ===");
```

Fonte: Elaboração própria (2026).

D.11 Teste 11: MiniCompilador

Código 55 — Programa do Teste 11 (MiniCompilador)

```
# Teste 11 — MiniCompilador de expressões aritméticas escrito em MiniPar.
# Contrato: C-T11-v1 (docs/02_CONTRATOS_TECNICOS.md §3). Referência do professor:
# docs/fontes/teste11/MiniCompiladorCompleto_ARMv7.java.
#
# Listas de strings estão fora do recorte (C-T11-v1 §3.4): tokens e nós da AST
# ficam em listas numéricas paralelas e todo texto é recuperado da própria
```

```
# entrada com substr(e, inicio, tamanho).

class MiniCompilador {
    var e = "";
    var erro = "";

    # Tokens: tipo (0 = NUMERO, 1 = OPERADOR, 2 = PARENTESE), início e tamanho do lexema.
    var tipos = [];
    var inicios = [];
    var tamanhos = [];

    # Nós da AST: índice do token e filhos (-1 = nenhum).
    var ntok = [];
    var nesq = [];
    var ndir = [];
    var raiz = 0;

    # Estado do parser: token atual e último nó construído.
    var pos = 0;
    var ultimo = 0;

    # Tabela de símbolos: valor numérico e token da primeira ocorrência de cada constante.
    var svals = [];
    var stoks = [];

    var temp = 1;
    var reg = 0;

    # --- utilitários -----------------------------------------------------

    func lexema(t) {
        return substr(this.e, this.inicios[t], this.tamanhos[t]);
    }

    func atual() {
        if (this.pos < len(this.tipos)) {
            return this.lexema(this.pos);
        }
        return "";
    }

    func digito(c) {
        return c >= "0" && c <= "9";
    }

    func letra(c) {
        return (c >= "a" && c <= "z") || (c >= "A" && c <= "Z") || c == "_";
    }

    func falhar(fase, msg) {
        if (this.erro == "") {
            this.erro = "Erro " + fase + ": " + msg;
        }
    }

    func coluna(i) {
        return "coluna " + to_string(i + 1);
    }

    func nomeTipo(tipo) {
        if (tipo == 0) {
            return "NUMERO";
        }
        if (tipo == 1) {
            return "OPERADOR";
        }
        return "PARENTESE";
    }

    func adicionarToken(tipo, inicio, tamanho) {
```

```
        this.tipos.append(tipo);
        this.inicios.append(inicio);
        this.tamanhos.append(tamanho);
    }

    # --- análise léxica ---------------------------------------------------

    func analisarLexico() {
        var n = len(this.e);
        var i = 0;
        while (i < n && this.erro == "") {
            var c = substr(this.e, i, 1);
            if (c == " " || c == "\t") {
                i = i + 1;
            } else {
                if (c == "#") {
                    i = n;
                } else {
                    if (this.digito(c)) {
                        var ini = i;
                        while (i < n && this.digito(substr(this.e, i, 1))) {
                            i = i + 1;
                        }
                        var num = substr(this.e, ini, i - ini);
                        if (i < n && substr(this.e, i, 1) == ".") {
                            this.falhar("léxico", "número decimal não suportado na " +
                            ↪  this.coluna(ini));
                        } else {
                            if (len(num) > 1 && substr(num, 0, 1) == "0") {
                                this.falhar("léxico", "zero à esquerda não suportado em '" + num
                                ↪  + "' na " + this.coluna(ini));
                            } else {
                                # Maior constante que LDR rN, =k carrega como inteiro positivo de
                                ↪  32 bits.
                                if (len(num) > 10 || to_number(num) > 2147483647) {
                                    this.falhar("léxico", "constante '" + num + "' fora do
                                    ↪  intervalo 0..2147483647 na " + this.coluna(ini));
                                } else {
                                    this.adicionarToken(0, ini, i - ini);
                                }
                            }
                        }
                    } else {
                        if (c == "+" || c == "-" || c == "*") {
                            this.adicionarToken(1, i, 1);
                        } else {
                            if (c == "(" || c == ")") {
                                this.adicionarToken(2, i, 1);
                            } else {
                                this.rejeitarCaractere(c, i);
                            }
                        }
                        i = i + 1;
                    }
                }
            }
        }
        if (this.erro == "" && len(this.tipos) == 0) {
            this.falhar("léxico", "entrada vazia");
        }
    }

    func rejeitarCaractere(c, i) {
        if (c == "/") {
            this.falhar("léxico", "divisão '/' não suportada na " + this.coluna(i));
        } else {
            if (c == "=") {
                this.falhar("léxico", "atribuição '=' não suportada na " + this.coluna(i));
            } else {
                if (this.letra(c)) {
```

```
                    this.falhar("léxico", "identificador não suportado na " + this.coluna(i));
                } else {
                    if (c == ".") {
                        this.falhar("léxico", "número decimal não suportado na " +
                        ↪  this.coluna(i));
                    } else {
                        if (c > " " && c <= "~") {
                            this.falhar("léxico", "caractere desconhecido '" + c + "' na " +
                            ↪  this.coluna(i));
                        } else {
                            this.falhar("léxico", "caractere não suportado na " + this.coluna(i));
                        }
                    }
                }
            }
        }
    }

    # --- análise sintática (descida recursiva expr -> term -> factor) -----
    # Cada regra deixa o nó construído em this.ultimo.

    func novoNo(t, esq, dir) {
        this.ntok.append(t);
        this.nesq.append(esq);
        this.ndir.append(dir);
        this.ultimo = len(this.ntok) - 1;
    }

    func parseExpr() {
        this.parseTerm();
        var no = this.ultimo;
        while (this.erro == "" && (this.atual() == "+" || this.atual() == "-")) {
            var t = this.pos;
            this.pos = this.pos + 1;
            this.parseTerm();
            this.novoNo(t, no, this.ultimo);
            no = this.ultimo;
        }
        this.ultimo = no;
    }

    func parseTerm() {
        this.parseFactor();
        var no = this.ultimo;
        while (this.erro == "" && this.atual() == "*") {
            var t = this.pos;
            this.pos = this.pos + 1;
            this.parseFactor();
            this.novoNo(t, no, this.ultimo);
            no = this.ultimo;
        }
        this.ultimo = no;
    }

    func parseFactor() {
        if (this.erro == "") {
            if (this.pos >= len(this.tipos)) {
                this.falhar("sintático", "expressão incompleta: esperado número ou '(' no fim da
                ↪  entrada");
            } else {
                var t = this.pos;
                var lx = this.lexema(t);
                if (lx == "(") {
                    this.pos = this.pos + 1;
                    this.parseExpr();
                    if (this.erro == "" && this.atual() != ")") {
                        if (this.pos >= len(this.tipos)) {
                            this.falhar("sintático", "'(' da " + this.coluna(this.inicios[t]) + "
                            ↪  sem ')' correspondente");
                        } else {
```

```
                            this.falhar("sintático", "esperado ')' e encontrado '" + this.atual()
                            ↪  + "' na " + this.coluna(this.inicios[this.pos]));
                        }
                    }
                    this.pos = this.pos + 1;
                } else {
                    if (this.tipos[t] == 0) {
                        this.pos = this.pos + 1;
                        this.novoNo(t, 0 - 1, 0 - 1);
                    } else {
                        if (lx == "-") {
                            this.falhar("sintático", "menos unário não suportado na " +
                            ↪  this.coluna(this.inicios[t]));
                        } else {
                            this.falhar("sintático", "esperado número ou '(' e encontrado '" + lx
                            ↪  + "' na " + this.coluna(this.inicios[t]));
                        }
                    }
                }
            }
        }
    }

    func analisarSintatico() {
        this.pos = 0;
        this.parseExpr();
        this.raiz = this.ultimo;
        if (this.erro == "" && this.pos < len(this.tipos)) {
            this.falhar("sintático", "token '" + this.atual() + "' inesperado depois do fim da
            ↪  expressão, na " + this.coluna(this.inicios[this.pos]));
        }
    }

    # --- saídas -------------------------------------------------------------

    func imprimirTokens() {
        print("\n🔍 Tokens gerados pelo analisador léxico:");
        var t = 0;
        while (t < len(this.tipos)) {
            print("[" + this.nomeTipo(this.tipos[t]) + ": '" + this.lexema(t) + "']");
            t = t + 1;
        }
    }

    # Ordem simétrica invertida (direita, nó, esquerda), dois espaços por nível.
    func imprimirAST(no, nivel) {
        if (no >= 0) {
            this.imprimirAST(this.ndir[no], nivel + 1);
            var recuo = "";
            var k = 0;
            while (k < nivel) {
                recuo = recuo + "  ";
                k = k + 1;
            }
            print(recuo + this.lexema(this.ntok[no]));
            this.imprimirAST(this.nesq[no], nivel + 1);
        }
    }

    # Pós-ordem; devolve o endereço (lexema ou temporário) do nó.
    func gerarTAC(no) {
        if (this.nesq[no] < 0) {
            return this.lexema(this.ntok[no]);
        }
        var a1 = this.gerarTAC(this.nesq[no]);
        var a2 = this.gerarTAC(this.ndir[no]);
        var t = "t" + to_string(this.temp);
        this.temp = this.temp + 1;
        print(t + " = " + a1 + " " + this.lexema(this.ntok[no]) + " " + a2);
        return t;
```

```
    }

    # Expressão totalmente parentizada, usada nos comentários do Assembly.
    func formatExpr(no) {
        if (this.nesq[no] < 0) {
            return this.lexema(this.ntok[no]);
        }
        return "(" + this.formatExpr(this.nesq[no]) + " " + this.lexema(this.ntok[no]) + " " +
        ↪  this.formatExpr(this.ndir[no]) + ")";
    }

    # Mnemônico já alinhado no campo de 6 colunas da referência (%-6s).
    func mnemonico(op) {
        if (op == "+") {
            return "ADD   ";
        }
        if (op == "-") {
            return "SUB   ";
        }
        return "MUL   ";
    }

    # Pós-ordem; registradores por contador global, sem reaproveitamento.
    func gerarARM(no) {
        if (this.nesq[no] < 0) {
            var rf = "r" + to_string(this.reg);
            this.reg = this.reg + 1;
            var k = this.lexema(this.ntok[no]);
            if (to_number(k) > 255) {
                print("    LDR    " + rf + ", =" + k);
            } else {
                print("    MOV    " + rf + ", #" + k);
            }
            return rf;
        }
        var a1 = this.gerarARM(this.nesq[no]);
        var a2 = this.gerarARM(this.ndir[no]);
        var r = "r" + to_string(this.reg);
        this.reg = this.reg + 1;
        print("    " + this.mnemonico(this.lexema(this.ntok[no])) + " " + r + ", " + a1 + ", " +
        ↪  a2 + "    @ resultado de '" + this.formatExpr(no) + "'");
        return r;
    }

    # Constantes sem repetição, em ordem crescente de valor.
    func registrarSimbolos() {
        var no = 0;
        while (no < len(this.ntok)) {
            if (this.nesq[no] < 0) {
                var v = to_number(this.lexema(this.ntok[no]));
                var novo = true;
                var j = 0;
                while (j < len(this.svals)) {
                    if (this.svals[j] == v) {
                        novo = false;
                    }
                    j = j + 1;
                }
                if (novo) {
                    this.svals.append(v);
                    this.stoks.append(this.ntok[no]);
                    j = len(this.svals) - 1;
                    while (j > 0 && this.svals[j - 1] > this.svals[j]) {
                        var tv = this.svals[j];
                        this.svals[j] = this.svals[j - 1];
                        this.svals[j - 1] = tv;
                        var tt = this.stoks[j];
                        this.stoks[j] = this.stoks[j - 1];
                        this.stoks[j - 1] = tt;
                        j = j - 1;
```

```
                    }
                }
            }
            no = no + 1;
        }
    }

    func imprimirSimbolos() {
        var j = 0;
        while (j < len(this.stoks)) {
            print(" ◇  " + this.lexema(this.stoks[j]) + " → CONST");
            j = j + 1;
        }
    }

    # Valor de cada nó; o ARM calcula em registradores de 32 bits, então todo
    # resultado intermediário precisa caber em inteiro com sinal de 32 bits.
    func avaliar(no) {
        if (this.nesq[no] < 0) {
            return to_number(this.lexema(this.ntok[no]));
        }
        var a = this.avaliar(this.nesq[no]);
        var b = this.avaliar(this.ndir[no]);
        var op = this.lexema(this.ntok[no]);
        var v = a * b;
        if (op == "+") {
            v = a + b;
        }
        if (op == "-") {
            v = a - b;
        }
        if (v > 2147483647 || v < 0 - 2147483648) {
            this.falhar("semântico", "o resultado de '" + this.formatExpr(no) + "' não cabe em 32
            ↪  bits com sinal");
        }
        return v;
    }

    func analisarSemantico() {
        # Um registrador por nó, de r0 a r12; r13–r15 são SP, LR e PC.
        if (len(this.ntok) > 13) {
            this.falhar("semântico", "a AST tem " + to_string(len(this.ntok)) + " nós e o
            ↪  Assembly comporta no máximo 13 (registradores r0 a r12)");
        }
    }

    func compilar(entrada) {
        this.e = entrada;
        this.analisarLexico();
        if (this.erro == "") {
            this.imprimirTokens();
            this.analisarSintatico();
        }
        if (this.erro == "") {
            this.analisarSemantico();
        }
        if (this.erro == "") {
            this.gerarSaidas(this.avaliar(this.raiz));
        }
        if (this.erro != "") {
            print("\n❌ " + this.erro);
        }
    }

    # Só é chamada depois de todas as fases de análise aceitarem a entrada.
    func gerarSaidas(resultado) {
        if (this.erro == "") {
            this.registrarSimbolos();
            print("\n🌳 Árvore Sintática Abstrata (AST):");
            this.imprimirAST(this.raiz, 0);
```

```
411             print("\n📦 Código de Três Endereços (TAC):");
412             this.gerarTAC(this.raiz);
413             print("\n⚙ Assembly ARMv7 (CPUlator):");
414             print(".global _start");
415             print(".text");
416             print("_start:");
417             this.gerarARM(this.raiz);
418             print("    b .    @ loop infinito para parar o programa");
419             print("\n📑 Tabela de Símbolos:");
420             this.imprimirSimbolos();
421             print("\n🧮 Resultado final da expressão:");
422             print("➡  " + to_string(resultado) + ".0");
423         }
424     }
425 }
426 
427 var entrada = input("Digite uma expressão infixada: ");
428 var mc = new MiniCompilador();
429 mc.compilar(entrada);
```

Fonte: Elaboração própria (2026).